

Amazing Maze Program

Toby Davies

Contents

Analysis:	6
Overview:.....	6
Target Audience:.....	6
Research:	7
Existing Solutions:.....	7
Mazesolver.xyz:	7
Mazesforfun.com:.....	9
The End User:	11
End User Survey:	11
End User Interview:.....	11
Prototype:	13
Project:.....	13
Language:.....	13
Possible Challenges:	14
Objectives:.....	14
Summary:	15
Design:.....	16
Overview:.....	16
The User Interface:	16
Buttons!	18
FSM:	19
The Solving Algorithm:	20
ActiveNodes Class:	20
Back to the Solving Algorithm:.....	22
The Generation Algorithm:.....	24
Maze Rules:	24
OpenContainedSpace Function:	26
Maze Exploration:	28
Hierarchy Chart and Conclusion:.....	29
Technical Solution:.....	30
Completeness Of Solution:	30

Maze Matrix:.....	33
Matrix Numbers:	34
A Note on Indexing:	35
Code Context Feature:	35
Program Flow:	36
User Interface:	37
UpdateWindow Function:	37
Buttons:	41
Sliders!.....	43
__init__ Function:.....	43
HandleSlider Function:	44
DrawSlider Function:	45
Where in the Code are They?	45
Pop up Menus:	46
Generation Options Window:.....	46
Saved Maze Window:	47
Saved Mazes:	49
Button 1:.....	50
Button 2 (Writing to file):.....	51
Button 3 (Reading from file):	53
ActiveNodes Class:	56
The Solving Algorithm:	59
Finding the Maze Exit:	59
Visualisation of the Solving Algorithm:	61
Finding the Shortest Path:	62
Calling the Solving Algorithm:	63
The Generation Algorithm:.....	65
The Lesser Maze Rules:	67
EliminateTwoBlockWalls:	67
RemoveSinglePixels:.....	68
Interlude About Random Numbers:.....	69
EliminateDiagonals:.....	70

EliminateTwoBlockCorridors:	71
The OpenContainedSpace Function:	73
Shadow Maze:	73
ConnectedEntranceCheck:	75
Calling the Generation Algorithm:.....	78
Maze Exploration:	80
Group A Techniques Used:	83
Testing:.....	84
Evaluation:	87
Objective Completeness:.....	87
End User Feedback:.....	90
Conversation:	90
Reflection:.....	90
Improvements:	91
Improving the Generation Algorithm:	91
Variable Window Size:.....	93
Final Project Reflection:	94
Code Appendix:	95
Amazing Maze Generator:.....	95
Maze Generator Main:	95
RemoveSinglePixels:	96
EliminateTwoBlockWalls:.....	97
EliminateTwoBlockCorridors:	97
EliminateDiagonals:	98
OpenContainedSpace:	98
ConnectedEntranceCheck:	99
DrawMaze:	101
Maze Solver:.....	102
Maze Solver Main:	102
FindRoute:.....	104
FindStart:	104
GetWidth, GetHeight:	105

ActiveNode Class:.....	106
Maze Exploration:	107
Stack Class:	107
HandleUserExploration:	108
Slider Class:.....	110
User Interface:	112
DrawText:.....	112
GetWidthHeight:	112
UpdateWindow:	113
DrawGenerationOptionsWindow:.....	114
DrawSavedMazeWindow:	114
DrawButtons:.....	115
UpdateMaze:	116
CleanMaze:	116
Setup:.....	117
The Main Function:	118
Opening:	118
Generate Maze Button Pressed:	119
Solve Maze Button Pressed:	120
Explore Maze Button Pressed:	120
SavedMazeWindow is True:.....	121
Button 1 Pressed:	121
Button 2 Pressed:	122
Button 3 Pressed:	123
Handle Generation Options Window:	124
Handle Saved Maze Window (featuring fps Clock):	124
References:	125

Analysis:

Overview:

You could be forgiven for not knowing how all the pieces move in chess, or for never having actually completed a crossword puzzle (I know I always give up halfway through). But your friends would all stare at you in shock if you told them that you had never solved a maze. I mean, *have* you ever met somebody who has never solved a maze? Yeah, neither have I. They probably all got lost.

Mazes are absolutely one of the most well-known forms of puzzle, dating back thousands of years. One of the most famous mazes in the world, the Hampton court maze, was even created by order of King William III himself! Yet despite their age, humanity still seems infatuated with these puzzles, and they still present many challenges today, both in generating and solving them. However, I feel there is a shortage of good maze generators online.

Therefore, I would like to create a piece of software all about mazes! It will generate mazes let the user solve the maze and show them the shortest path through it, all packaged into an intuitive user interface.

Target Audience:

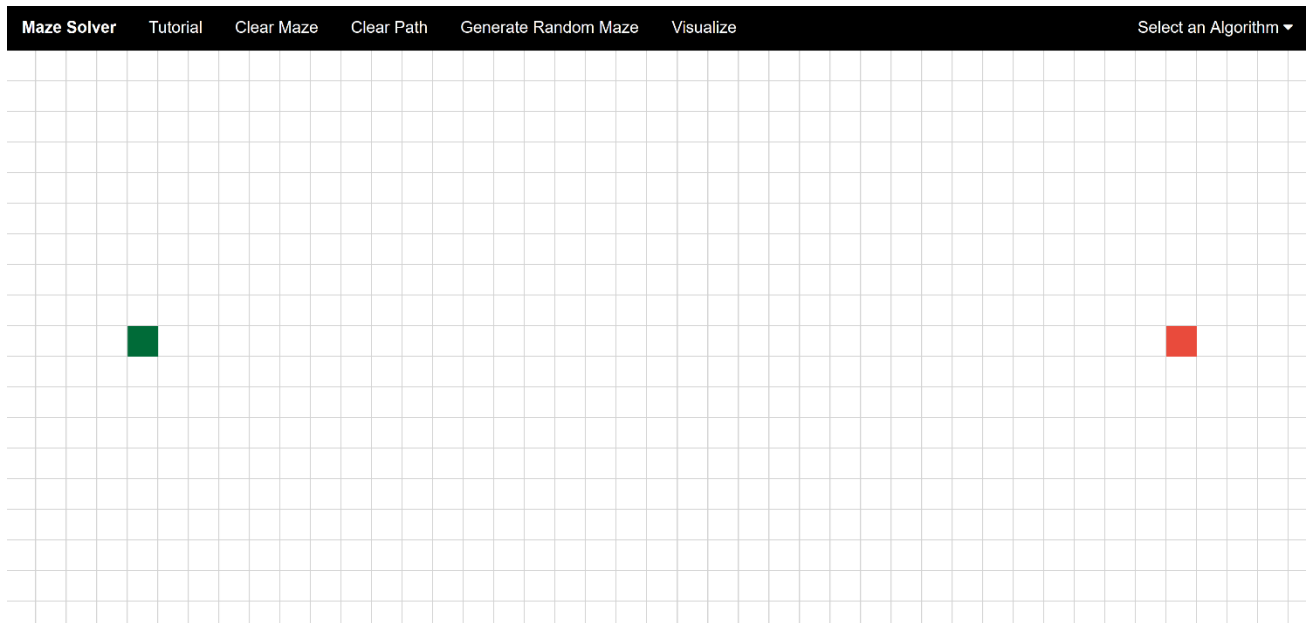
Mazes are for everyone! However, some people will enjoy solving mazes more than others. This software is designed for people who have finished all of the maze puzzle books and need a fresh supply of mazes to solve, matched up with their solutions.

Research:

Existing Solutions:

Mazesolver.xyz:

The first piece of existing software I want to show is [mazesolver](#) . When you load up the website, it looks like this:



I especially want to bring attention to this lovely menu bar at the top:

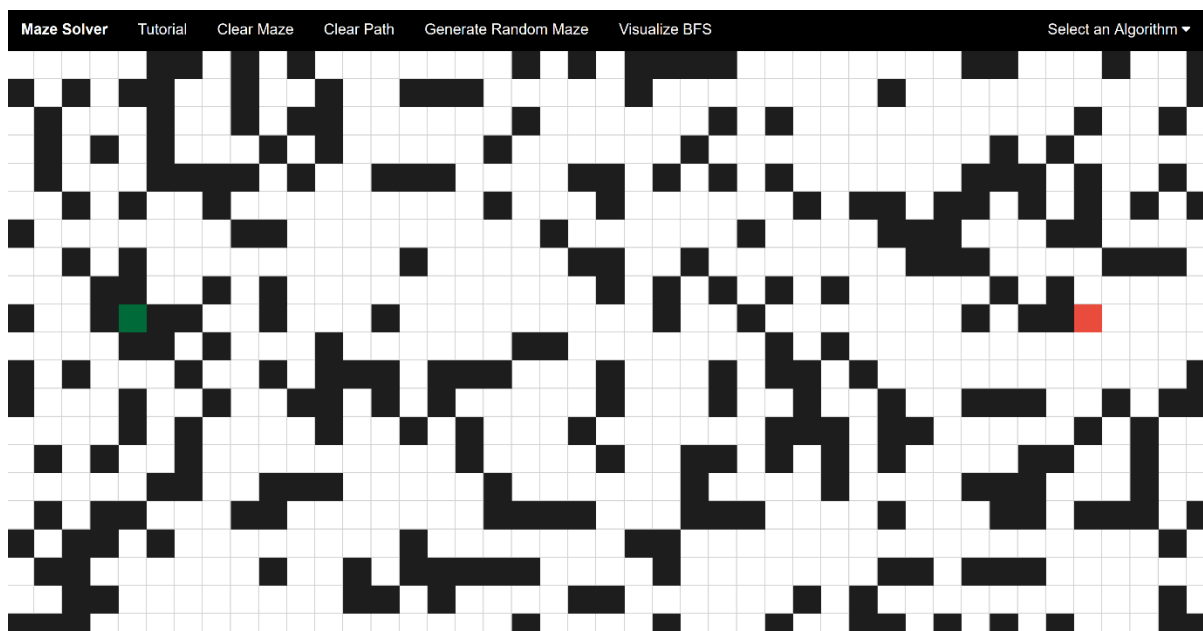


The simplicity of this menu is invaluable – it's so easy to tell what each button does! I definitely want my project to have a menu bar just like this one.

Now, as far as functionality goes, the website can generate a maze, solve the maze (using a variety of algorithms) and... that's it? This website doesn't actually allow the user to solve the maze themselves. But that's the best part!

Now let's try the generate random maze button...

Ah.



It seems the generation algorithm doesn't quite compare to the user interface. In fact, I'm not sure there actually is a generation algorithm at work here – it looks suspiciously like the website has just randomly set each pixel to black or white. It isn't even solvable; the green pixel marking the start is surrounded by wall! Even Microsoft word's auto-generated alt text thinks this 'maze' looks like a crossword puzzle.

So to conclude, [mazesolver](#):

- + Has a very intuitive user interface
- + Gives the user a choice of which solving algorithm to use
- The user cannot solve the generated maze themselves
- The maze generation algorithm is shocking

Mazesforfun.com:

[Mazesforfun](https://mazesforfun.com) looks like this:

MazesForFun



Maze Generator

Popular Mazes: Circle

Maze Generator

Create unlimited mazes of different sizes and levels to print with our Maze Puzzle Generator. Make mazes for kids and adults of all ages. Download as printable PDF, SVG or PNG image. You don't need to register or install anything on your computer. These printable maze puzzles are free to download and ready to print.

Settings

Shape: Square

Level: Hard

Cell Width: 20

Size (WxH): 20 20

Generate

☐ Show Solution

Download

PDF Letter Size

Download

Printable Maze Puzzles for Kids and Adults

Do you love mazes? What about making your own mazes? Now you can with our free maze puzzle maker! This online tool is perfect to create mazes of different sizes and complexities. Plus, there are a variety of different designs to choose from.

Teachers, parents, and educators can use this tool in the classroom, education center, or at home. It's also great for birthday parties, family game nights, or picnics.

So whether you're trying to challenge your kids or just want something fun to do on a rainy day, mazes are a great choice.

How to create your own maze puzzle?

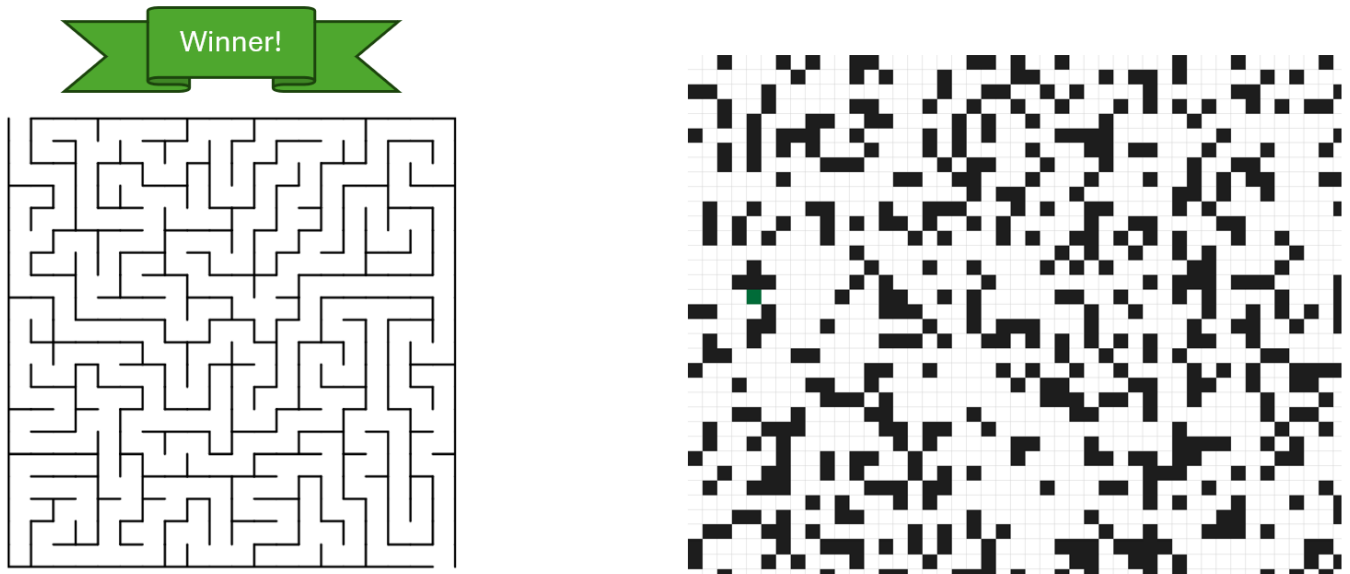
Create your own mazes easily with a few adjustments.

- **Shape:** Select the maze shape from the drop down menu. You have the options to make square, triangle, hexagonal, diamond, squaretriangles mazes and more.
- **Method:** Setting this option sets the difficulty level of your maze. You can select from very easy, easy, medium and hard.
- **Cell Width:** Adjust the cell width by selecting 10, 20 or 30. These sizes will make a smaller or larger maze.
- **Size:** How much width and height you wish to set, can be adjusted with this option.
- Click on the blue "Generate" button to generate your maze.
- To see the solution, check the "Solution" button.
- Download your custom maze by clicking on the green "Download" button.

This website isn't nearly as clean as [mazesolver](https://mazesolver.com) – in fact, all that the text at the bottom of the screen does is explain how to use the interface. This wouldn't even be necessary if the interface was more intuitive!

However, there is a little more functionality here, such as the ability to download the generated mazes and far more parameters around maze generation (although still no option for the user to solve the maze themselves).

Speaking of maze generation, the mazes generated by [mazesforfun](#) are far superior to [mazesolver](#):



(Although I don't like the look of the line walls as much...)

So to summarise, [mazesforfun](#):

- + Has a much better generation algorithm
- + Has more freedom in how mazes are generated (different sizes, shapes, etc)
- Uses line walls, which don't look as nice
- User interface is messier
- Still no option for the user to solve the maze themselves!

The End User:

As mazes are such a universal puzzle that almost everybody has experience with, I have given a short survey to a range of people to get a vague idea of the maze solving habits of the average person, as well as asking what sort of features would be looked for in the program. I will also interview a specific end user to get more detailed feedback.

End User Survey:

Out of the 12 people interviewed:

- 10/12 interviewed enjoy solving mazes
- Yet only 1/12 solves mazes regularly

When asked where they found mazes to solve, common answers were a newspaper or puzzle book – both of which are limited in the number of puzzles they offer. This does suggest that maze solving software would be popular – most of the people who took the survey love solving mazes but only have a limited supply of them. Enter, maze generating software!

The second part of the survey asked recipients what features that they'd like to see in a program such as mine. Common responses were, predictably, random generation and a solving system. Many people also wrote that the program should let the user solve the maze themselves, backing up the research I performed into current solutions.

One more interesting idea was the ability for the user to be able to draw their own custom route and generate a maze to fit it. This may not make it into the final project but could be interesting to look into in the future, as users could have the solution to their custom maze make a personalised picture or message – a very cool idea!

End User Interview:

My end user, a friend from school, has used the two websites I looked at in the current solutions section: [online maze solver](#) and [mazesforfun](#). Here you can see the (mostly unfiltered) interview I had with her.

Q: Do you enjoy solving mazes generally?

A: Who doesn't!

Q: What did you enjoy most about [online maze solver](#)?

A: Well, I liked how it was a bit more comprehensive? So, you could get it to generate a maze, or you could draw the maze yourself, and then it would solve it for you, using one of several solving methods. It was good to have that freedom. The user interface had some nice touches too. Like, when you were drawing the maze, the pixel would pulse as you placed it, and when you hover over a button it changes colour, that sort of thing.

Q: So, I'm hearing you liked that the user could create a maze themselves? I wasn't thinking of including that in my program, but should I?

A: Well, ok. In the case of this website, it's kind of necessary, as their generation bit is poor at best. In other cases, I guess it's a nice feature to have, but not super important. It takes a long time to actually make a maze from hand, and if you do, you already know the solution, so what's the point?

Q: What improvements would you make to [online maze solver](#)?

A: Like I've said, the generation was really bad. Aside from that, it's more difficult to think of any. I guess it would be nice to be able to choose the size of the maze? Right now, I think it just uses screen size, so if you zoom way out on your browser it makes a bigger maze. It would just be nice to have something like that without having to make the buttons so small.

Q: Alright, what did you like about [mazesforfun](#)?

A: This one was much freer in how you could generate mazes, so it does all sorts of sizes, as well as shapes too. It also lets you download the maze which I guess is cool, but not all of us maze solving fanatics have printers on hand all the time! What I'm saying is it would be nice to be able to solve it within the browser too. Oh, and also, the mazes it generates are much better than that first website.

Q: Were there any improvements you would make to the second website, [mazesforfun](#)?

A: I guess I've already talked about the downloading-not-solving-in-the-app problem, so I won't go into that again. Apart from that, the user interface just wasn't quite as good. On its own it's probably fine, but when you compare it to the other website it's just not quite as nice. It's all just standard button-y stuff whereas the other website had cool hover over things and blocks expanding when you click them and stuff.

Q: Overall, which website was better?

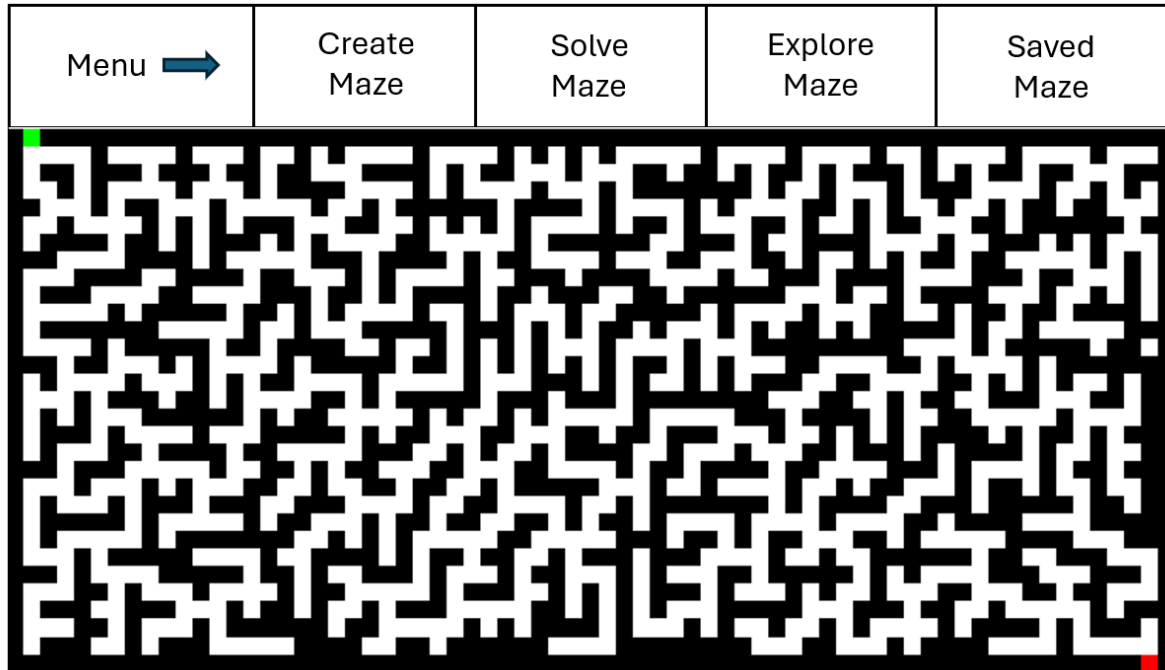
A: Sadly, probably the second one. Even though the buttons are boring it's much better at actually doing what it's supposed to, that is, generating mazes. But I wish the user interface was more fun.

Q: What do you think are key elements that make a good maze?

A: Hmm. If I think about that first website, that maze just looked really ugly. It's harder to pinpoint why that is though. I guess there was too much empty space? Like you could often find 3x3 or even 4x4 sections of just blank space which isn't really great maze design. There were also some bits just completely blocked off from the centre but at the same time some single pixels, which again didn't look great.

Prototype:

In light of this research, here's what the final program could look like. The bar at the top definitely resembles that of mazesolver.xyz in its simplicity, however the maze shown is of a much higher quality.



Project:

Language:

For this project, I will use the python language. Python is excellent at rapid development due to it being interpreted at runtime, rather than compiled first. This means that small snippets of code can easily be tested out of context, speeding up the coding process. Python is also very readable, making writing this documentation easier.

The libraries I will use are:

- Pygame – for the user interface, as allows much more freedom than libraries like Tkinter.
- NumPy – used mostly for its `argwhere()` function, which makes iterating through matrices to find certain terms much, much faster than it would be using for loops.
- Random – used to generate random numbers (otherwise all the mazes would be the same!)

Possible Challenges:

- Maze generation can be challenging – need to make sure there are multiple paths the user can follow, all corridors are connected, etc.
- There are lots of ways to solve a maze, which one is best?? If multiple paths to finish possible, need to make sure the solver finds the shortest one.
- The Pygame library has no built in UI classes (no buttons, sliders, etc) so the user interface will have to be built from the ground up.
- How will the user explore the maze? Backtracking could be difficult

Objectives:

(Green = Base scope, *Red with italics = Stretch*)

1. Maze generation algorithm

- a) Add a generate maze tab at the top of the user interface
 - i. When the tab is selected, allow the user to change the width and height of the maze in a separate menu
 - ii. Include a generate maze button in this menu
- b) Allow the user to specify the dimensions of the maze
 - i. Use sliders or text inputs so the user can give width and height
 - ii. Have an upper and lower bound for the size of the maze
- c) Once dimensions selected and generate maze pressed, use an algorithm to generate a maze
- d) *Show the maze on screen as it goes through stages of generation*
- e) *Let the user input a numerical seed, where inputting the same seed twice will generate the same maze*

2. Explore maze mode

- a) Add explore maze tab to the menu bar at the top of the interface
 - i. When the tab pressed, enter explore maze mode
 - ii. *Visually indicate on the menu button that explore maze mode is active*
 - iii. *If tab pressed again, exit explore maze mode and reset their solution*
- b) When explore maze mode active, arrow keys can be used to move around the maze
 - i. Path taken by the user shown
 - ii. User's current location indicated by a different coloured pixel
- c) *Display a congratulations when the user reaches the end of the maze*

3. Maze solving algorithm

- a) Add maze solving tab to the menu bar
 - i. When tab pressed, reveal maze solution
 - ii. *If the tab pressed again, hide maze solution*
- b) Implement breadth first search algorithm to find the exit of the maze
- c) When exit found, reveal the shortest route
 - i. *Quickly reveal one space at a time for an animation sort of effect*

4. Functionality to save a maze using file handling

- a) Add save maze button to the menu bar
 - i. Reveals save/load maze button when clicked
- b) Create a file to save maze in
 - i. Saving a second maze overwrites the file
 - ii. *Use multiple files for ability to save multiple mazes*

5. User interface

- a) Menu bar at the top has buttons for each action that the user can take
 - i. Create maze
 - ii. Explore maze
 - iii. Solve maze
 - iv. Save/load maze
 - v. *When hovering over each button, it changes colour*
 - vi. *Buttons change colour when active, e.g. generate maze button is a different colour while generating a maze*
- b) *Dropdown menus when some buttons are pressed for further options, e.g. when choosing maze width and height*
- c) Until first maze generated, stop all actions except generate maze or read maze from file
 - i. Show an error message if the user tries to do something with an empty maze

Summary:

To summarise, I have decided to create a piece of software that will generate mazes, allow the user to explore the mazes generated, show the shortest path through the mazes. The user will also be able to save any mazes that they have generated to a file and load any previously saved mazes.

Now, onto the design section, where we'll write some pseudocode for key functions of the program.

Design:

Overview:

As mentioned in the analysis section, my project will be written in the python language, using Pygame to create a user interface and NumPy to process the arrays representing the mazes.

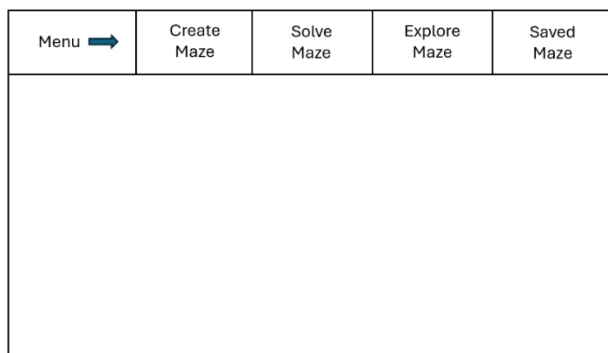
The program will have the functionality to generate mazes, solve mazes, let the user solve the maze themselves and save mazes to a file.

The User Interface:

The user interface will consist primarily of a menu bar, like the one used by online maze solver (from the existing solutions in the analysis section). Below the bar will be a large space which will remain empty until the first maze is generated or loaded. The buttons in the bar will be:

- Create maze
- Explore maze
- Solve maze
- Save/load maze

Below the bar will be a large space for mazes. The space will be empty until the first maze is generated or loaded and will then contain a maze for the remainder of the program's run time.



This would be the user interface before a maze is generated or loaded

```
# parameters depend on size of the window

drawMazeButton = pygame.rect(X, Y, width, height)
solveMazeButton = pygame.rect([parameters])
exploreMazeButton = pygame.rect([parameters])
fileButton = pygame.rect([parameters])

buttonList = [drawMazeButton, solveMazeButton, exploreMazeButton, savedMazeButton]

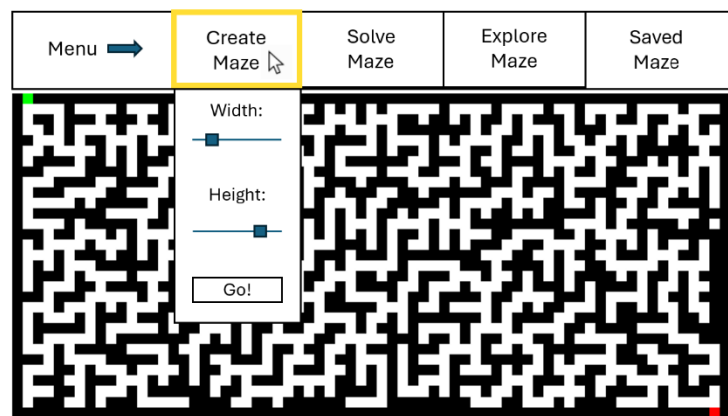
for rect in buttonList:
    if rect.collidepoint(mouseX, mouseY) == True:
        # do something
```

The above code defines each button at the top of the user interface as a rectangle object, adding them to a list that can be iterated through. The parameters in the above example are open to interpretation, as they only describe the location and size of the buttons. Using the collidepoint function the program can check if the coordinates of the mouse (which are found elsewhere in the program) collide with any rectangle in the list.

The create maze button will reveal a menu when it is hovered over, allowing the user to set the width and height of the maze, before selecting a second generate maze button within the menu.

This is the dropdown menu that could appear when the create maze button is hovered over.

There will not always be a maze present, or if there is, it will not always take up the entire space depending on its size.



```
# rectList[0] is the drawMazeButton

if rectList[0].collidepoint(mouseX, mouseY) == True:

    if mouseButtonDown == True:
        generateMaze()

    else:
        drawPopUpMenu()
```

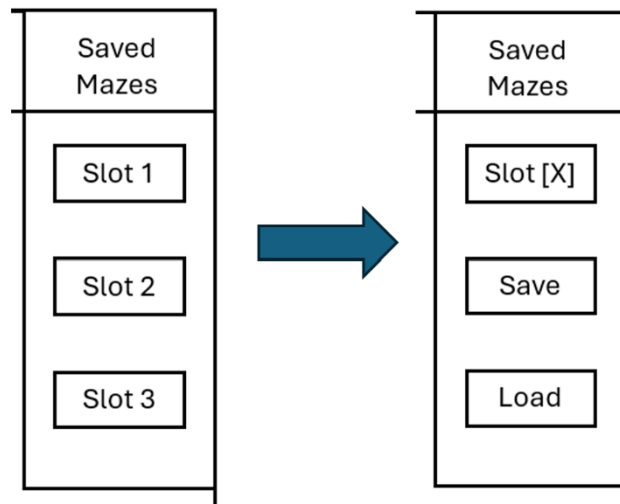
The code on the left shows how the buttonList (or rectList as it is now named) could be used with the collidepoint function to check if the mouse is over a certain button.

The explore maze and solve maze buttons will be toggleable, allowing the user to show and hide the shortest path, as well as switching between standard and explore maze mode, where they will be able to use the arrow keys to create their own solution to the maze.

The saved mazes button will have a dropdown menu of a similar size to the create maze menu. When hovered over, it will look like this:

These are the two stages of the saved mazes menu.

The left image is where the user selects a slot to read/write from. The right image is what is shown once a slot is selected.



Buttons!

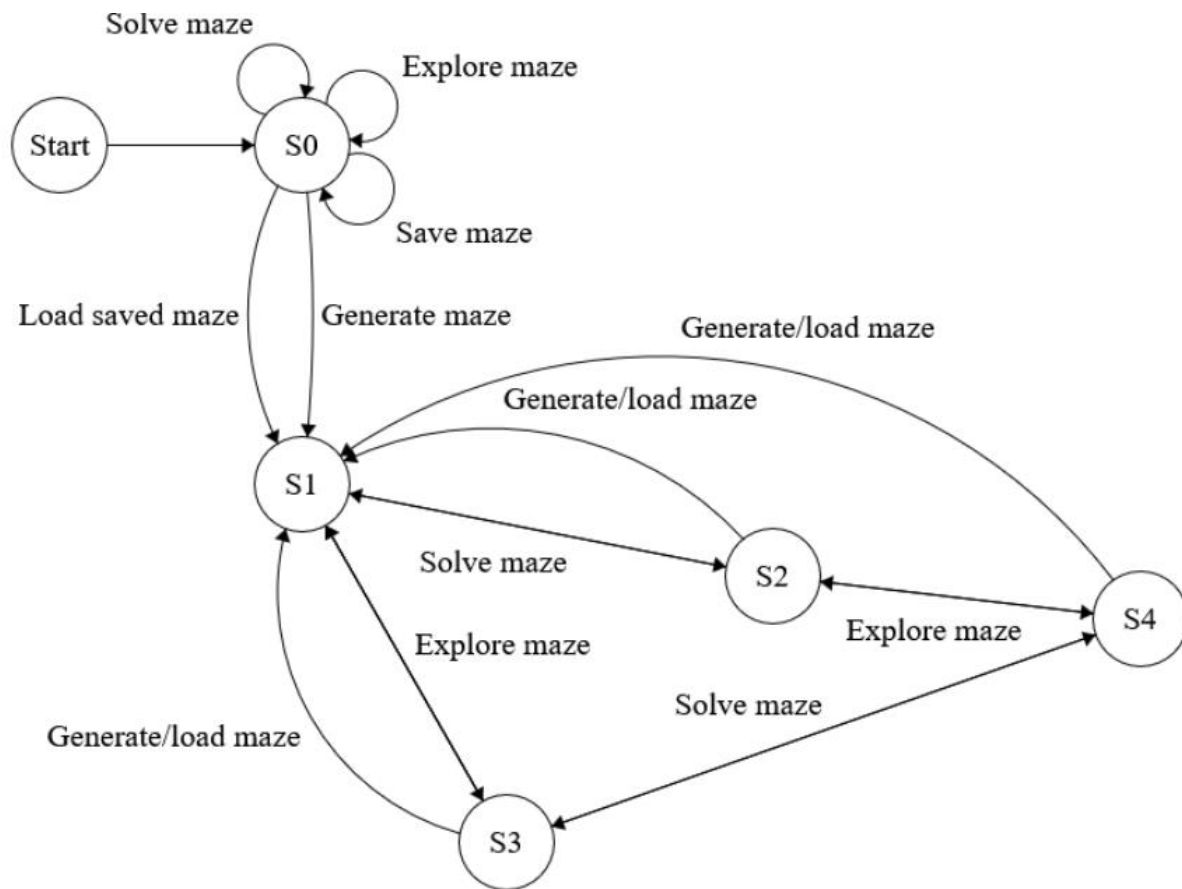
These rules will dictate the appearance of all buttons when hovered over and clicked, using the solve maze button as an example.

	Solution not showing	Solution showing
Mouse not over button		
Mouse over button		

So, an inactive button will be black until hovered over, at which point it will turn yellow. A button that is currently doing something, such as the solve maze button when the solution is showing, or the generate maze button while a maze is generating will always be blue.

FSM:

This finite state machine models the workings of the user interface.



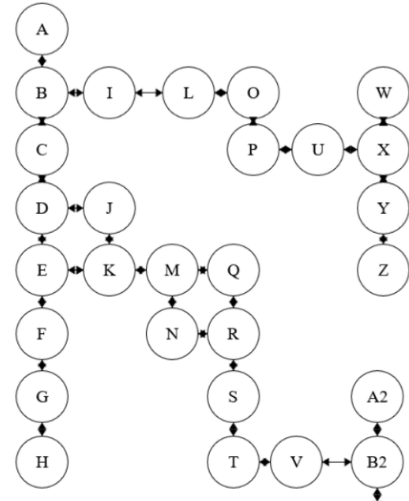
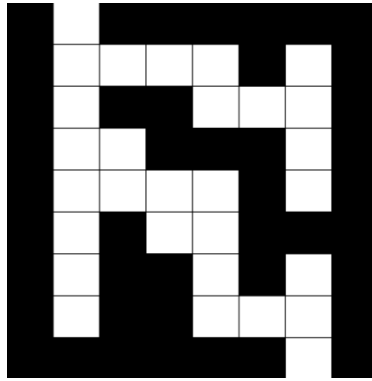
State key:

- S0 – No maze generated
- S1 – Maze generated
- S2 – Maze generated and solution showing
- S3 – Maze generated and user exploring maze
- S4 – Maze generated and solution showing and user exploring maze

The Solving Algorithm:

To solve the maze, I have opted to use the breadth first search algorithm, as it is guaranteed to find the shortest path through an unweighted graph. The maze will be treated as a graph, any unit of corridor being a single node.

For example, the graph of this maze would look like this:



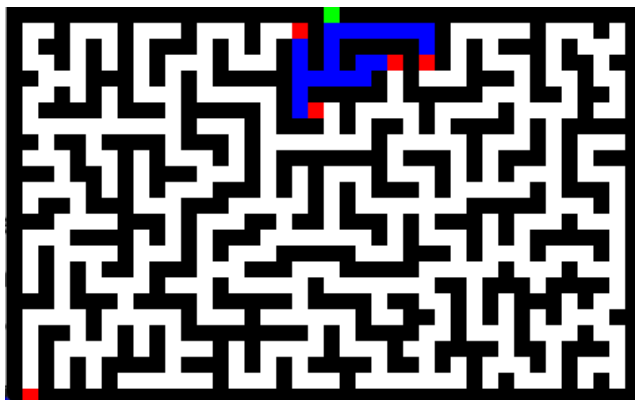
Starting by adding the maze entrance to a list, the breadth first algorithm repeatedly:

- Marks all nodes currently in the list as visited
- Removes all nodes from the list
- Adds all adjacent unvisited nodes to the list

This process continues until the exit to the maze is found.

ActiveNodes Class:

To implement this solving algorithm, we need to define a specialist class to represent the nodes in the maze. This is the purpose of the `ActiveNodes` class.



In this example, the green pixel is the start of the maze, the blue pixels have already been explored by the algorithm, and the red pixels are the current active nodes (except the one marking the exit of the maze).

Each instance of the class will have three attributes; the node's X coordinate, the node's Y coordinate, and the 'parent node', the node that the instance was first visited by. This parent node attribute will be used to find the shortest route once the finish is found.

The ActiveNodes class will also include a function to find valid adjacent nodes to expand to. This findNewNodes function would return a list of valid adjacent nodes, as well as a Boolean set to True if the maze finish has been found, and False otherwise.

Pseudocode for the ActiveNodes class could look like this:

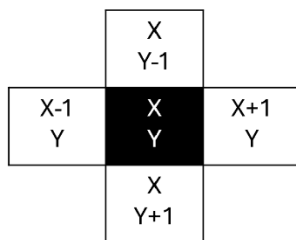
```
# let 0s denote empty corridors, 1s be walls, 9 be the start of the maze, and 8 be the exit

class ActiveNode():
    def __init__(self, X, Y, parentNode):
        self.X = X
        self.Y = Y
        self.parentNode = parentNode
```

This function initialises the variables for the instance of the class.

```
def findNewNodes(self, maze):
    foundExit = False

    possibleMovements = [[self.X - 1, self.Y], [self.X + 1, self.Y], [self.X, self.Y - 1], [self.X, self.Y + 1]]
    validMovements = []
```



This diagram shows all the orthogonally adjacent nodes as considered by this part of the function.

```
for row, column in possibleMovements:
    if maze[row][column] == 8:
        foundExit = True
    elif maze[row][column] == 0:
        validMovements.append([row, column])

return validMovements, foundExit
```

If any of the possible movements are the exit (marked in the array by an 8), set foundExit to True.

To summarise, each instance of the ActiveNodes class has attributes that determine its position in the maze and can recursively find the shortest route from itself to the entrance using the parentNode variable. Each instance of the class also has access to the findNewNodes function, returning a list of empty nodes adjacent to itself. Now we understand this, let's have a look at how the solving algorithm can make use of this class.

Back to the Solving Algorithm:

As described earlier, the breadth first algorithm starts by adding the maze entrance to a list, before repeatedly:

- Marking all nodes currently in the list as visited
- Removing all visited nodes from the list
- Adding all adjacent unvisited nodes to the list

This process continues until the exit to the maze is found.

Using the `ActiveNodes` class, code for the solving algorithm could look like this:

```
def solveMaze(maze):

    startingCoords = getMazeStart()
    startNode = ActiveNode(startingCoords[0], startingCoords[1], False)

    newActiveNodesList = []
    activeNodesList = []
    activeNodesList.append(startNode)

    foundExit = False
```

Here, we are just defining variables we will need later in the algorithm. `getMazeStart()` should return the coordinates of the maze entrance in the form `[XCoord, YCoord]`. Using this, we can define our first `ActiveNode`, setting its `parentNode` to `False`.

This section of code is the core of the solving algorithm. For each current `ActiveNode`, the program:

- Marks the node as visited
- Adds all adjacent unvisited nodes to the `newActiveNodesList`

Once the `activeNodesList` has been fully iterated through, it is emptied and all nodes currently in the `newActiveNodesList` are added to it. The `newActiveNodesList` is then cleared, ready for the next iteration of the while loop.

Once the exit is found by any node, the function ends and returns the node that found the exit, so the shortest route can be found using the `parentNode` attribute.

```
while not foundExit:

    for node in activeNodesList:

        maze[node.X][node.Y] = 2 # mark path taken so no backtracking later

        # for each ActiveNode, find adjacent, unvisited nodes
        validMovementsList, foundExit = node.findNewNodes(maze)

        if foundExit == True:
            finishNode = node

        for coordinates in validMovementsList:

            # for each valid movement, create a new ActiveNode
            # don't add it to the activeNodesList yet, as still iterating through it
            newActiveNode = ActiveNode(coordinates[0], coordinates[1], node)
            newActiveNodesList.append(newActiveNode)

    # now we are done iterating through the activeNodesList
    # we can now clear the list, and add all our new active nodes
    activeNodesList.clear()

    for node in newActiveNodesList:
        activeNodesList.append(node)

    # also clear newActiveNodesList, ready for the next while loop iteration
    newActiveNodesList.clear()

return finishNode
```

The only other thing that needs to be considered is an algorithm to find the route that was taken by the node that first finds the finish. This could look something like this:

```
def findRoute(maze, finishNode):  
  
    run = True  
    finishPath = []  
  
    while run:  
  
        # add the current node to the finishPath list  
        finishPath.append(finishNode)  
  
        # check if start of maze has been reached  
        if finishNode.parentNode == False:  
            run = False  
  
        else:  
            # progress to the next node in the path to the entrance  
            finishNode = finishNode.parentNode  
  
    return finishPath
```

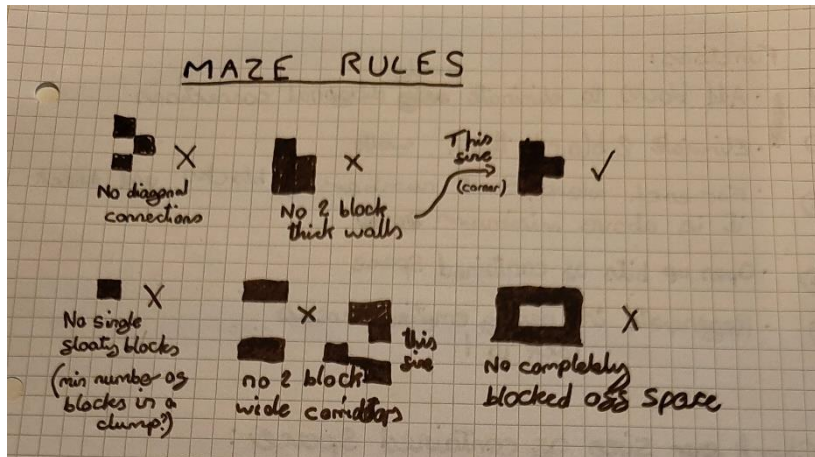
This part is fairly self-explanatory. Add the finish node to the finishPath list, progress to its parentNode, and repeat with the new node.

This could be made recursive if the finishPath list were defined outside the function and passed in as a parameter.

The Generation Algorithm:

Maze Rules:

Rather than using a conventional maze generation algorithm, I have opted for an iterative generator. Each maze will start as a randomised mix of black and white pixels (walls and corridors) and have a series of algorithms executed on it in sequence. This will gradually fix features that should not be allowed in the finished maze, until, after enough iterations, the maze will completely stop changing.



My first draft of what is and isn't allowed in a maze.

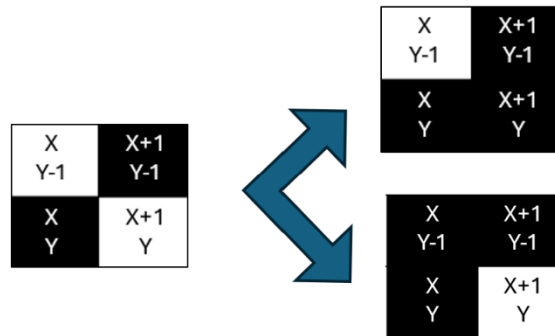
(Maybe add more detail on the different rules?)

For each rule, I will make a separate function to find and correct breakings of that rule. An example implementation of a function to remove purely diagonal connections is shown below:

Here is the first part of the function:

```
def removeDiagonalConnections(maze):
    # let 0s be corridors and 1s be walls
    for x, y where maze == 1:
        if maze[x+1][y-1] == 1 and maze[x+1][y] == 0 and maze[x][y-1] == 0:
            randomNumber = random.randint(1, 2)
            if randomNumber == 1:
                maze[x+1][y] = 1
            elif randomNumber == 2:
                maze[x][y-1] = 1
```

This first *If* statement finds the formation on the left and randomly changes it to one on the right, thus removing the purely diagonal connection.



The function continues:

```

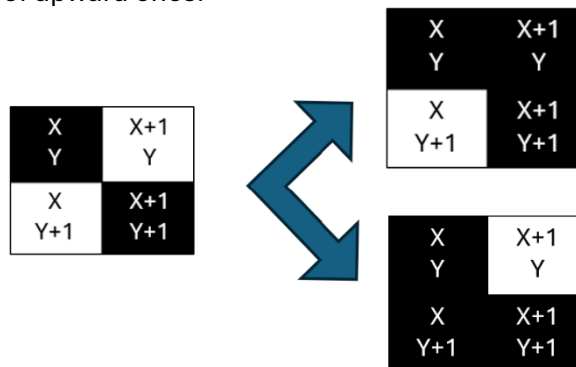
if maze[x+1][y+1] == 1 and maze[x+1][y] == 0 and maze[x][y+1] == 0:
    randomNumber = random.randint(1, 2)

    if randomNumber == 1:
        maze[x+1][y] = 1

    elif randomNumber == 2:
        maze[x][y+1] = 1

```

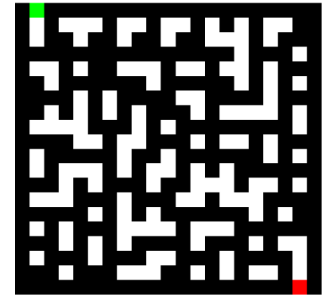
This *If* statement performs a very similar function, but is looking for downward sloping diagonals, instead of upward ones.



Although I have only described one of the functions here, this should be enough to get an idea of how the generation algorithm works. Imagine all of these functions repeatedly iterating in sequence, all breaking other rules while enforcing their own, until eventually they all come to an agreement and the maze stops changing.

OpenContainedSpace Function:

The function to make sure all sections of the maze are connected is the most complex part of the generation algorithm. Without it, each maze would be generated in a fraction of the time, but would look like this:



So, how can we tell whether a part of the maze is connected to the rest of the maze? A simple way to do this would be to check that each node of the maze is connected to the entrance. If every node in the maze is connected to the entrance node, they must all also be connected to each other. There may be a way to optimise this, by looking at whether a node is connected to another node that is connected to the entrance.

Here is some example code for how the function could work.

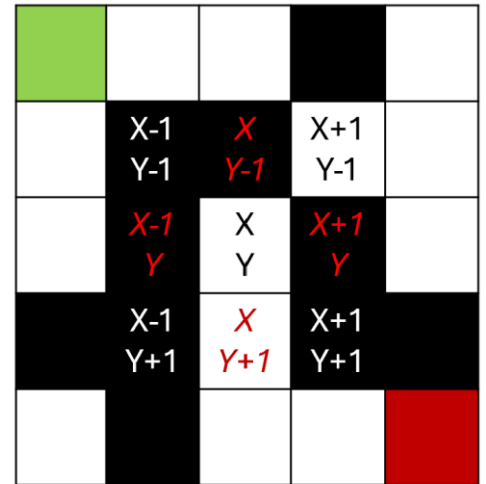
```
def openContainedSpace(maze):  
    for x, y where maze == 0:  
        # entranceConnectedCheck is a variation on the solving algorithm  
        # it returns True if the given coordinates are connected to the maze entrance  
        # it returns False otherwise  
        connected = entranceConnectedCheck(maze, [x, y])  
  
        if connected == False:  
            randomNumber = random.randint(1, 4)  
  
            if randomNumber == 1:  
                maze[x+1][y] = 0  
  
            elif randomNumber == 2:  
                maze[x][y+1] = 0  
  
            elif randomNumber == 3:  
                maze[x-1][y] = 0  
  
            elif randomNumber == 4:  
                maze[x][y-1] = 0
```

This function looks at every entry in the array representing the maze and checks to see if it is a corridor (represented by a 0). If it is, it runs the entranceConnectedCheck() function on it, which will return True if the corridor is connected to the maze entrance, and False otherwise.

If it returns False, one of the orthogonally adjacent entries in the array will be set to 0, even if it is already a corridor. This means that the function could take many iterations to randomly select the correct wall that, when removed, will connect to the maze entrance.

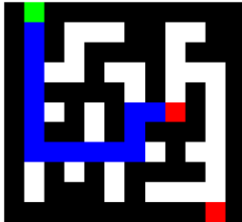
This diagram illustrates the workings of this function. The central unit, coordinates (X, Y), is not currently connected to the green entrance tile. This means that the function will set one of the adjacent units (marked with italics and red font) to corridor at random. It is possible that (X, Y+1) will be chosen despite the fact that it is already a corridor. This would mean that the formation stays the same this iteration.

In fact, only 2 of the 4 adjacent units marked would result in (X, Y) being connected to the maze entrance on removal. This demonstrates why multiple iterations will often be needed before a unit is connected to the entrance, even when only one wall needs to be removed. (This may come up again later).



Maze Exploration:

So, what happens when the user presses the explore maze button? Well, if no maze is generated, nothing. But if there is a maze present, they will be able to explore the maze themselves using the arrow keys.



The user's current position in the maze will be marked by a red pixel. The path that they have taken so far will be marked in blue. They should be able to backtrack on themselves by just moving back the way they came.

The below pseudocode only handles one arrow key, but the others would work in the same sort of way. I will program a stack class with the 'push', 'pop', and 'peek' functions. ExploreStack here is an instance of that class.

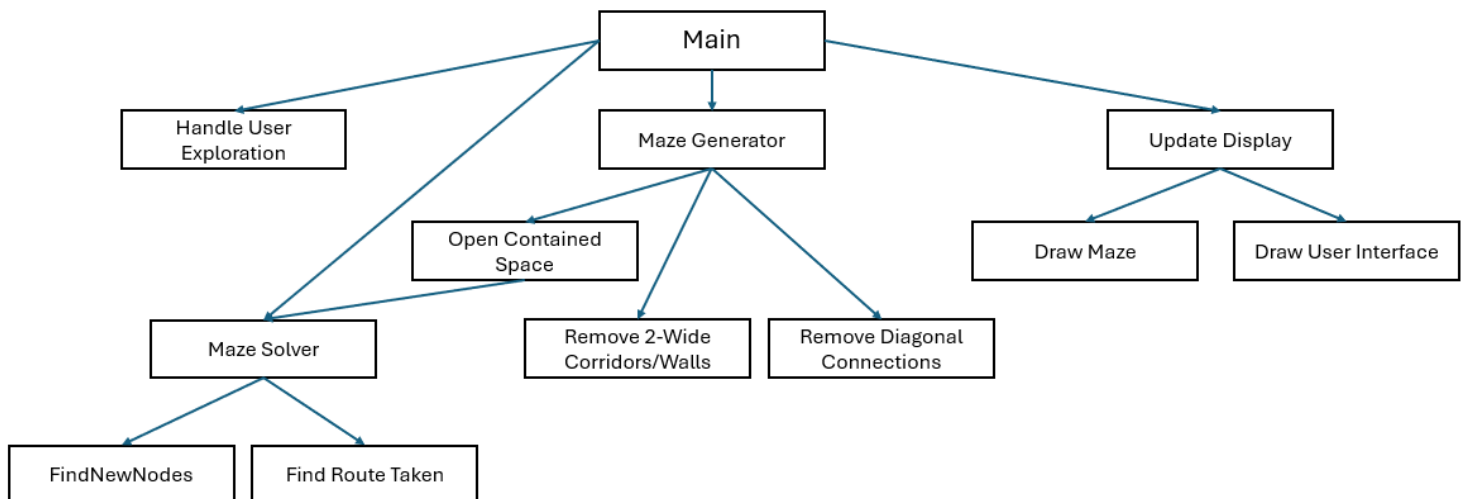
```
def userExploration(maze, currentCoords, exploreStack, exploredList):  
    if keypressed() [key.UP]:  
        if exploreStack.peek() == "down":  
            exploreStack.pop()  
            currentCoords = [currentCoords[0], currentCoords[1]-1]  
            exploredList.remove(currentCoords)  
        else:  
            exploredList.append(currentCoords)  
            exploreStack.push("up")  
            exploreCoords = [exploreCoords[0], exploreCoords[1]-1]
```

When the user presses the up arrow, we first look at the top of the stack to check whether we are undoing a past action or performing a new one. If the top of the stack is "down", then we are undoing an action as we have just performed the opposite action. To do this, we pop the top of the stack, move one unit upwards and then remove the current coordinates from exploredList.

If the last action performed is something other than "down", we add the current coordinates to exploredList before pushing "up" onto the stack and moving one unit up.

Hierarchy Chart and Conclusion:

Now we have seen most of the functions the program will use, here is a nice hierarchy chart showing how they will all fit together. Obviously, the program will use many more functions than just these, but these are the key ones that aid understanding of the program. (There is also no mention of the saved mazes feature here).



Now, onto the technical solution section, to see all this (and more) in detail!

Technical Solution:

Completeness Of Solution:

Let's first go back and have a look at our objectives from the analysis section, to check that we have achieved them.

Objective Number	Objective	Objective Complete?	Link to Code in Appendix
1	Maze Generator		
1a	Add generate maze button in the UI bar.	The generate maze button is the first one in the bar.	DrawButtons
1ai	Allow user to adjust maze width and height of maze in a separate menu.	The user can hover over the generate maze button to show the menu	Draw Generation Options Window
1aii	Include a generate maze button in this menu	The user can press the generate maze button to generate the maze	DrawButtons, Generate Maze Button Pressed
1b	Allow user to specify width and height of maze	The user can specify the width and height of the maze generated.	Arguments of Maze Generator Main
1bi	Use sliders so user can give width and height	There is a width slider and a height slider in the pop up menu.	Slider Class
1bii	Have an upper and lower bound for the size of the maze	The sliders can only slide between the values 10 and 120 for width, and 10 and 60 for height.	Slider Class
1c	Use an algorithm to generate maze	Uses an iterative improvement(?) algorithm to generate maze	Amazing Maze Generator
1d	Show the maze on screen as it generates	When the generate maze button is pressed, a constantly shifting maze is shown, which eventually settles into the final maze.	DrawMaze, Maze Generator Main
1e	Let the user input a seed, where inputting the same seed twice will generate the same maze.	This wouldn't be hard to do if I had more time, the main challenge being where in the UI to put the seed input. However, the ability to save mazes provides a similar effect of being able to bring back mazes that you like.	
2	Ability for user to explore maze		
2a	Add explore maze button in the UI bar.	The explore maze button is the third button along in the user interface bar.	DrawButtons
2ai	When explore maze button pressed, enter explore maze mode	Whenever a fully generated maze is on the screen, the user can enter explore maze mode by pressing the relevant button.	DrawButtons, Explore Maze Button Pressed
2aii	Visually indicate on the button when explore maze mode is active	The button turns blue when explore maze mode is active.	UpdateWindow
2aiii	If button pressed again, exit explore maze mode and reset user's solution	The user can press the explore maze button a second time to exit explore maze mode, causing the button to cease being blue. This also resets their solution.	Explore Maze Button Pressed

2b	When in explore maze mode, arrow keys used to move around the maze	The user can freely navigate the maze using the arrow keys when in explore maze mode.	HandleUser Exploration
2bi	Path taken by user shown	The path taken by the user is shown in blue	UpdateWindow
2bii	User's current location indicated	The user's current location is indicated with a red pixel	UpdateWindow
2c	Display a congratulations when the user reaches the end of the maze	Not included, so as not to stifle the intrinsic joy of completing the challenge. (I didn't have time.)	
3	Maze solver		
3a	Add maze solving button to UI bar.	The solve maze button is the second one along in the bar.	DrawButtons
3ai	When button pressed, reveal maze solution.	The solve maze button solves the maze, revealing the shortest path from the maze entrance to the maze exit.	Maze Solver, Solve Maze Button Pressed
3aii	Press button a second time to hide maze solution.	Similar to the explore maze button, the maze solution can be hidden by pressing the button a second time.	Solve Maze Button Pressed
3b	Implement breadth first search algorithm to find the maze exit.	The maze solver uses the breadth first search algorithm, treating the maze as an unweighted graph.	Maze Solver Main, FindRoute, ActiveNode Class
3c	When exit found reveal the shortest route.	The shortest route, once found, is displayed to the screen.	DrawMaze, FindRoute
3ci	Reveal one space at a time for cool animation effect.	The shortest route is revealed one node at a time.	SolveMaze Button Pressed
4	Ability for user to save mazes to a file		
4a	Add save maze button to the menu bar.	The saved mazes button is the fourth and final button on the menu bar.	DrawButtons
4ai	Reveals save/load maze button when clicked.	When hovering over the button, the saved mazes window is revealed.	DrawSavedMaze Window
4b	Create a file to save maze in	There are three files that the user can save and load mazes from, allowing multiple mazes to be kept at once.	SavedMazeWindow Is True
4bi	Saving a second maze overwrites the file	If a maze is saved to a file, and the user saves another maze to that same file, the first maze is overwritten.	Button 2 Pressed
4bii	Use multiple files for the ability to save multiple mazes.	As previously stated, there are three files to save mazes in.	SavedMazeWindow Is True
5	User Interface		
5a(i-iv)	Menu bar at the top of user interface has buttons for each action user can take.	See objectives 1a, 2a, 3a and 4a.	
5av	When hovering over each button, it changes colour	Buttons turn yellow when hovered over with the mouse.	DrawButtons
5avi	Buttons change colour when active.	Buttons turn blue when they are active, e.g. solve maze button is blue when maze solution showing.	UpdateWindow

5b	Dropdown menus when some buttons are pressed.	'Generate maze' and 'saved mazes' buttons have dropdown windows when hovered over.	Draw Generation Options Window , Draw Saved Mazes Window
5c	Until first maze generated, stop all actions except generate maze or read maze from file.	The user can't use the solve maze or explore maze buttons when no maze is present.	Conditions for Solve Maze Button Pressed and Explore Maze Button Pressed

Matrix Numbers:

Maze matrixes primarily use 0s and 1s, with 0s representing walls, and 1s representing corridors. The number 9 is used to mark the entrance, and the number 8 the exit. There should only ever be a single 9 and a single 8 per maze. And that's it... in the final maze.

During generation and solving, other numbers are used to represent information about nodes in the maze. It remains true that a 1 is a wall and any other number is a corridor. So, a complete key for maze numbers is as follows:

- 0 is an unmarked corridor
- 1 is a maze wall
- 2s are corridors marked as connected to the maze entrance
 - Used only by the generation algorithm
- 3s are corridors currently being explored
 - In terms of the generation algorithm, it is not yet known if these nodes are connected to the maze entrance. They will be changed to 2 or 4 once this is found out.
 - In the solving algorithm, all nodes that have been traversed in the past are marked with a 3.
- 4s are not connected to the maze entrance
 - Used only by the generation algorithm
- 7s mark the shortest route to the finish
 - Used only by the solving algorithm

A Note on Indexing:

Mazes are stored as lists of lists, looking something like this:

```
[1, 2, 3]
[4, 5, 6]
[7, 8, 9]
```

Let's say we want to retrieve the number 7 from this array. Although basic coordinate geometry would think it to be at [0,2], we actually have to decide the row first. So, in the index [0,2], the 0 refers to the first list in the maze, which is the row [1,2,3] and the 2 would mean it returns '3'.

If we did want to get the 7, we would have to index maze[2][0], getting first the bottom row of the maze, and then the first item in it.

So, if you see a line like this:

```
for y, x in np.argwhere(maze == 1):
```

I haven't mixed up my x and y, indexing is just backwards!

Code Context Feature:

In some of my screenshots you may see something looking like this:

```
while run == True:
    for event in pygame.event.get():
        if clicked == True:
            if rectList[0].collidepoint(mouseX, mouseY) == True:
                elif mazeCreated == True and rectList[1].collidepoint(mouseX, mouseY) == True:
                    elif mazeCreated == True and rectList[2].collidepoint(mouseX, mouseY) == True:

                        exploreCoords = [1, 0]
                        pastExploreList = []

                        exploreMaze = not exploreMaze
```

The grey bit at the top is 'code context': it gives all the statements above what you are looking at that are relevant, basically anything at a lower or equal indent level (with lines between cut out). I'm using this feature as it helps the reader understand where the code extract is in relation to everything else.

Program Flow:

The main body of the program is encased in a while loop and a Pygame event handling loop. This means the majority of the main function is run over and over again. The event handling loop means that when the user provides no input, the program doesn't actually do much processing. This is great, as you can just leave it running in the background without too many negative effects.

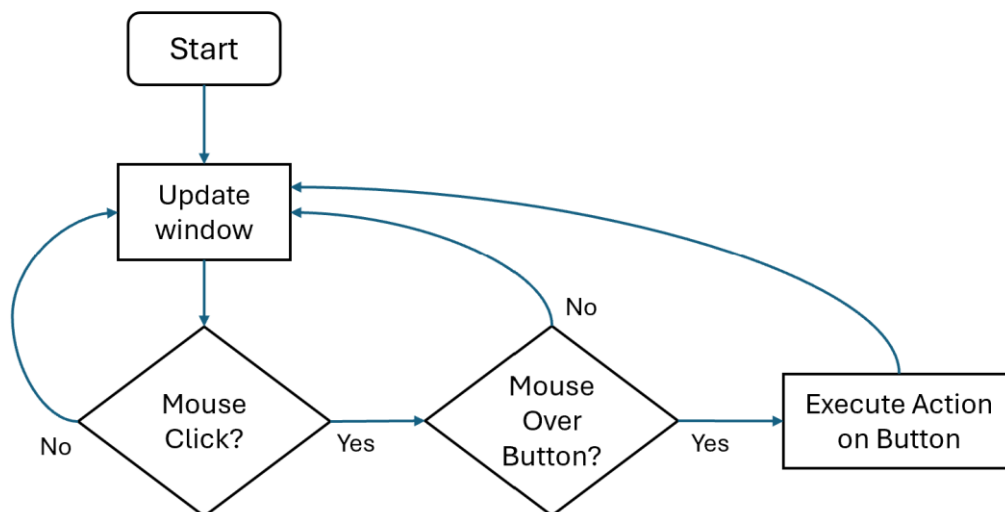
```
run = True
while run == True:

    rectList, mouseX, mouseY, maze1Button, maze2Button, maze3Button = updateWindow(

    for event in pygame.event.get():
```

The only thing called outside of the event handling loop is the updateWindow function, which handles almost all of what the user sees. This is just because if the user abruptly stopped what they were doing, it would lead to some odd looking freeze frames until the next input.

This is a very simplified diagram showing basically what the program does and how it loops.



The program continues following this diagram until the user terminates it somehow. There is more to the program than this, such as menus that appear when hovering over buttons, but this gives you a good idea of the structure of the main loop.

User Interface:

UpdateWindow Function:

This function is a sort of master function dictating what the user sees. It is only called in one place, right at the start of the main while loop, as just mentioned in the program flow section.

Here it is:

```
def updateWindow(fillWhite, buttonUpdate, mazeUpdate, optionsWindow, savedMazeWindow, mouseX,\
mouseY, clicked, solveMaze, exploreMaze, exploreCoords, pastExploreList, mazeTopLeft, saveSlotChoice):

    if fillWhite == True:

        # completely fill the screen with white
        window.fill(WHITE)

    if exploreMaze == True:

        # draw in blue the nodes user has visited in the past
        for coordinates in pastExploreList:
            pygame.draw.rect(window, BLUE, (mazeTopLeft[0] + \
            coordinates[0]*10, mazeTopLeft[1] + coordinates[1]*10, 10, 10))

        # draw in red the user's current location
        pygame.draw.rect(window, RED, (mazeTopLeft[0] + \
            exploreCoords[0]*10, mazeTopLeft[1] + exploreCoords[1]*10, 10, 10))

    if mazeCreated == True and mazeUpdate == True:

        # redraw the maze to the screen
        updateMaze()

    if optionsWindow == True:

        # draw the options window under generate maze button
        drawGenerationOptionsWindow(window, mouseX, mouseY, clicked)

    if savedMazeWindow == True:

        # draw options window under saved mazes button, returns rectangles for the three mini buttons
        maze1Button, maze2Button, maze3Button = drawSavedMazeWindow(window, mouseX, mouseY, saveSlotChoice)
    else:

        # buttons still have to exist if the window isn't open, so just make them unclickable
        maze1Button = pygame.Rect(0, 0, 0, 0)
        maze2Button = pygame.Rect(0, 0, 0, 0)
        maze3Button = pygame.Rect(0, 0, 0, 0)

    if buttonUpdate == True:

        # draw the buttons to the screen and get a list of their rectangles
        rectList = drawButtons()

    if exploreMaze == True:

        # draw blue rectangle around explore maze button if it is active
        pygame.draw.rect(window, BLUE, (800, 0, 200, 100), 10)

    if solveMaze == True:

        # draw blue rectangle around solve maze button if it is active
        pygame.draw.rect(window, BLUE, (600, 0, 200, 100), 10)

    pygame.display.update()

    return rectList, maze1Button, maze2Button, maze3Button
```

There's a lot to take in here, so let's first look at the parameters it takes – and wow, there's a lot of them.

```
def updateWindow(fillWhite, buttonUpdate, mazeUpdate, optionsWindow, savedMazeWindow, mouseX,\nmouseY, clicked, solveMaze, exploreMaze, exploreCoords, pastExploreList, mazeTopLeft, saveSlotChoice):
```

FillWhite, buttonUpdate and mazeUpdate are actually unnecessary in the current version of the program. They are all booleans that determine whether the function performs their respective actions. This was meant to make the function more flexible, for example if you wanted to update the maze without changing the rest of the user interface, you could leave fillWhite and buttonUpdate as false in your call. This is never actually used however, as the function is only called once.

OptionsWindow and savedMazeWindow are also booleans. If optionsWindow is set to true, the menu under the generate maze button that lets the user change the maze width and height is drawn to the screen. Similarly, if savedMazeWindow is true, the window under the saved mazes button is drawn to the screen.

MouseX and mouseY are the X and Y coordinates of the mouse.

Clicked is *another* boolean, this one set to true if either mouse button is down, false if they are both up.

SolveMaze is set to true if the solution to the maze is currently showing.

ExploreMaze is set to true if the user is currently exploring the maze.

ExploreCoords holds the user's current location when exploring the maze. PastExploreList is a list of all the coordinates the user has previously explored.

MazeTopLeft holds the coordinates that the top left corner of the maze will be drawn at.

SaveSlotChoice is set to 1, 2, 3 or False, depending on whether the user has chosen a file slot in the savedMazeWindow.

This is obviously a very complex function in terms of parameters, but this is important to support its flexibility, as well as how comprehensive it is. Now, onto the actual code:

```
if fillWhite == True:\n\n    # completely fill the screen with white\n    window.fill(WHITE)
```

This first part is pretty self explanatory. We fill the entire window with white, providing a blank slate to draw everything else on top of.

```

if exploreMaze == True:
    # draw in blue the nodes user has visited in the past
    for coordinates in pastExploreList:
        pygame.draw.rect(window, BLUE, (mazeTopLeft[0] + \
            coordinates[0]*10, mazeTopLeft[1] + coordinates[1]*10, 10, 10))

    # draw in red the user's current location
    pygame.draw.rect(window, RED, (mazeTopLeft[0] + \
        exploreCoords[0]*10, mazeTopLeft[1] + exploreCoords[1]*10, 10, 10))

```

If the user is in explore maze mode, we have to show their current location and the route that they have taken to get there. The above code draws a red unit at their current location and fills in the path taken to get there in blue.

```

if mazeCreated == True and mazeUpdate == True:
    # redraw the maze to the screen
    updateMaze()

```

Next, if a maze has been created, call the updateMaze function.

```

if optionsWindow == True:
    # draw the options window under generate maze button
    drawGenerationOptionsWindow(window, mouseX, mouseY, clicked)

```

The generation options window is drawn by a separate function (which we will look at in the slider section), so all that this 'master function' does is call that other function.

```

if savedMazeWindow == True:
    # draw options window under saved mazes button, returns rectangles for the three mini buttons
    maze1Button, maze2Button, maze3Button = drawSavedMazeWindow(window, mouseX, mouseY, saveSlotChoice)
else:
    # buttons still have to exist if the window isn't open, so just make them unclickable
    maze1Button = pygame.Rect(0, 0, 0, 0)
    maze2Button = pygame.Rect(0, 0, 0, 0)
    maze3Button = pygame.Rect(0, 0, 0, 0)

```

Similarly with the saved maze window.

```

if buttonUpdate == True:
    # draw the buttons to the screen and get a list of their rectangles
    rectList = drawButtons()

```

This drawButtons function returns a list of the rectangle hitboxes of the buttons (as well as actually drawing them).

```
if exploreMaze == True:
    # draw blue rectangle around explore maze button if it is active
    pygame.draw.rect(window, BLUE, (800, 0, 200, 100), 10)

if solveMaze == True:
    # draw blue rectangle around solve maze button if it is active
    pygame.draw.rect(window, BLUE, (600, 0, 200, 100), 10)
```

If exploreMaze or solveMaze modes are active, we outline the buttons in blue to show this. We don't need one of these for generate maze, as this function is not actually called while maze generation is in progress.

`pygame.display.update()` is quite important – without it, none of the changes that we've made are actually sent to the screen!

And here are the variables we return!

```
return rectList, maze1Button, maze2Button, maze3Button
```

These are all just rectangle hitboxes of buttons, except for rectList which is a list of buttons.

Buttons:

So, we have this row of buttons at the top of the screen:



But how does the program know when the buttons have been pressed? In the `updateWindow` function, we saw a function called `drawButtons`. Let's have a look at the code.

```
def drawButtons():
    """
    Draws the top row of buttons to the screen.
    Changes colour of buttons when mouse is hovering over them.
    Returns a list of 4 rectangles representing the buttons.
    """

    # write all the labels on the buttons

    drawText("Menu ->", 200, 50)
    drawText("Generate", 500, 28)
    drawText("Maze", 500, 70)
    drawText("Solve", 700, 28)
    drawText("Maze", 700, 70)
    drawText("Explore", 900, 28)
    drawText("Maze", 900, 70)
    drawText("Saved", 1100, 28)
    drawText("Mazes", 1100, 70)

    # draw black box around 'menu' label
    # don't need the rectangle, as isn't a pressable button
    pygame.draw.rect(window, BLACK, (0, 0, 400, 100), 10)

    # rectangle around 'generate maze' label
    drawMazeRect = pygame.draw.rect(window, BLACK, (400, 0, 200, 100), 10)

    # 'solve maze'
    solveMazeRect = pygame.draw.rect(window, BLACK, (600, 0, 200, 100), 10)

    # 'explore maze'
    exploreMazeRect = pygame.draw.rect(window, BLACK, (800, 0, 200, 100), 10)

    # 'saved mazes'
    fileRect = pygame.draw.rect(window, BLACK, (1000, 0, 200, 100), 10)

    # list of the rectangles just created
    rectList = [drawMazeRect, solveMazeRect, exploreMazeRect, fileRect]

    # check if the mouse collides with any of the buttons
    # if it does, draw a yellow rectangle over the top
    for rect in rectList:
        if rect.collidepoint(mouseX, mouseY) == True:
            pygame.draw.rect(window, YELLOW, (rect), 10)

    return rectList
```

(`drawText` is a user defined shorthand function for about 4 lines of code that write text to the screen)

So, to start with, we draw all the text that lies within the buttons, using the `drawText` function. Then, draw the rectangles around each of the 4 menu buttons: generate maze, solve maze, explore maze, and saved mazes. These are all saved as rectangle objects and added to `rectList`, which is the only return of the function. The function also checks whether the user is hovering over any button in the `rectList` and changes its colour if so.

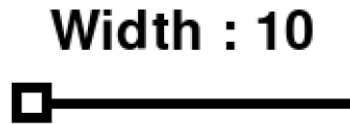
So, now we have the `rectList`, how does this help us?

Generate Maze	Solve Maze	Explore Maze	Saved Mazes
<code>rectList[0]</code>	<code>rectList[1]</code>	<code>rectList[2]</code>	<code>rectList[3]</code>

As you can see from this diagram, we can index the `rectList` to get the hitbox of any of the four buttons. To check if the mouse is over any of the buttons, we can use the line `rectList[x].collidepoint(mouseX, mouseY)`, which will return true if the mouse is over the x^{th} button in the list.

Sliders!

I needed a way for the user to input their desired maze width and height. And what better way to do that than with sliders! So here is the code for my very own, fully functional, slider class!



Each instance of the class looks like this when drawn to the screen.

The class has three functions: `__init__`, which is just called to create each instance, `drawSlider` and `handleSlider`.

`__init__` Function:

Let's first look at `__init__` and all the attributes held by each member of the class.

```
class Slider():
    def __init__(self, XCoord, YCoord, lowerBound, upperBound, text):
        self.XCoord = XCoord
        self.YCoord = YCoord
        self.lowerBound = lowerBound
        self.upperBound = upperBound
        self.currentValue = lowerBound
        self.sliderWidth = 150
        self.sliderHeight = 50
        self.text = text
        self.selected = False
```

- XCoord and YCoord are the coordinates of the top left(ish) corner of the slider.
- lowerBound and upperBound are the largest and smallest numerical values that the slider can represent.
- currentValue is the current number held by the slider. It starts at the lowest possible value.
- sliderWidth and sliderHeight are the physical size of the slider (in pixels).
- Text is the writing that will be displayed above the slider.
- Selected is a boolean that is only true when the user is clicking down on the slider.

HandleSlider Function:

```
def handleSlider(self, window, mouseX, mouseY, clicked):  
    # call the drawslider function  
    sliderRect, sliderXPosition = self.drawSlider(window)  
  
    # if mouse button down and mouse over slider button, slider is selected  
    if clicked == True:  
        if sliderRect.collidepoint(mouseX, mouseY) == True:  
            self.selected = True  
    else:  
        self.selected = False  
  
    # if slider is selected and mouse is to the right of slider button:  
    # increase number on slider, thus moving it right  
    if self.selected == True and mouseX >= sliderXPosition:  
        while mouseX > sliderXPosition + 10 and self.currentValue < self.upperBound:  
            self.currentValue += 1  
  
        # call drawSlider again to show the changes to the movable part  
        sliderRect, sliderXPosition = self.drawSlider(window)  
  
    # if slider is selected and mouse is to the left of slider button:  
    # decrease number on slider, moving it left  
    if self.selected == True and mouseX < sliderXPosition:  
        while mouseX < sliderXPosition and self.currentValue > self.lowerBound:  
            self.currentValue -= 1  
  
        sliderRect, sliderXPosition = self.drawSlider(window)
```

This function does a few things, firstly checking whether the user is clicking on the movable part of the slider and updating the 'selected' variable accordingly.

Then, if the mouse X value is greater than the slider X value, we know that the mouse is to the right of the slider, so the value on the slider increases. Vice versa, if the mouse's X value is less than the slider X value, the mouse is to the left of the slider, so the value on the slider decreases.

Since this function is called inside the main while loop, even though the slider only increases or decreases one unit at a time, it quickly catches up to the mouse.

DrawSlider Function:

```
def drawSlider(self, window):
    # write whatever the user has named the slider,
    # followed by the slider's current value at a point centered above the slider
    drawText(window, (self.text + " : " + str(self.currentValue)), \
               self.XCoord + self.sliderWidth//2 + 10, self.YCoord - 10)

    # get the X-Coordinate of the moveable part of the slider
    sliderXPosition = self.XCoord + ((self.currentValue-self.lowerBound)\
                                       /(self.upperBound-self.lowerBound))*self.sliderWidth

    # draw the slider bar
    pygame.draw.line(window, BLACK, (self.XCoord, self.YCoord + self.sliderHeight//2), \
                     (self.XCoord + self.sliderWidth - 1 + 20, self.YCoord + self.sliderHeight//2), 5)

    # if slider being moved, draw the movable part of the slider in a different colour
    if self.selected == True:
        sliderRect = pygame.draw.rect(window, YELLOW, (sliderXPosition, \
                                                         self.YCoord + self.sliderHeight//2 - 10, 20, 20), 5)
    else:
        # draw the movable part of the slider
        sliderRect = pygame.draw.rect(window, BLACK, (sliderXPosition, \
                                                         self.YCoord + self.sliderHeight//2 - 10, 20, 20), 5)

    # draw the little white box inside the movable part of the slider
    pygame.draw.rect(window, WHITE, (sliderXPosition+5, self.YCoord \
                                       + self.sliderHeight//2 - 5, 10, 10))

    return sliderRect, sliderXPosition
```

There's lots of complicated looking coordinate-based drawing in this function, however it's much simpler than it looks. The logic is explained in the comments above, so I won't repeat it here.

This drawSlider function is only ever called by the handleSlider function. They could just be one function, but I feel that it's nice to have them separated, since they do distinct things.

Where in the Code are They?

The sliders are created at the start of the main function, outside of any loops, as they only need to be created once.

```
widthSlider = slider.Slider(415, 150, 10, 120, "Width")
heightSlider = slider.Slider(415, 300, 10, 60, "Height")
```

The handleSlider function is called in the drawGenerationOptionsWindow function, which is in turn called by the updateWindow function:

```
def drawGenerationOptionsWindow(window, mouseX, mouseY, clicked):
    pygame.draw.rect(window, WHITE, (400, 90, 200, 350))
    pygame.draw.rect(window, BLACK, (400, 90, 200, 350), 10)

    heightSlider.handleSlider(window, mouseX, mouseY, clicked)
    widthSlider.handleSlider(window, mouseX, mouseY, clicked)
```

Pop up Menus:

As nice as the little bar at the top of the screen is, it only has four buttons. When we want to specify the width and height of the maze generated or choose which file slot to save a maze to, we're going to need more buttons – the only logical solution to this is pop up menus!

There's nothing fancy going on here, just some hard coded graphics for each window. Let's have a look at them anyway:

Generation Options Window:

```
def drawGenerationOptionsWindow(window, mouseX, mouseY, clicked):
    pygame.draw.rect(window, WHITE, (400, 90, 200, 350))
    pygame.draw.rect(window, BLACK, (400, 90, 200, 350), 10)

    heightSlider.handleSlider(window, mouseX, mouseY, clicked)
    widthSlider.handleSlider(window, mouseX, mouseY, clicked)
```

All this does is draw a white rectangle to block out any existing graphics, draw a black rectangle as a border, and update the sliders. The function is called in the updateWindow function when the optionsWindow variable is true. But when is the optionsWindow variable set to true?

This code is called within the main program while loop:

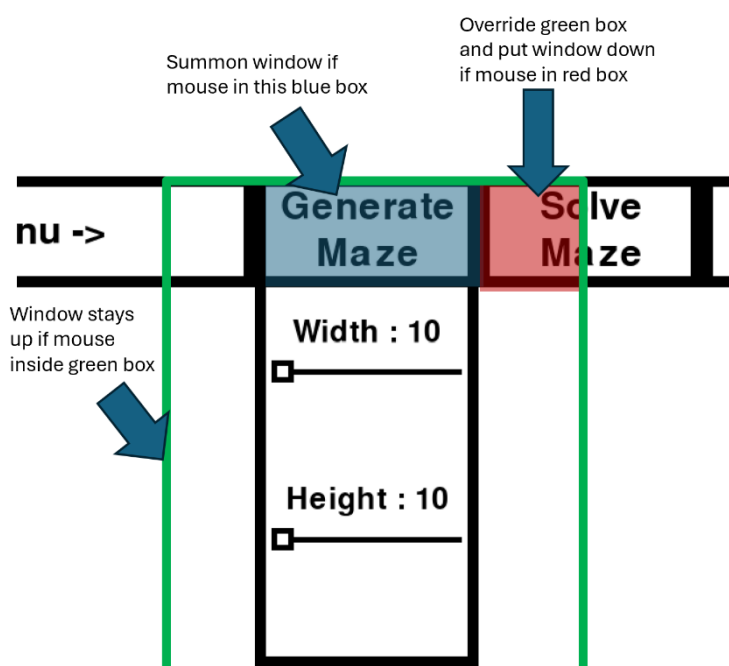
```
# get rid of the generation options window when mouse over solve maze button
if rectList[1].collidepoint(mouseX, mouseY) == True:
    optionsWindow = False

# draw options window if mouse over draw maze button
elif rectList[0].collidepoint(mouseX, mouseY):
    optionsWindow = True

# if options window already showing, be more lenient about when to hide it
elif optionsWindow == True and mouseX >= 300 and mouseX <= 700 and mouseY <= 440:
    optionsWindow = True

else:
    optionsWindow = False
```

This diagram sums up what the code does, showing the rules for when optionsWindow is true, and therefore when the window will be active.



Saved Maze Window:

The code to draw this menu is much longer, as I haven't made a button class that handles everything like the slider class did in the last menu.

```
def drawSavedMazeWindow(window, mouseX, mouseY, saveSlotChoice):

    # draw a white box with a black border to make up the window
    pygame.draw.rect(window, WHITE, (1000, 90, 200, 350))
    pygame.draw.rect(window, BLACK, (1000, 90, 200, 350), 10)

    # draw the three buttons in the window, and get their rect objects
    maze1Button = pygame.draw.rect(window, BLACK, (1030, 130, 140, 60), 7)
    maze2Button = pygame.draw.rect(window, BLACK, (1030, 230, 140, 60), 7)
    maze3Button = pygame.draw.rect(window, BLACK, (1030, 330, 140, 60), 7)

    # draw yellow rectangles around the buttons if they are being hovered over
    if maze1Button.collidepoint(mouseX, mouseY):
        pygame.draw.rect(window, YELLOW, (1030, 130, 140, 60), 7)

    elif maze2Button.collidepoint(mouseX, mouseY):
        pygame.draw.rect(window, YELLOW, (1030, 230, 140, 60), 7)

    elif maze3Button.collidepoint(mouseX, mouseY):
        pygame.draw.rect(window, YELLOW, (1030, 330, 140, 60), 7)

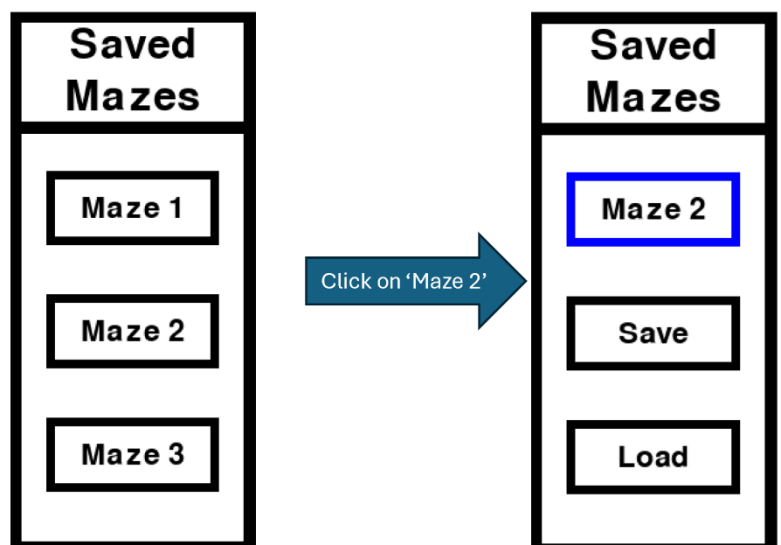
    # when no slot selected, give the user the choice of all 3 slots
    # draw according text
    if saveSlotChoice == False:
        drawText("Maze 1", 1100, 160, True)
        drawText("Maze 2", 1100, 260, True)
        drawText("Maze 3", 1100, 360, True)

    else:

        # when a slot is selected, display the slot number on first button
        # then 'save' and 'load' on 2 lower buttons
        drawText(str("Maze " + str(saveSlotChoice)), 1100, 160, True)
        drawText("Save", 1100, 260, True)
        drawText("Load", 1100, 360, True)
        pygame.draw.rect(window, BLUE, (1030, 130, 140, 60), 7)

    return maze1Button, maze2Button, maze3Button
```

The code here just draws all these rectangles and changes the text when a save slot is clicked on, as shown here. The function also returns the rectangle objects of the three buttons.



Here is the code in the updateWindow function that calls the drawSavedMazeWindow function.

```
if savedMazeWindow == True:

    # draw options window under saved mazes button, returns rectangles for the three mini buttons
    maze1Button, maze2Button, maze3Button = drawSavedMazeWindow(window, mouseX, mouseY, saveSlotChoice)

else:

    # buttons still have to exist if the window isn't open, so just make them unclickable
    maze1Button = pygame.Rect(0, 0, 0, 0)
    maze2Button = pygame.Rect(0, 0, 0, 0)
    maze3Button = pygame.Rect(0, 0, 0, 0)
```

And here is the code that determines when the savedMazeWindow boolean is true.

```
# if window is closed, forget any choice the user made
if savedMazeWindow == False:
    saveSlotChoice = False

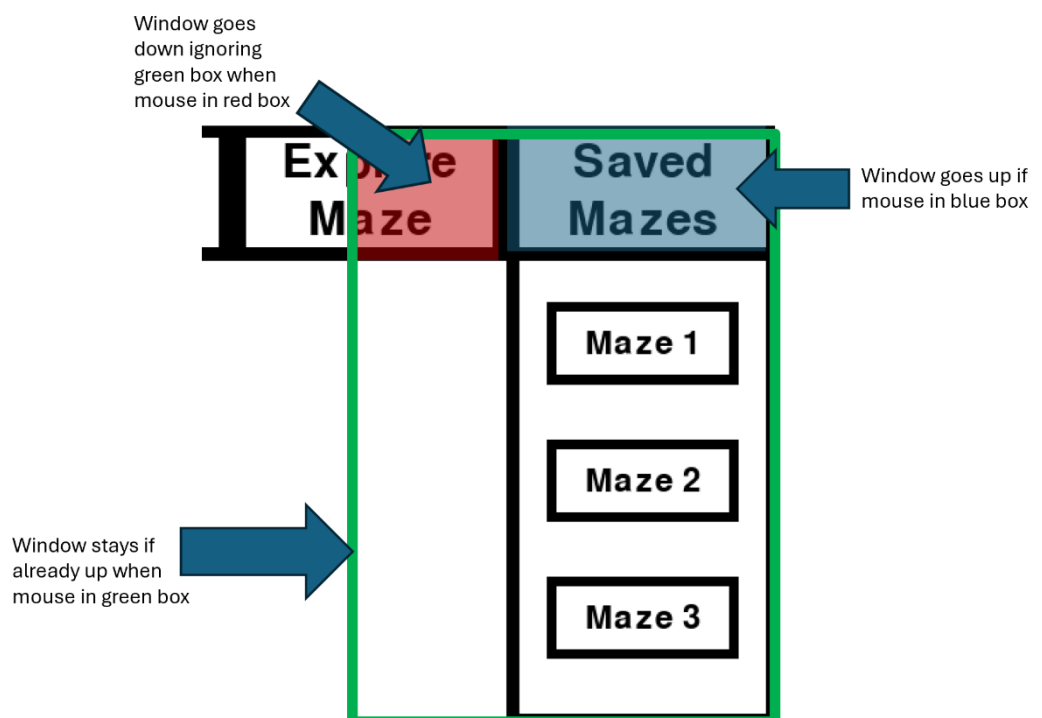
# get rid of save maze window when mouse over explore maze button
if rectList[2].collidepoint(mouseX, mouseY):
    savedMazeWindow = False

# show the window if the mouse is hovering over it
if rectList[3].collidepoint(mouseX, mouseY):
    savedMazeWindow = True

# if window already showing, be lenient on when to hide it
elif savedMazeWindow == True and mouseX >= 900 and mouseY <= 440:
    savedMazeWindow = True

else:
    savedMazeWindow = False
```

Again, here are the conditions specified in the above code for the window being up.



Saved Mazes:

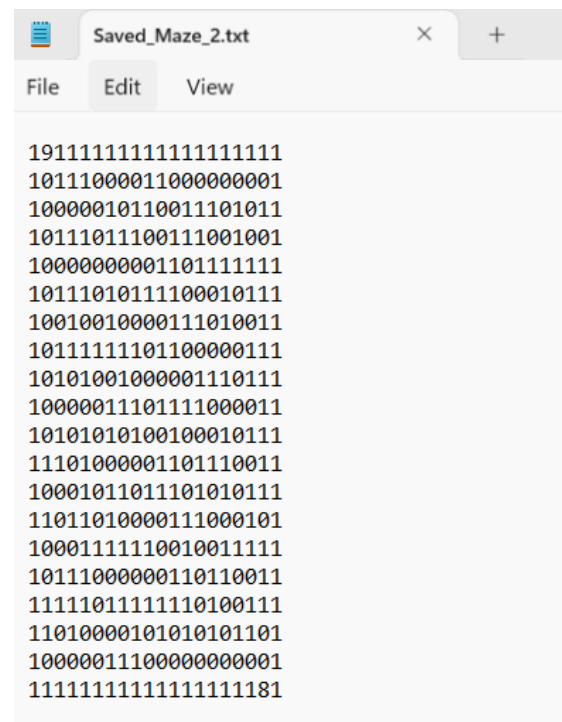
 Saved_Maze_1.txt

 Saved_Maze_2.txt

 Saved_Maze_3.txt

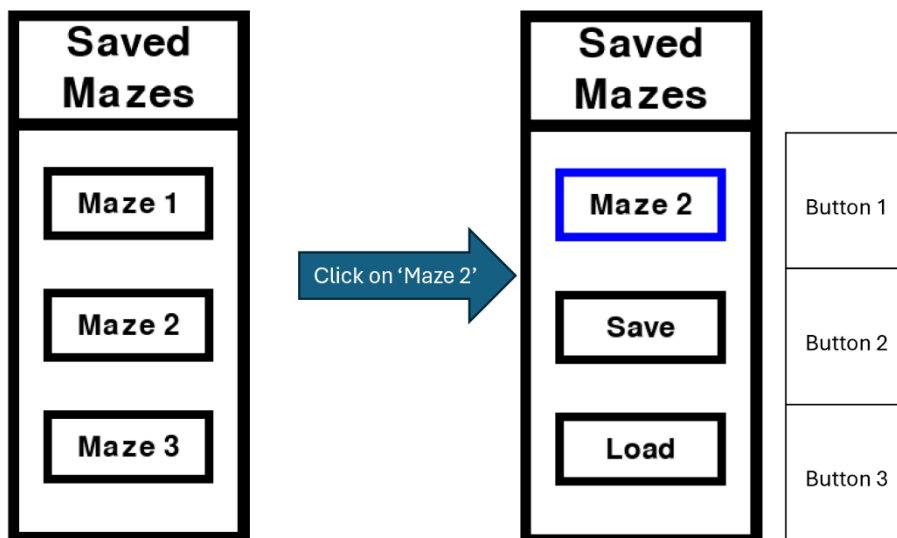
The program uses three text files to save mazes, meaning that the user can save three unique mazes at once. Let's have a look at how the program reads from and writes to these files.

This is what the saved mazes look like as a text file (this example is a 20x20 unit maze). It consists of only 0s and 1s, except the 9 at the start and 8 at the end. Characters have no spaces between them.



```

19111111111111111111
10111000011000000001
10000010110011101011
10111011100111001001
10000000011011111111
10111010111100010111
10010010000111010011
10111111101100000111
10101001000001110111
10000011101111000011
10101010100100010111
11101000001101110011
10001011011110101111
11011010000111000101
10001111110010011111
10111000000110110011
11111011111110100111
11010000101010101101
10000011100000000001
11111111111111111181
  
```



Remember these buttons from the saved maze window? Here, I'll go through the code for each of them.

(I have named them buttons 1 through 3)

Button 1:

This button is the simplest, as it never actually does any saving or loading.

```
while run == True:
    for event in pygame.event.get():
        if clicked == True:
            if rectList[0].collidepoint(mouseX, mouseY) == True:
                elif mazeCreated == True and rectList[1].collidepoint(mouseX, mouseY) == True:
                elif mazeCreated == True and rectList[2].collidepoint(mouseX, mouseY) == True:
                elif savedMazeWindow == True:

                # if top button pressed
                if maze1Button.collidepoint(mouseX, mouseY):
                    mouseX, mouseY = 0, 0    # stops accidental double clicking

                    # if no slot selected, select slot 1
                    if saveSlotChoice == False:
                        saveSlotChoice = 1

                    else:
                        # if slot selected, go back to main menu
                        saveSlotChoice = False
```

When the button is pressed, if no save slot is currently selected, we select save slot 1. If a save slot is currently selected, deselect the current save slot.

Button 2 (Writing to file):

This is the code for button 2. Remember, this will always be the save maze button when a slot has been selected.

```
while run == True:
    for event in pygame.event.get():
        if clicked == True:
            if rectList[0].collidepoint(mouseX, mouseY) == True:
            elif mazeCreated == True and rectList[1].collidepoint(mouseX, mouseY) == True:
            elif mazeCreated == True and rectList[2].collidepoint(mouseX, mouseY) == True:
            elif savedMazeWindow == True:
                if maze1Button.collidepoint(mouseX, mouseY):
                elif maze2Button.collidepoint(mouseX, mouseY):
                    mouseX, mouseY = 1000, 200    # stops accidental double clicking
                    clicked = False

                    # if no slot selected, select slot 2
                    if saveSlotChoice == False:
                        saveSlotChoice = 2

                    # if save slot selected, button 2 is the save button
                    elif mazeCreated == True:

                        # save to slot 1
                        if saveSlotChoice == 1:
                            file = open("Saved_Maze_1.txt", "w")

                        # save to slot 2
                        elif saveSlotChoice == 2:
                            file = open("Saved_Maze_2.txt", "w")

                        # save to slot 3
                        elif saveSlotChoice == 3:
                            file = open("Saved_Maze_3.txt", "w")

                        # get rid of all 2s, 3s and 4s in the maze
                        mazeToSave = maze.copy()
                        mazeToSave = cleanMaze(mazeToSave)

                        for line in mazeToSave:

                            # write each number in the line to file
                            for character in line:
                                file.write(str(character))

                            # start new line in file
                            file.write("\n")

                        # done writing, so close file
                        file.close()
```



Let's break down the code.

To start with, if no slot has yet been selected, select slot 2. (Going from the left to the right menu in the above image.)

```
# if no slot selected, select slot 2
if saveSlotChoice == False:
    saveSlotChoice = 2
```

```
# if save slot selected, button 2 is the save button
elif mazeCreated == True:
```

```
    # save to slot 1
    if saveSlotChoice == 1:
        file = open("Saved_Maze_1.txt", "w")

    # save to slot 2
    elif saveSlotChoice == 2:
        file = open("Saved_Maze_2.txt", "w")

    # save to slot 3
    elif saveSlotChoice == 3:
        file = open("Saved_Maze_3.txt", "w")
```

If a save slot is selected, we first check that a maze has actually been generated. If it has, we open the file corresponding to the selected save slot.

We also have to clear any 2s 3s and 4s from the maze, so that is not saved with the shortest route showing, for example.

```
# get rid of all 2s, 3s and 4s in the maze
mazeToSave = maze.copy()
mazeToSave = cleanMaze(mazeToSave)
```

```
for line in mazeToSave:

    # write each number in the line to file
    for character in line:
        file.write(str(character))

    # start new line in file
    file.write("\n")

# done writing, so close file
file.close()
```

Now, we actually save the maze to the file. And we're done!

Button 3 (Reading from file):

```

while run == True:
    for event in pygame.event.get():
        if clicked == True:
            if rectList[0].collidepoint(mouseX, mouseY) == True:
            elif mazeCreated == True and rectList[1].collidepoint(mouseX, mouseY) == True:
            elif mazeCreated == True and rectList[2].collidepoint(mouseX, mouseY) == True:
            elif savedMazeWindow == True:
                if maze1Button.collidepoint(mouseX, mouseY):
                elif maze2Button.collidepoint(mouseX, mouseY):
                elif maze3Button.collidepoint(mouseX, mouseY):
                    mouseX, mouseY = 1000, 200    # stops accidental double clicking
                    clicked = False

                    # if no save slot selected, choose slot 3
                    if saveSlotChoice == False:
                        saveSlotChoice = 3

                    # if save slot selected, button 3 is load maze button
                    else:

                        # load from slot 1
                        if saveSlotChoice == 1:
                            file = open("Saved_Maze_1.txt", "r")

                        # load from slot 2
                        elif saveSlotChoice == 2:
                            file = open("Saved_Maze_2.txt", "r")

                        # load from slot 3
                        elif saveSlotChoice == 3:
                            file = open("Saved_Maze_3.txt", "r")

                    # sort out some settings
                    mazeCreated = True
                    solveMaze = False
                    exploreMaze = False

                    # start with an empty maze
                    maze = []

                    # lines in the file are a string of numbers, with no characters between
                    for line in file:

                        # list(line) returns a list of all the characters in the line
                        list1 = list(line)

                        # remove the newline character from the end of the list
                        list1.remove(list1[-1])

                        # cast each number from string to integer
                        for i in range(len(list1)):
                            list1[i] = int(list1[i])

                        # append the list to the maze
                        maze.append(list1)

                    # change maze data type to numpy.array
                    maze = np.array(maze)

                    # finished reading, so close file
                    file.close()

                    # get data about maze to draw it to screen
                    mazeWidth, mazeHeight = getWidthHeight(maze)
                    mazeTopLeft = [((windowWidth-(mazeWidth*10))/2), \
                                   |(((windowHeight + 100) - (mazeHeight*10))/2)]

```



There is a little more going on here compared to button 2, as we have to actually change the current maze, as well as doing file handling stuff. Let's have a look at the code.

```

# if no save slot selected, choose slot 3
if saveSlotChoice == False:
    saveSlotChoice = 3
  
```

Similar to the other buttons, if no save slot has been selected, this button selects slot 3.

If a slot has already been selected, the user has just pressed the load maze button.

Determine which file to load based on which save slot they have chosen.

```

# if save slot selected, button 3 is load maze button
else:
  
```

```

    # load from slot 1
    if saveSlotChoice == 1:
        file = open("Saved_Maze_1.txt", "r")
  
```

```

    # load from slot 2
    elif saveSlotChoice == 2:
        file = open("Saved_Maze_2.txt", "r")
  
```

```

    # load from slot 3
    elif saveSlotChoice == 3:
        file = open("Saved_Maze_3.txt", "r")
  
```

```

# sort out some settings
mazeCreated = True
solveMaze = False
exploreMaze = False
  
```

```

# start with an empty maze
maze = []
  
```

Since we are loading a new maze, we want solveMaze and exploreMaze to be turned off by default. We must also set mazeCreated to true.

```
# lines in the file are a string of numbers, with no characters between
for line in file:

    # list(line) returns a list of all the characters in the line
    list1 = list(line)

    # remove the newline character from the end of the list
    list1.remove(list1[-1])

    # cast each number from string to integer
    for i in range(len(list1)):
        list1[i] = int(list1[i])

    # append the list to the maze
    maze.append(list1)

# change maze data type to numpy.array
maze = np.array(maze)

# finished reading, so close file
file.close()
```

Above is the actual reading from file bit. Each line of the file will be in the form "10111010101010" (for example), while each line in the maze needs to be in the form [1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0].

So, we use the list() function. List(line) of the example would return ["1", "0", "1", "1", "1", "0", "1", "0", "1", "0", "1", "0", "1", "0", (newline character)]

We then use the line `list1.remove(list1[-1])` to remove the newline character from the end of each list, before converting all the numbers from string to integer. Now the line is in the correct format, we can simply append it to the maze.

```
# get data about maze to draw it to screen
mazeWidth, mazeHeight = getWidthHeight(maze)
mazeTopLeft = [((windowWidth-(mazeWidth*10))/2), \
                (((windowHeight + 100) - (mazeHeight*10))/2)]
```

The last thing we need to do is set some variables based on the dimensions of the new maze.

ActiveNodes Class:

The final code for the ActiveNodes class is very similar to what was shown in the design section.

```
class ActiveNode():
    def __init__(self, XValue, YValue, parentNode):
        '''
        Initialise each instance of node object
        Takes parameters XValue, YValue and parentNode
        XValue and YValue refer to the node's position in the maze
        parentNode is the node that the node originated from
        parentNode = False for the starting node in the maze
        '''

        self.XValue = XValue
        self.YValue = YValue
        self.parentNode = parentNode

    def findNewNodes(self, maze):
        '''
        Given a node in the maze, find adjacent nodes that can be moved to.
        Returns a list of all valid movements,
        as well as a boolean that holds whether the exit has been found.
        '''

        # locate an adjacent space to move to
        # start by looking at all possible movements

        possibleMovements = [[self.XValue - 1, self.YValue], [self.XValue + 1, self.YValue]\
                               , [self.XValue, self.YValue - 1], [self.XValue, self.YValue + 1]]
        validMovements = []

        foundExit = False # not yet found the exit to the maze
        foundStart = False # not yet found the start of the maze

        for row, column in possibleMovements:
            try:
                if maze[row][column] == 8:
                    foundExit = True # found finish!

                elif maze[row][column] == 9:
                    foundStart = True

                elif maze[row][column] == 2:
                    foundStart = True # spaces connected to the start are marked with a 2

                elif maze[row][column] == 0:
                    validMovements.append([row, column]) # add each valid movement to a list
                    maze[row][column] = 3

            except IndexError: # index error expected when at the edge of the maze
                pass

        return validMovements, foundStart, foundExit
```

```
class ActiveNode():  
    def __init__(self, XValue, YValue, parentNode):  
        '''  
        Initialise each instance of node object  
        Takes parameters XValue, YValue and parentNode  
        XValue and YValue refer to the node's position in the maze  
        parentNode is the node that the node originated from  
        parentNode = False for the starting node in the maze  
        '''  
  
        self.XValue = XValue  
        self.YValue = YValue  
        self.parentNode = parentNode
```

This `__init__` function is just called when each `ActiveNode` is created. Each instance of the class has three attributes:

- `XValue` is the node's horizontal position in the maze
- `YValue` is the node's vertical position in the maze
- `ParentNode` is the node that created the current one. It is set to false for the node at the maze entrance

```
possibleMovements = [[self.XValue - 1, self.YValue], [self.XValue + 1, self.YValue]\  
                    , [self.XValue, self.YValue - 1], [self.XValue, self.YValue + 1]]  
validMovements = []
```

As for the `findNewNodes` function, we first create a list of all the possible nodes that the program could theoretically move to, as well as creating an empty list for the yet to be determined valid movements.

```
foundExit = False    # not yet found the exit to the maze  
foundStart = False   # not yet found the start of the maze
```

Then, set the initial values of `foundExit` and `foundStart` to false.

```
for row, column in possibleMovements:
    try:

        if maze[row][column] == 8:
            foundExit = True    # found finish!

        elif maze[row][column] == 9:
            foundStart = True

        elif maze[row][column] == 2:
            foundStart = True    # spaces connected to the start are marked with a 2

        elif maze[row][column] == 0:
            validMovements.append([row, column])    # add each valid movement to a list
            maze[row][column] = 3

    except IndexError: # index error expected when at the edge of the maze
        pass

return validMovements, foundStart, foundExit
```

This if statement first checks if any of the possibleMovements are the finish node or the start node. It then checks whether any of the possibleMovements are marked by a 2, meaning that the possible movement is connected to the maze entrance, so the node itself is connected to the maze entrance. Otherwise, if the possibleMovement is previously unexplored, it is added to the validMovements list.

The Solving Algorithm:

Finding the Maze Exit:

You'll remember from the analysis section that the solver uses the breadth first exploration algorithm to find the maze exit.

Generally, the way this algorithm works using a list of all the currently considered nodes (active nodes). It then repeatedly:

- Marks all nodes currently in the list as visited
- Removes all visited nodes from the list
- Adds all adjacent unvisited nodes to the list

Until one of the adjacent nodes is the exit, at which point it returns the path taken to get there. Let's have a look at some of the code:

This is the opening of the function. We start by creating an `ActiveNode` at the start of the maze, before adding it to the `ActiveNodes` list.

```
def main(maze):
    startingCoords = findStart(maze)

    activeNodes = [] # list which holds the IDs of all active node objects
    newActiveNodes = [] # temporary list that is used when creating new active node objects

    # get the starting node of the maze
    newNode = ActiveNode(startingCoords[0], startingCoords[1], False)
    activeNodes.append(newNode)
```

Now, for each `ActiveNode` in the list, we first mark that node of the maze as visited, before finding out which nodes are available to traverse to, using the `findNewNodes` function.

The function stops running and returns the node closest to the exit if the `findNewNodes` function finds the exit.

It then adds the coordinates from `validMovements` to the `newActiveNodes` list.

```
run = True
while run == True:
    for node in activeNodes:
        # mark path taken
        maze[node.XValue, node.YValue] = 3

        # find valid spaces to move to
        validMovements, _, foundExit = node.findNewNodes(maze)

        # if exit has been found, we can stop exploring the maze
        # and start looking for the shortest path
        if foundExit == True:
            run = False
            return node

        # move from one space to another
        # add all spaces that are in validMovements list to newActiveNode list
        for coordPair in validMovements:
            newNode = ActiveNode(coordPair[0], coordPair[1], node)
            newActiveNodes.append(newNode)
```

Now, we rotate all items from the newActiveNodes list to the ActiveNodes list. The ActiveNodes list is cleared, and all items from newActiveNodes are added. NewActiveNodes is then cleared, ready for the next iteration of the algorithm.

```
# now that we are done iterating through the activeNodes list,
# we can clear it of all current nodes and add the next lot

# remove all items from activeNodes

activeNodes = []

# add items from newActiveNodes to activeNodes

for item in newActiveNodes:
    activeNodes.append(item)

# clear newActiveNodes

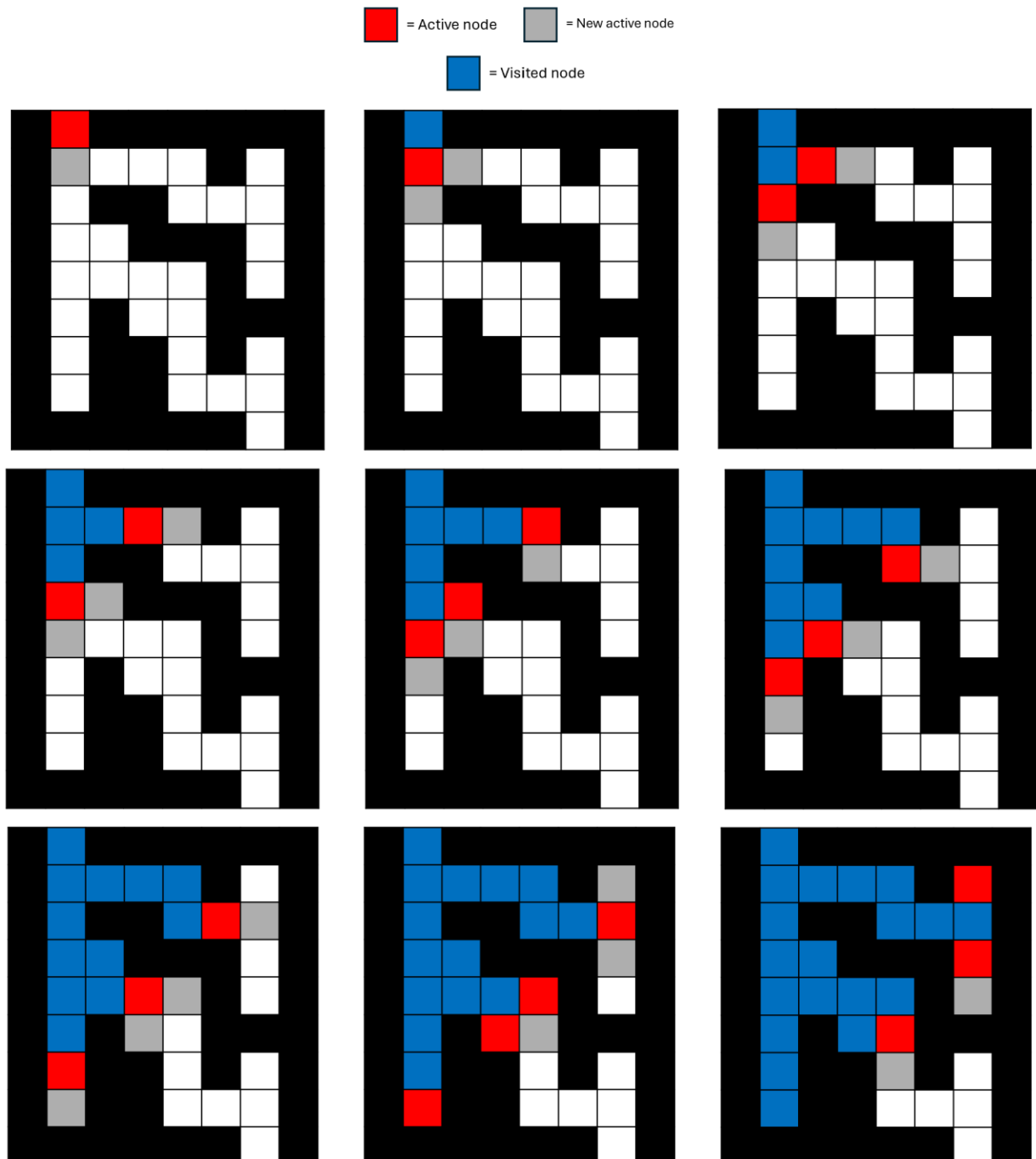
newActiveNodes = []

# allows the user to quit the window while the program is running

for event in pygame.event.get():
    if event.type == pygame.QUIT:
        pygame.quit()
        sys.exit()
```

Visualisation of the Solving Algorithm:

This is what the solving algorithm looks like visually, with different types of node labelled with different colours. You can see it starts at a single point and expands to fill the maze, splitting into 2 whenever it finds an intersection.



This continues until an ActiveNode is adjacent to the maze exit, at which point the route it took is traced back to get the shortest path.

Finding the Shortest Path:

Using the parentNode attribute of the ActiveNode class, it is easy to trace back the route any one node has taken through the maze.

```
def findRoute(finishNode, maze, window):
    run = True
    finishPath = []

    while run == True:

        # add the current node to the path taken
        # then, find the node that that node originated from and repeat

        finishPath.append(finishNode)
        finishNode = finishNode.parentNode

        # when the parent node = False, reached the start of the maze

        if finishNode == False:
            run = False

    # get rid of all the 3s in the maze so that just the finish path is shown
    for x, y in np.argwhere(maze == 3):
        maze[x][y] = 0

    return maze, finishPath
```

All this does is add the finishing node to a list, then repeat with that node's parent node, until the parent node is false, which is only true of the node at the maze entrance. The finished list then contains only the shortest path.

Calling the Solving Algorithm:

Here is the entire code that handles the solving algorithm:

```
while run == True:
    for event in pygame.event.get():
        if clicked == True:
            if rectList[0].collidepoint(mouseX, mouseY) == True:
                elif mazeCreated == True and rectList[1].collidepoint(mouseX, mouseY) == True:
                    optionsWindow = False
                    mouseX, mouseY = 0, 0 # stops accidental double clicking
                    solveMaze = not solveMaze

            if solveMaze == True:
                pygame.draw.rect(window, BLUE, (600, 0, 200, 100), 10) # draw blue box over solve maze button

                # actually call the solving algorithm and get shortest path
                finishNode = mazeSolver.main(maze)
                maze, finishPath = mazeSolver.findRoute(finishNode, maze, window)

                # reveal the shortest path one pixel at a time
                for counter in range(len(finishPath)-1, -1, -1):
                    maze[finishPath[counter].XValue][finishPath[counter].YValue] = 7

                    updateMaze()
                    pygame.display.update()

                # if solveMaze now false, remove all 7s in the maze, hiding the shortest path
            else:
                for y,x in np.argwhere(maze == 7):
                    maze[y][x] = 0
```

Now, let's go through it a bit at a time.

This if statement is triggered if a maze has been created and the solve maze button has been pressed.

```
elif mazeCreated == True and rectList[1].collidepoint(mouseX, mouseY) == True:
    optionsWindow = False
    mouseX, mouseY = 0, 0 # stops accidental double clicking
    solveMaze = not solveMaze
```

It first gets the generation options window out the way, so it's not hiding the solution. It then sets the mouse coordinates to (0, 0) so that the user does not accidentally click on anything during the shortest route reveal. It then switches the value of the boolean solveMaze.

```
if solveMaze == True:
    pygame.draw.rect(window, BLUE, (600, 0, 200, 100), 10) # draw blue box over solve maze button

    # actually call the solving algorithm and get shortest path
    finishNode = mazeSolver.main(maze)
    maze, finishPath = mazeSolver.findRoute(finishNode, maze, window)
```

If solveMaze is now true, we actually call the maze solver, which gives us the node that first reaches the maze exit. We then call the findRoute function, which uses the parentNode attribute of each node to return a list of ActiveNodes along the shortest path.

```
# reveal the shortest path one pixel at a time
for counter in range(len(finishPath)-1, -1, -1):

    maze[finishPath[counter].XValue][finishPath[counter].YValue] = 7

    updateMaze()
    pygame.display.update()
```

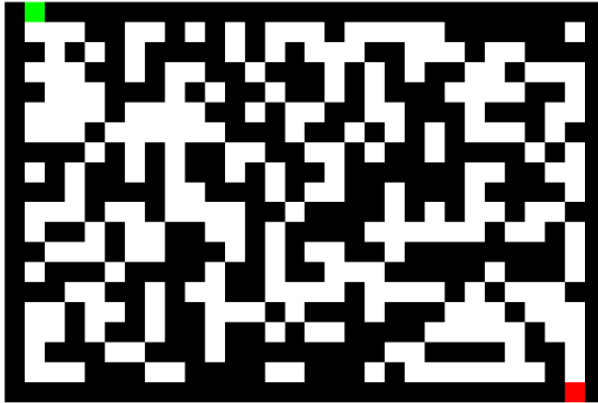
This is the dramatic reveal of the shortest path, one node at a time. We iterate backwards through the finishPath list (as the node at the beginning of the list is at the maze exit). For each node, set the unit at that node's position in the array to a 7. The updateMaze function draws a red pixel whenever it encounters a 7 in the maze. By using a for loop, we alternate between setting a node to 7 and updating the maze, creating a one by one reveal.

```
# if solveMaze now false, remove all 7s in the maze, hiding the shortest path
else:
    for y,x in np.argwhere(maze == 7):
        maze[y][x] = 0
```

Alternatively, if solveMaze has just gone from true to false, this section removes all 7s (and thus the visualisation of the shortest path) from the maze. Although we don't then update the maze here, it will be changed next screen redraw.

The Generation Algorithm:

To recap what we talked about in the analysis section, each maze starts life as a random combination of corridors and walls, possibly looking like this:



A randomly generated grid of corridors and walls.

And the code that does it!

```
# generate a matrix of specified width and height, entirely consisting of '-1's
# these will be replaced shortly

mazeMatrix = np.array([[ -1 for x in range(mazeWidth)] for y in range(mazeHeight)])

# replace all '-1's with a randomly generated 1 or 0

for y, x in np.argwhere(mazeMatrix == -1):
    mazeMatrix[y][x] = random.randint(0, 1)
```

All mazes start as something like the above and then are iteratively improved until they stop changing.

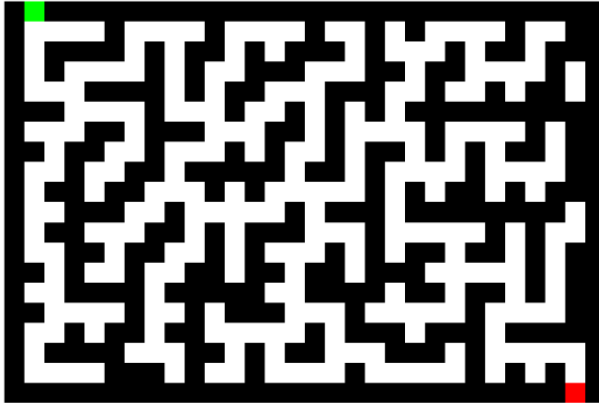
In the analysis section, we also talked about maze rules. These are the ways we iteratively improve the maze. All of the maze rules are called from the same place, in this block of code:

```
mazeMatrix = openContainedSpace(mazeMatrix, mazeWidth, mazeHeight)
mazeMatrix = eliminateTwoBlockCorridors(mazeMatrix, mazeWidth, mazeHeight)
mazeMatrix = eliminateTwoBlockWalls(mazeMatrix, mazeWidth, mazeHeight)
mazeMatrix = eliminateDiagonals(mazeMatrix)
mazeMatrix = removeSinglePixels(mazeMatrix)
```

(The entire maze generator is basically this chunk of code executed over and over with a bit of extra stuff deciding when to stop running and updating the screen.)

A good way to visualise what these functions do is to see what a generated maze looks like without any one of them, which is very easy to do by commenting out lines above. We will have a go at this later, when exploring the individual functions.

For reference, here is a maze generated by the program when all functions are active.



The Lesser Maze Rules:

EliminateTwoBlockWalls:

```
def eliminateTwoBlockWalls(maze, mazeWidth, mazeHeight):
    """
    Takes maze as argument and returns it
    For each 2x2 block of wall, set the bottom right pixel to corridor
    """

    for x, y in np.argwhere(maze == 1):

        # if not out of bounds and pixel to the right = 1
        # and pixel below = 1 and pixel to the right and below = 1

        if x+1 < mazeHeight and y+1 < mazeWidth and maze[x+1][y] == 1 \
            and maze[x][y+1] == 1 and maze[x+1][y+1] == 1:

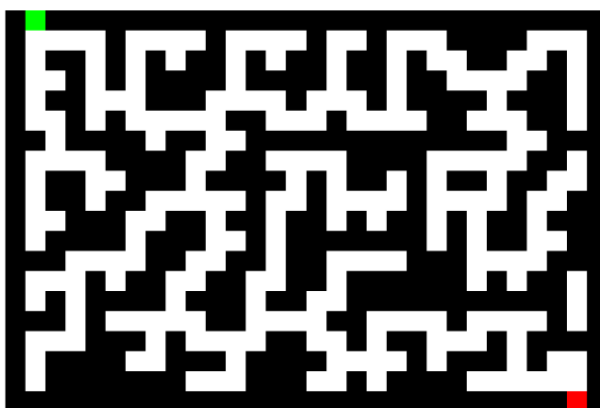
            # set pixel to the right and below to 0
            maze[x+1][y+1] = 0

    return maze
```

This function finds a 2x2 block of wall and switches the lower right pixel to corridor.



And here is an example of a maze without this function.



Removing this function isn't the end of the world. However, all that extra corridor means that there's less actual maze for the user to explore!

RemoveSinglePixels:

Probably the least interesting function, this one is fairly simple.

```
def removeSinglePixels(maze):
    """
    Takes maze as argument and returns it
    For each wall in the maze that has no other walls orthogonally adjacent, add a wall.
    """

    for i, j in np.argwhere(maze == 1):
        try:
            # if pixel to the right = 0 and pixel below = 0
            # and pixel to the left = 0 and pixel above = 0

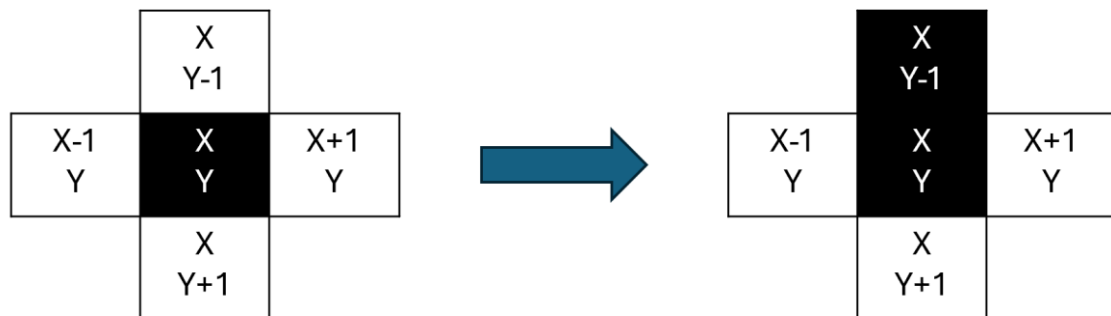
            if maze[i+1][j] == 0 and maze[i][j+1] == 0 \
                and maze[i-1][j] == 0 and maze[i][j-1] == 0:

                # set pixel above to 1
                maze[i][j-1] = 1

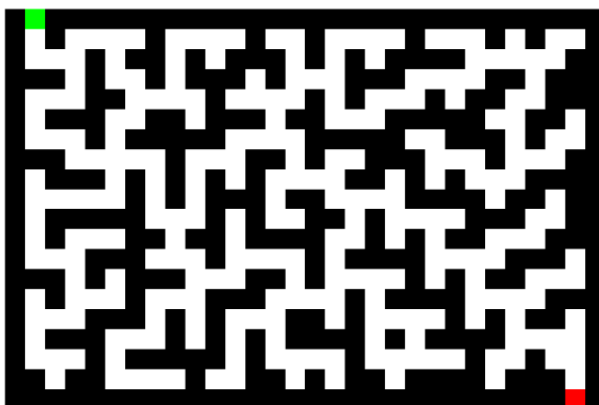
        except IndexError:
            pass # index error expected at the end of each row

    return maze
```

Its workings are summarised with this diagram. If a wall is completely orthogonally isolated, it will add a wall directly above it.



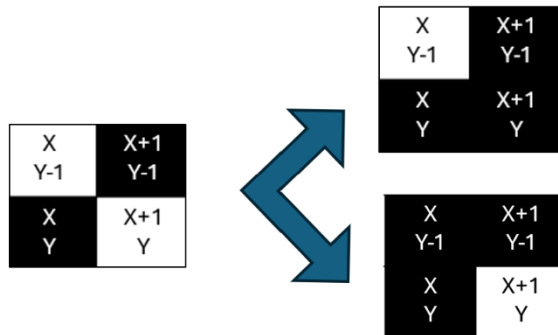
And here is a maze generated without the function. There is actually only one floaty pixel in this example – this function really isn't used much, especially in smaller mazes.



A maze generated without this function looks pretty normal.

Interlude About Random Numbers:

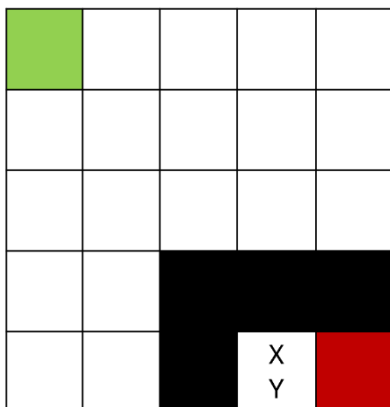
You may have noticed that the above functions always remove the same block, with no random element. However, when we talked about the `eliminateDiagonals` function in the analysis section, I showed this picture:



The `eliminateDiagonals` function was originally going to change the pattern on the left to *one of* the ones on the right.

Originally, I planned for each of these functions to use a random number generator to choose which node to change. However, this meant that the maze being generated would keep changing and never settle, as the functions were just too random and unpredictable. However, the change to non-random numbers does little to hurt the variety of mazes generated, as they all still start with a random layout, so I decided to change all functions except `eliminateTwoBlockCorridors` (to be explained soon) and `openContainedSpace` (to be explained momentarily) to be fundamentally predictable.

We can use the diagram below to understand why the `openContainedSpace` function *must* use random numbers. If it were made predictable like the other functions – let's say it always removed the block to its right – then the tile marked (X, Y) would never get connected to the entrance.



A demonstration of why `openContainedSpace` must remain random

EliminateDiagonals:

This function was covered in the analysis, but let's cover the final version, now with less random numbers!

```
def eliminateDiagonals(maze):
    """
    Each square of wall in the maze checks if the square to the top right
    or the square to the bottom right is purely diagonally connected to it.
    If this is found to be the case, it adds a wall immediately to the right of itself.
    """

    for i, j in np.argwhere(maze == 1):
        try:

            # square to the upper right is purely diagonally connected
            if maze[i+1][j-1] == 1 and maze[i+1][j] == 0 and maze[i][j-1] == 0:

                # set square immediately to the right to a 1
                maze[i+1][j] = 1

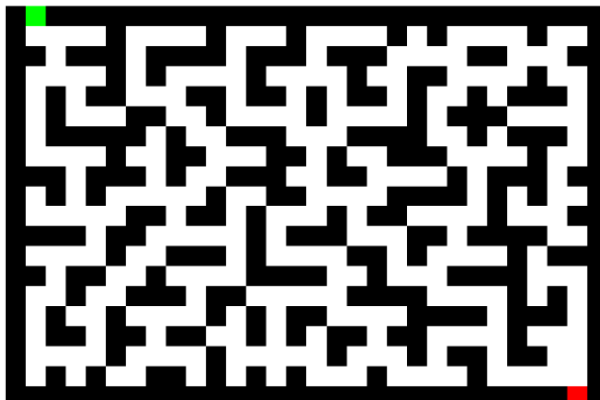
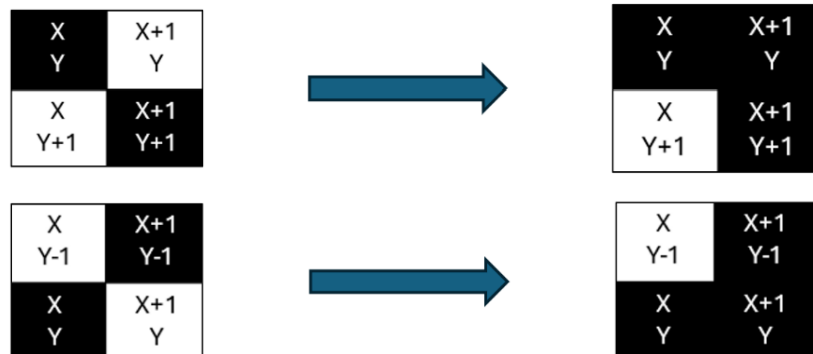
            # square to the lower right is purely diagonally connected
            if maze[i+1][j+1] == 1 and maze[i+1][j] == 0 and maze[i][j+1] == 0:

                # set square immediately to the right to a 1
                maze[i+1][j] = 1

        except IndexError:
            pass # index error expected at the end of each row

    return maze
```

These diagrams show the two patterns that this function looks for, and how it changes them.



This is what a maze generated without eliminateDiagonals would look like. In a way, it's quite nice looking – maybe the user should be able to toggle this function.

EliminateTwoBlockCorridors:

This one is the inverse of eliminateTwoBlockWalls, except it uses some random numbers.

Why Random?

Here is a snippet of a larger maze generated without the randomness in this function:



You can see that the very top line of the maze is strangely empty. This is because across all functions, there is more wall removal along the top row than wall adding (eliminateTwoBlockWalls and openContainedSpace being the main contributors to wall removal). As a consequence, the top row tends towards being empty.

X	X+1
Y	Y
X	X+1
Y+1	Y+1



X	X+1
Y	Y
X	X+1
Y+1	Y+1

A seemingly obvious solution would just be to do this, right? That way the eliminateTwoBlockCorridors function is adding wall to the top row, solving the problem. Let's try it!

The good news is this change fixes the top row. The bad news is this is our new bottom row:



Now we've stopped adding pixels to the bottom row using this function, it has become empty in the same way the top row did. It seems we can have the top row looking good, *or* the bottom row looking good. Which should we choose?

How about both! Oh, the wonders of random numbers.

```
def eliminateTwoBlockCorridors(maze, mazeWidth, mazeHeight):
    '''
    Takes maze as argument and returns it
    For each 2x2 block of corridor, set the bottom right pixel to wall
    '''
```

So, the final version of the function:

```
for y, x in np.argwhere(maze == 0):
    # if not out of bounds and pixel to the right = 0
    # and pixel below = 0 and pixel to the right and below = 0
    if y+1 < mazeHeight and x+1 < mazeWidth and maze[y][x+1] == 0 \
        and maze[y+1][x] == 0 and maze[y+1][x+1] == 0:

        choice = random.randint(1, 2)

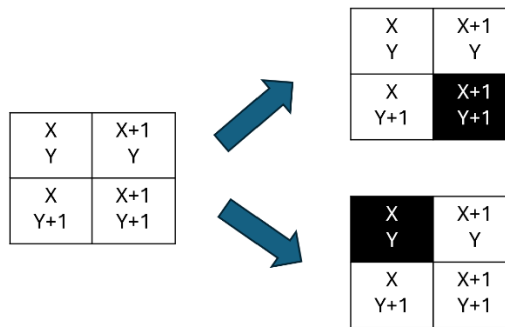
        if choice == 1:
            # set pixel to the lower right to 1
            maze[y+1][x+1] = 1

        elif choice == 2:
            # set pixel (upper left of 2x2) to 1
            maze[y][x] = 1

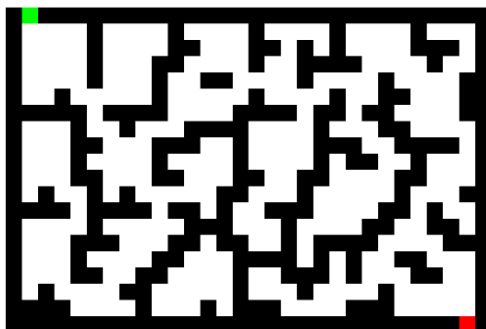
return maze
```

The workings of this function are illustrated in the figure below. It finds the pattern on the left and changes it to one of those on the right.

A visualisation of the eliminateTwoBlockCorridors() function.



Here is what would happen if we temporarily removed this function from the generator:



Without this function, the maze still generates without errors, but there is a lot of empty space.

The OpenContainedSpace Function:

```
def openContainedSpace(maze):
    for x, y where maze == 0:
        # entranceConnectedCheck is a variation on the solving algorithm
        # it returns True if the given coordinates are connected to the maze entrance
        # it returns False otherwise
        connected = entranceConnectedCheck(maze, [x, y])

        if connected == False:
            randomNumber = random.randint(1, 4)

            if randomNumber == 1:
                maze[x+1][y] = 0

            elif randomNumber == 2:
                maze[x][y+1] = 0

            elif randomNumber == 3:
                maze[x-1][y] = 0

            elif randomNumber == 4:
                maze[x][y-1] = 0
```

(Pseudocode from the design section for reference)

The final code for this function is relatively unchanged, just a couple of extra conditions in the If statement to prevent index errors, and... a *shadow* maze?

```
def openContainedSpace(maze, mazeWidth, mazeHeight):
    '''For each unmarked space in the maze, checks if a path exists to the maze entrance'''

    shadowMaze = maze.copy()

    for x, y in np.argwhere(maze == 0):
        shadowMaze, connected = connectedEntranceCheck(shadowMaze, [x, y])

        if connected == False:
            blockToLose = random.randint(1, 4)
            if x+1 < mazeHeight and blockToLose == 1 and maze[x+1][y] == 1:
                maze[x+1][y] = 0
            elif y+1 < mazeWidth and blockToLose == 2 and maze[x][y+1] == 1:
                maze[x][y+1] = 0
            elif x-1 > -1 and blockToLose == 3 and maze[x-1][y] == 1:
                maze[x-1][y] = 0
            elif y-1 > -1 and blockToLose == 4 and maze[x][y-1] == 1:
                maze[x][y-1] = 0

    return maze
```

Shadow Maze:

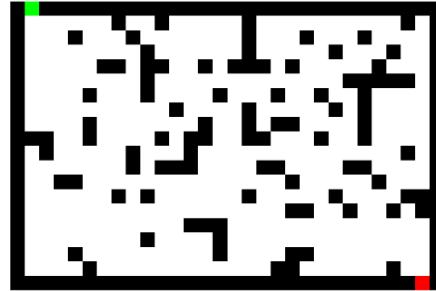
The openContainedSpace function shown just above contains the only references in the code to a 'shadow maze'. It is a copy of the maze that uses all the extra 2s 3s and 4s which we don't want clogging up the main maze when openContainedSpace is the only function that needs them. The best way to understand why we need it is to see how well we do without it.

If I change part of the above code to this, removing all mentions of the shadowMaze and changing them to just the normal maze...

```
#shadowMaze = maze.copy()

for x, y in np.argwhere(maze == 0):
    maze, connected = connectedEntranceCheck(maze, [x, y])
```

We generate a maze looking like this! But why?



Well, the shadowMaze of a smaller maze would look something like this:

```
[1 9 1 1 1 1 1 1 1 1]
[1 1 1 4 4 1 4 4 4 1]
[1 4 4 4 1 1 4 1 4 1]
[1 4 1 4 1 4 4 1 1 1]
[1 4 1 4 1 4 1 1 4 1]
[1 4 1 1 1 4 4 4 1 1]
[1 1 1 4 4 4 1 4 4 1]
[1 4 4 4 1 1 1 1 4 1]
[1 4 1 4 4 4 1 4 4 1]
[1 1 1 1 1 1 1 1 8 1]
```

(You'll notice that all the 0s that would usually represent the corridors in the maze have been replaced by 4s.)

In this version of the maze, functions like `eliminateTwoBlockCorridors` become useless, as all of the corridors that it is looking for are marked with 4s instead of 0s.

However, the continued presence of 1s in the maze means that functions like `eliminateTwoBlockWalls` still work. Many of the walls in the maze are turned to corridors, but the corridors can't be turned to walls, creating a maze like the one shown above.

Hence, we copy the maze (creating the shadow maze) for the sections of the code when extra markings of corridors are needed but continue to use the original maze the rest of the time.

ConnectedEntranceCheck:

This is one function that really needs the maze to contain all those confusing 2s, 3s, and 4s. Like I said in the analysis section, it is fairly similar to the solving algorithm, but with a few optimisations. The solving algorithm only has to run once, but this function is run once for every corridor in the maze, every iteration of the algorithm!

To optimise this, when a node that is *not* connected to the entrance is found, we mark that node with a 4.

```
# if run out of activeNodes without finding the entrance, point is not connected
# mark unconnected pixels with a 4

if activeNodes == []:
    for x, y in np.argwhere(maze == 3):
        maze[x][y] = 4

    run = False
```

And when a node that *is* connected to the entrance is found, it is marked with a 2.

```
# if entrance found, we know that all points currently marked as 3 are connected
# mark connected points with a 2

if connected == True:
    for x, y in np.argwhere(maze == 3):
        maze[x][y] = 2

    run = False
```

Here is the call of findNewNodes in the connectedEntranceCheck function. It should return three variables: validMovements, foundStart and foundExit. However, foundExit is only needed in the maze solver, so we just mark it as an underscore and forget about it. (We have the function return both values to make it flexible between use cases.)

```
# find valid spaces to move to

validMovements, connected, _ = node.findNewNodes(maze)
```

So, because of this snippet of the findNewNodes function in ActiveNodes, nodes marked with 4 are never added as ActiveNodes and if a 2 is found, foundStart is set to true and the connectedEntranceCheck function stops running and returns connected as true.

```
for row, column in possibleMovements:
    try:
        if maze[row][column] == 8:
            foundExit = True    # found finish!

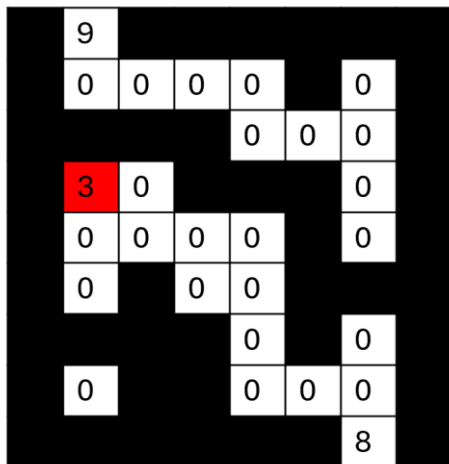
        elif maze[row][column] == 9:
            foundStart = True

        elif maze[row][column] == 2:
            foundStart = True    # spaces connected to the start are marked with a 2

        elif maze[row][column] == 0:
            validMovements.append([row, column])    # add each valid movement to a list
            maze[row][column] = 3

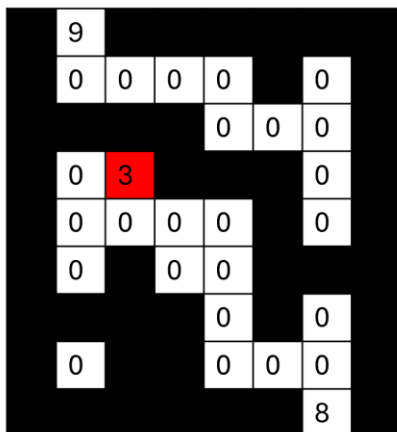
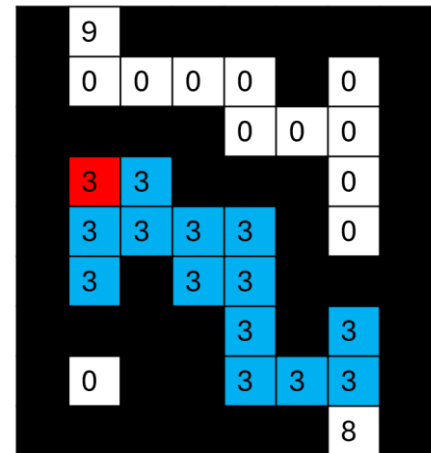
    except IndexError: # index error expected when at the edge of the maze
        pass
```

To give an example of how this function helps speeds things up, let's look at an example without it.



Here, the program wants to find out if the red node marked with a 3 is connected to the maze entrance.

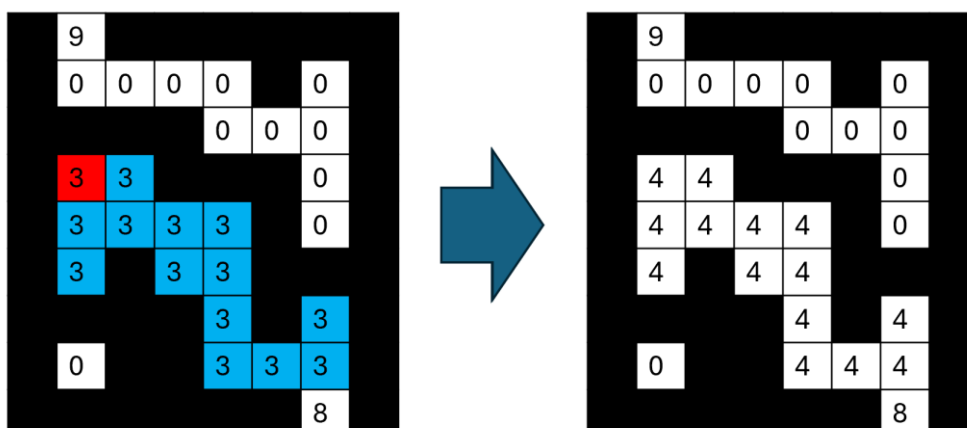
It traverses the graph as far as it can but doesn't find the entrance. Therefore, the node marked in red is not connected to the entrance.

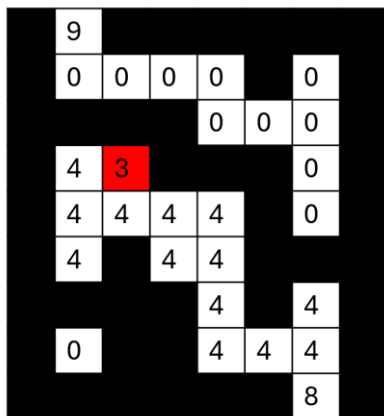


Now, we go again from the next node! Let's run the entire solving algorithm again, shall we?

This seems like an inefficient way to do things.

So, if the graph is discovered to be unconnected, we change all the 3s to 4s. (If it were connected, they'd be changed to 2s instead.)





This means that the next iteration instead looks like this, and ends immediately, as 4s cannot become ActiveNodes.

The addition of this optimisation greatly increases the program's speed – it goes from taking about 5 minutes to create the largest sized maze to taking about 10 seconds.

Calling the Generation Algorithm:

```
while run == True:
    for event in pygame.event.get():
        if clicked == True:

            if rectList[0].collidepoint(mouseX, mouseY) == True:

                mouseX, mouseY = 0, 0    # stops accidental double clicking

                optionsWindow = False
                clicked = False

                if rectList[0].collidepoint(mouseX, mouseY) == True:
                    # clear maze area
                    pygame.draw.rect(window, WHITE, (0, 100, windowWidth, windowHeight - 100))

                    # draw blue rectangle around generate maze button
                    pygame.draw.rect(window, BLUE, (400, 0, 200, 100), 10)

                    # get rid of blue rect around solve maze button
                    pygame.draw.rect(window, BLACK, (600, 0, 200, 100), 10)

                    # get rid of blue rect around explore maze button
                    pygame.draw.rect(window, BLACK, (800, 0, 200, 100), 10)

                    exploreMaze = False
                    solveMaze = False

                    # get width and height from sliders
                    mazeWidth, mazeHeight = [widthSlider.currentValue, heightSlider.currentValue]

                    # work out where to draw the maze from based on its width and height so that it is centred
                    mazeTopLeft = [((windowWidth - (mazeWidth * 10)) // 2), (((windowHeight + 100) - (mazeHeight * 10)) // 2)]

                    # actually generate maze
                    maze = mazeGenerator.main(mazeWidth, mazeHeight, window, mazeTopLeft[0], mazeTopLeft[1])
                    mazeCreated = True
```

To start with, this part of the code is triggered when the mouse button is down (clicked == True), and the mouse collides with the generate maze button (rectList[0].collidepoint...). We then clear any currently visible maze, by drawing a big white rectangle over it.

We don't want exploreMaze or solveMaze modes to be active for a newly generated maze, so those can pretty easily be set to false. However, while the maze generator runs there are no whole screen refreshes, so the solve maze and explore maze buttons remain blue while the next maze is generating!



This isn't a huge problem as they turn back to black as soon as generation is complete, but it is still wrong. This is why we redraw the black rectangles in the above code before running the generator.

Maze Exploration:

This is the full code to handle the user's exploration of the maze. It could be improved by using a dictionary to get each opposite movement... but it works!

```
def handleUserExploration(maze, exploreCoords, pastExploreList, exploreDirectionStack):
    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_UP:
            # user tries to move upwards

            if maze[exploreCoords[1]-1][exploreCoords[0]] != 1 and exploreCoords[1]-1 >= 1: # move is valid

                # if last movement was down, undo previous move
                if exploreDirectionStack.peek() == "down":
                    exploreDirectionStack.pop()

                    exploreCoords = [exploreCoords[0], exploreCoords[1]-1]
                    pastExploreList.remove(exploreCoords)

                # else, add new movement to stack and change exploreCoords
                else:
                    pastExploreList.append(exploreCoords)
                    exploreDirectionStack.push("up")

                    exploreCoords = [exploreCoords[0], exploreCoords[1]-1]

        if event.key == pygame.K_DOWN:
            # user tries to move downwards

            if maze[exploreCoords[1]+1][exploreCoords[0]] != 1 and exploreCoords[1]+2 < mazeHeight: # move is valid

                # if last movement was up, undo previous move
                if exploreDirectionStack.peek() == "up":
                    exploreDirectionStack.pop()

                    exploreCoords = [exploreCoords[0], exploreCoords[1]+1]
                    pastExploreList.remove(exploreCoords)

                # else, add new movement to stack and change exploreCoords
                else:
                    pastExploreList.append(exploreCoords)
                    exploreDirectionStack.push("down")

                    exploreCoords = [exploreCoords[0], exploreCoords[1]+1]

        if event.key == pygame.K_LEFT:
            # user tries to move left

            if maze[exploreCoords[1]][exploreCoords[0]-1] != 1: # move is valid

                # if last movement was right, undo previous move
                if exploreDirectionStack.peek() == "right":
                    exploreDirectionStack.pop()

                    exploreCoords = [exploreCoords[0]-1, exploreCoords[1]]
                    pastExploreList.remove(exploreCoords)

                # else, add new movement to stack and change exploreCoords
                else:
                    pastExploreList.append(exploreCoords)
                    exploreDirectionStack.push("left")

                    exploreCoords = [exploreCoords[0]-1, exploreCoords[1]]

        if event.key == pygame.K_RIGHT:
            # user tries to move right

            if maze[exploreCoords[1]][exploreCoords[0]+1] != 1: # move is valid

                # if last movement was left, undo previous move
                if exploreDirectionStack.peek() == "left":
                    exploreDirectionStack.pop()

                    exploreCoords = [exploreCoords[0]+1, exploreCoords[1]]
                    pastExploreList.remove(exploreCoords)

                # else, add new movement to stack and change exploreCoords
                else:
                    pastExploreList.append(exploreCoords)
                    exploreDirectionStack.push("right")

                    exploreCoords = [exploreCoords[0]+1, exploreCoords[1]]

    return maze, exploreCoords, pastExploreList, exploreDirectionStack
```

The above code consists of four parts, one for each arrow key. Each just checks the top item of the stack to see if it has to undo the previous move, and if not, adds the arrow key pressed to the stack.

```
if event.key == pygame.K_UP:
    # user tries to move upwards

    if maze[exploreCoords[1]-1][exploreCoords[0]] != 1 and exploreCoords[1]-1 >= 1: # move is valid

        # if last movement was down, undo previous move
        if exploreDirectionStack.peek() == "down":
            exploreDirectionStack.pop()

            exploreCoords = [exploreCoords[0], exploreCoords[1]-1]
            pastExploreList.remove(exploreCoords)

        # else, add new movement to stack and change exploreCoords
        else:
            pastExploreList.append(exploreCoords)
            exploreDirectionStack.push("up")

            exploreCoords = [exploreCoords[0], exploreCoords[1]-1]
```

This is the code for the stack:

```
class Stack():
    def __init__(self): # initialise the stack and the location of the top pointer
        self.stack = []
        self.topOfStackPointer = -1

    def __str__(self): # return the contents of the stack if it is printed
        return str(self.stack)

    def push(self, item): # add an item to the stack and increment the top pointer
        self.stack.append(item)
        self.topOfStackPointer += 1

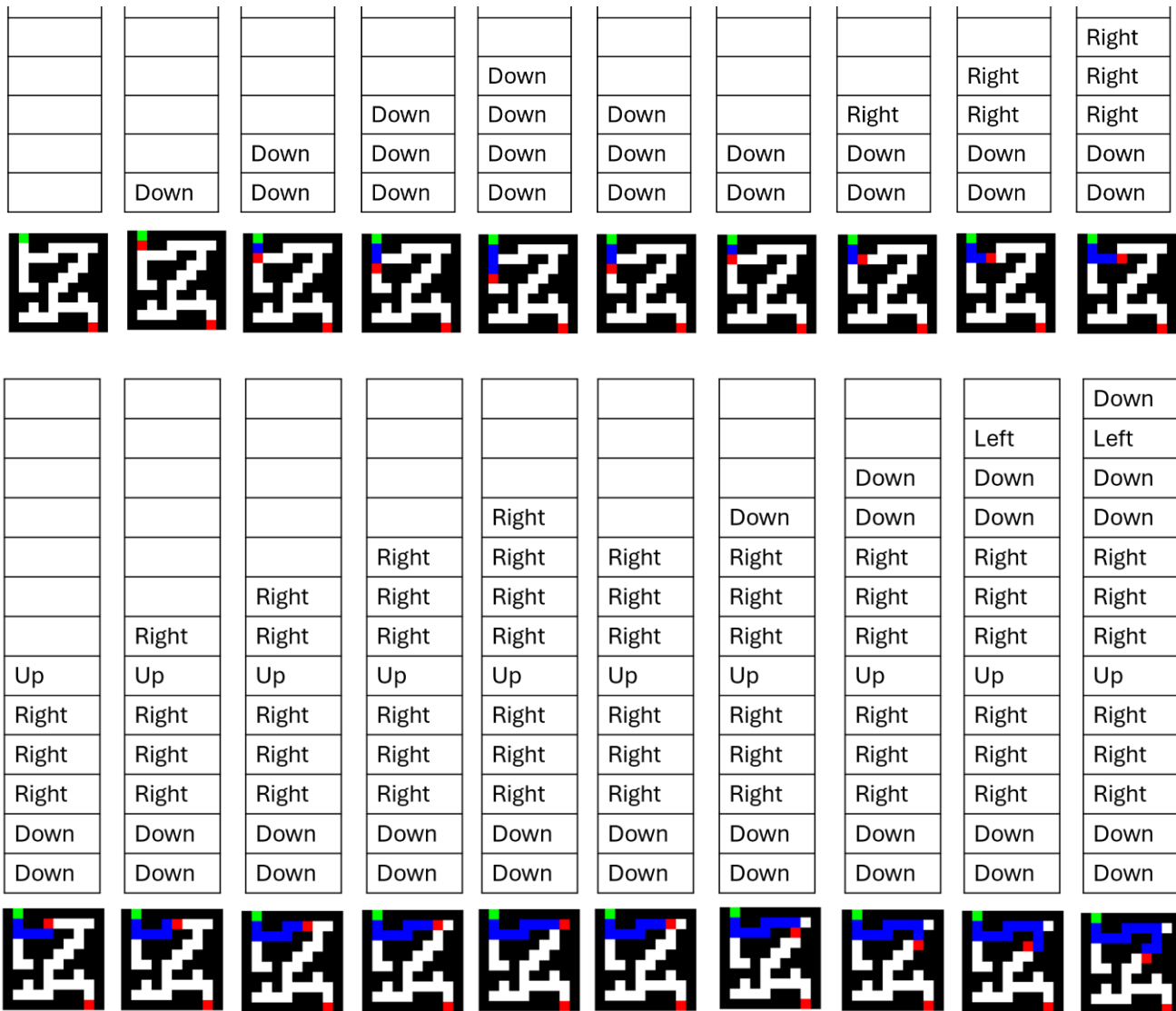
    def pop(self): # remove the item closest to the top of the stack and return it
        # also decrease top of stack pointer by 1
        if self.topOfStackPointer > -1:
            data = self.stack[self.topOfStackPointer]
            del self.stack[-1]
            self.topOfStackPointer -= 1

        else: # if the stack pointer = -1, the stack is empty
            print("No item to pop")
            data = False

        return data

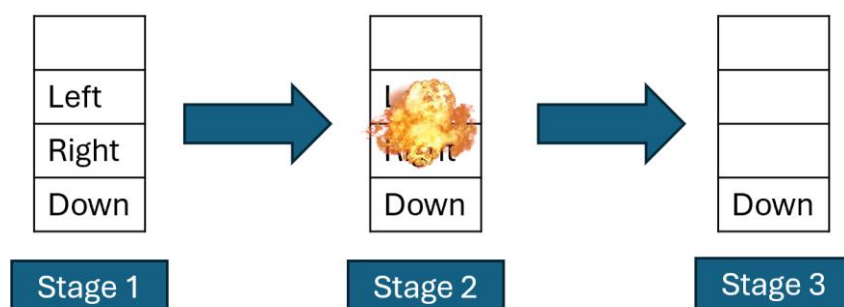
    def peek(self): # return the item at the top of the stack without removing it
        if self.topOfStackPointer > -1:
            data = self.stack[self.topOfStackPointer]
        else:
            data = False # if stack empty, return false

        return data
```



In these diagrams, you can see how the use of the stack allows the user to retrace their steps by recording their past movements.

Effectively, whenever there are two opposite movements next to each other in the stack (left and right, or up and down), they cancel each other out. (This isn't exactly how the code works, but it's a good way to understand it.) I like to think of it looking like this:



So, if the user moves down, right and then left, they end up in the same place as they would if they had just moved down.

The way we call the `handleUserExploration` function is fairly straightforward.

```
if clicked == True:
    if rectList[0].collidepoint(mouseX, mouseY) == True:
    elif mazeCreated == True and rectList[1].collidepoint(mouseX, mouseY) == True:
    elif mazeCreated == True and rectList[2].collidepoint(mouseX, mouseY) == True:

        exploreCoords = [1, 0]
        pastExploreList = []

        exploreMaze = not exploreMaze
```

If the user clicks on `rectList[2]` (which is the explore maze button), we wipe the memory of the user's past explorations and flip the `exploreMaze` boolean.

```
if exploreMaze == True:
    maze, exploreCoords, pastExploreList, exploreDirectionStack = \
        handleUserExploration(maze, exploreCoords, pastExploreList, exploreDirectionStack)
```

Elsewhere in the code, we have this lovely long function call. The if statement here just calls the `handleUserExploration` function when the `exploreMaze` boolean is set to true.

Group A Techniques Used:

Technique	Description	Link
Stack operations	A stack is used to record which nodes in the maze have been explored by the user. The push, peek and pop stack operations are used.	Maze Exploration
Graph traversal	The maze solver (and by extension the maze generator) uses the breadth first traversal algorithm, treating the maze as a graph and each square unit in the maze as a node.	Finding the Maze Exit
Files organised for direct access	There are three distinct file slots that the user can save mazes to or load mazes from. They are accessed from buttons in the menu.	Saved Mazes
Complex user defined algorithms	Maze generator, maze solver.	The Generation Algorithm, The Solving Algorithm
Dynamic generation of objects	Active node objects are constantly being created as the maze generates and solves.	

Testing:

I have recorded a video separately to this documentation, which fully tests that all the functions of the program work as intended. In this table, I go through each objective and describe any tests performed that are linked to that objective, providing a video timestamp.

Objective Number	Objective	Type of User Input	Expected Test Outcome	Pass/Fail	Video Timestamp
1	Maze Generator				0:30
1a	Add generate maze button in the UI bar.	Normal	On program start, the generate maze button is present at the top of the screen.	Pass	0:30
1ai	Allow user to adjust maze width and height of maze in a separate menu.	Normal	When the generate maze button is hovered over, the menu appears.	Pass	0:31
		Normal	User can move mouse a little way off the menu box before it disappears.	Pass	0:37
1aii	Include a generate maze button in this menu	Normal	When generate maze button is pressed, it generates a maze of specified width and height.	Pass	0:58
1b	Allow user to specify width and height of maze		Same as below	Pass	0:42
1bi	Use sliders so user can give width and height	Normal	User can adjust width and height of maze using sliders.	Pass	0:42
1bii	Have an upper and lower bound for the size of the maze	Boundary	Width slider stays within range 10-120 pixels, height slider within range 10-60.	Pass	0:46
1c	Use an algorithm to generate maze	Normal	An algorithm generates a maze.	Pass	0:59
1d	Show the maze on screen as it generates	Normal	The maze is shown on screen as it goes through stages of generation.	Pass	0:59
1e	Let the user input a seed, where inputting the same seed twice will generate the same maze.	Incomplete		Fail	
2	Ability for user to explore maze				3:56
2a	Add explore maze button in the UI bar.	Normal	On program start, the explore maze button is present at the top of the screen.	Pass	3:56
2ai	When explore maze button pressed, enter explore maze mode	Erroneous	Explore maze button cannot be pressed when the program has just started up, i.e. no maze present.	Pass	3:59

2aii	Visually indicate on the button when explore maze mode is active	Normal	Explore maze button turns blue when pressed.	Pass	4:11
2aiii	If button pressed again, exit explore maze mode and reset user's solution	Normal	When explore maze button pressed a second time, solution disappears.	Pass	5:38
2b	When in explore maze mode, arrow keys used to move around the maze	Normal	When explore maze mode active (indicated by blue explore maze button), user can explore maze with arrow keys.	Pass	4:17
2bi	Path taken by user shown	Normal	Path previously taken by the user indicated in blue.	Pass	4:18
2bii	User's current location indicated	Normal	User's current location indicated in red.	Pass	4:18
2c	Display a congratulations when the user reaches the end of the maze	Incomplete		Fail	
3	Maze solver				1:54
		Erroneous	Solve maze button cannot be pressed when no maze present.	Pass	1:57
3a	Add maze solving button to UI bar.	Normal	On program start, the solve maze button is present at the top of the screen.	Pass	1:54
3ai	When button pressed, reveal maze solution.	Normal	The maze solution is revealed once the solve maze button is pressed.	Pass	2:23
3aii	Press button a second time to hide maze solution.	Normal	The shortest route is hidden upon a second button press.	Pass	5:38
3b	Implement breadth first search algorithm to find the maze exit.	Normal	The program used breadth first search to solve the maze. (Visualized in separate demo.)	Pass	2:46
3c	When exit found reveal the shortest route.	Normal	The shortest route through the maze is shown.	Pass	2:23
3ci	Reveal one space at a time for cool animation effect.	Normal	The solution is revealed one node at a time.	Pass	2:23
4	Ability for user to save mazes to a file				5:44
4a	Add save maze button to the menu bar.	Normal	On program start, the save maze button is present at the top of the screen.	Pass	5:44
4ai	Reveals save/load maze button when clicked.	Normal	The saved mazes menu, which lets the user choose a save slot is revealed when the user hovers over the saved mazes button	Pass	5:44

		Normal	The window stays up if the user moves the mouse a little off it.	Pass	5:50
4b	Create a file to save maze in	Normal	If a file is not found already, it is created.	Pass	6:25
4bi	Saving a second maze overwrites the file	Normal	Successfully saving a maze to a file overwrites whatever was last in the file.	Pass	6:25
		Erroneous	Saving to a slot before a maze is generated does not overwrite the previous maze.	Pass	6:45
4bii	Use multiple files for the ability to save multiple mazes.	Normal	There are three different save slots.	Pass	5:50
5	User Interface				Always present
5a(i-iv)	Menu bar at the top of user interface has buttons for each action user can take.	Normal	All buttons appear on program launch.	Pass	0:00
5av	When hovering over each button, it changes colour	Normal	Buttons turn yellow when hovered over (except if active).	Pass	Happens constantly e.g. 0:30
5avi	Buttons change colour when active.	Normal	Buttons turn blue if active.	Pass	e.g. 1:07
5b	Dropdown menus when some buttons are pressed.	Normal	Dropdown menus appear when generate maze and saved mazes buttons are hovered over.	Pass	0:31, 5:44
5c	Until first maze generated, stop all actions except generate maze or read maze from file.	Erroneous	Mentioned in previous tests	Pass	1:57, 3:59, 6:45

Evaluation:

Objective Completeness:

To start off the evaluation, let's revisit all the objectives from the analysis section, to make sure all (or most) of them are complete. This table is very similar to the completeness of solution table (from the technical solution section), but links to the relevant section in the technical solution instead of the code in the appendix.

Objective Number	Objective	Objective Complete?	Link to Technical Solution Section
1	Maze Generator		
1a	Add generate maze button in the UI bar.	The generate maze button is the first one in the bar.	Buttons
1ai	Allow user to adjust maze width and height of maze in a separate menu.	The user can hover over the generate maze button to show the menu	Generation Options Window
1aii	Include a generate maze button in this menu	The user can press the generate maze button to generate the maze	Calling the Generation Algorithm
1b	Allow user to specify width and height of maze	The user can specify the width and height of the maze generated.	Sliders!
1bi	Use sliders so user can give width and height	There is a width slider and a height slider in the pop up menu.	Sliders!
1bii	Have an upper and lower bound for the size of the maze	The sliders can only slide between the values 10 and 120 for width, and 10 and 60 for height.	Sliders!
1c	Use an algorithm to generate maze	Uses an iterative improvement(?) algorithm to generate maze	The Generation Algorithm
1d	Show the maze on screen as it generates	When the generate maze button is pressed, a constantly shifting maze is shown, which eventually settles into the final maze.	The Generation Algorithm
1e	Let the user input a seed, where inputting the same seed twice will generate the same maze.	This wouldn't be hard to do if I had more time, the main challenge being where in the UI to put the seed input. However, the ability to save mazes provides a similar effect of being able to bring back mazes that you like.	
2	Ability for user to explore maze		
2a	Add explore maze button in the UI bar.	The explore maze button is the third button along in the user interface bar.	Buttons
2ai	When explore maze button pressed, enter explore maze mode	Whenever a fully generated maze is on the screen, the user can enter explore maze mode by pressing the relevant button.	Maze Exploration
2aii	Visually indicate on the button when explore maze mode is active	The button turns blue when explore maze mode is active.	Maze Exploration

2aiii	If button pressed again, exit explore maze mode and reset user's solution	The user can press the explore maze button a second time to exit explore maze mode, causing the button to cease being blue. This also resets their solution.	Maze Exploration
2b	When in explore maze mode, arrow keys used to move around the maze	The user can freely navigate the maze using the arrow keys when in explore maze mode.	Maze Exploration
2bi	Path taken by user shown	The path taken by the user is shown in blue	Maze Exploration
2bii	User's current location indicated	The user's current location is indicated with a red pixel	Maze Exploration
2c	Display a congratulations when the user reaches the end of the maze	Not included, so as not to stifle the intrinsic joy of completing the challenge. (I didn't have time.)	
3	Maze solver		
3a	Add maze solving button to UI bar.	The solve maze button is the second one along in the bar.	Buttons
3ai	When button pressed, reveal maze solution.	The solve maze button solves the maze, revealing the shortest path from the maze entrance to the maze exit.	Calling the Solving Algorithm
3aii	Press button a second time to hide maze solution.	Similar to the explore maze button, the maze solution can be hidden by pressing the button a second time.	Calling the Solving Algorithm
3b	Implement breadth first search algorithm to find the maze exit.	The maze solver uses the breadth first search algorithm, treating the maze as an unweighted graph.	Finding the Maze Exit
3c	When exit found reveal the shortest route.	The shortest route, once found, is displayed to the screen.	Finding the Shortest Path
3ci	Reveal one space at a time for cool animation effect.	The shortest route is revealed one node at a time.	Calling the Solving Algorithm
4	Ability for user to save mazes to a file		
4a	Add save maze button to the menu bar.	The saved mazes button is the fourth and final button on the menu bar.	Buttons
4ai	Reveals save/load maze button when clicked.	When hovering over the button, the saved mazes window is revealed.	Saved Maze Window
4b	Create a file to save maze in	There are three files that the user can save and load mazes from, allowing multiple mazes to be kept at once.	Saved Mazes
4bi	Saving a second maze overwrites the file	If a maze is saved to a file, and the user saves another maze to that same file, the first maze is overwritten.	Saved Mazes
4bii	Use multiple files for the ability to save multiple mazes.	As previously stated, there are three files to save mazes in.	Saved Mazes
5	User Interface		
5a(i-iv)	Manu bar at the top of user interface has buttons for each action user can take.	See objectives 1a, 2a, 3a and 4a.	Buttons

5av	When hovering over each button, it changes colour	Buttons turn yellow when hovered over with the mouse.	Buttons
5avi	Buttons change colour when active.	Buttons turn blue when they are active, e.g. solve maze button is blue when maze solution showing.	Buttons
5b	Dropdown menus when some buttons are pressed.	'Generate maze' and 'saved mazes' buttons have dropdown windows when hovered over.	Pop-up Menus
5c	Until first maze generated, stop all actions except generate maze or read maze from file.	The user can't use the solve maze or explore maze buttons when no maze is present.	Calling the Solving Algorithm

End User Feedback:

Conversation:

I showed the finished project to my end user and asked her a few questions. Here is our discussion:

Q: What did you think of the project?

A: I really like it! The maze generator is really fun to watch, even if it looks a bit like a QR code. The solving is great, and it's awesome that you can explore the maze yourself, unlike all the programs from earlier. It definitely has the aesthetics of that first one you showed me ([mazesolver](#)) but obviously the maze generation is way better, plus it has more extra features like the exploration and saved mazes.

Q: How does it compare to those programs from before was going to be my next question! Ok then, what did you think of the user interface?

A: I guess if anything it was a little bare. But on the other hand, that makes it super easy to understand. Oh and it's nice to have the colour on the buttons when you hover over and press them. Without that, it would definitely have been too bland and black and white.

Q: Is there anything that needs improving?

A: The only thing I'd maybe say, as cool as it looks, the generation algorithm? Often, the mazes are a little too open I guess, meaning that the solution is fairly straight through the middle. It would be nice if there was more backtracking if that makes sense.

Q: Ok, anything that you'd want to see being added?

A: I can't really think of much to be honest. You seem to have hit all the main maze related things, which is great!

Reflection:

I have to admit, I do agree with my end user about the maze generation algorithm. I suspect it's a flaw with the unconventional 'iterative' generation algorithm I have used, although I have thought of a few things that could improve it which I haven't had time to attempt.

The user interface too, I can concede does look a little bland. But it's functional, and very intuitive, so I don't really want to confuse it by making it look pretty.

Apart from this, all of the feedback seems pretty positive, especially her saying 'I can't really think of much' to add to the program!

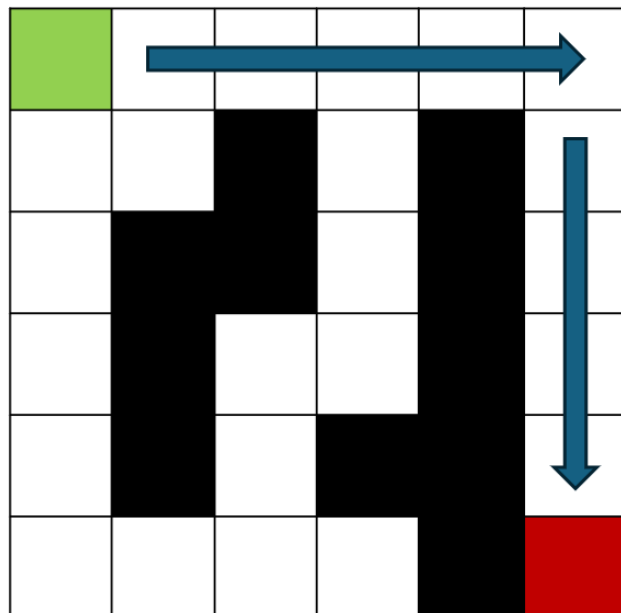
Improvements:

Improving the Generation Algorithm:

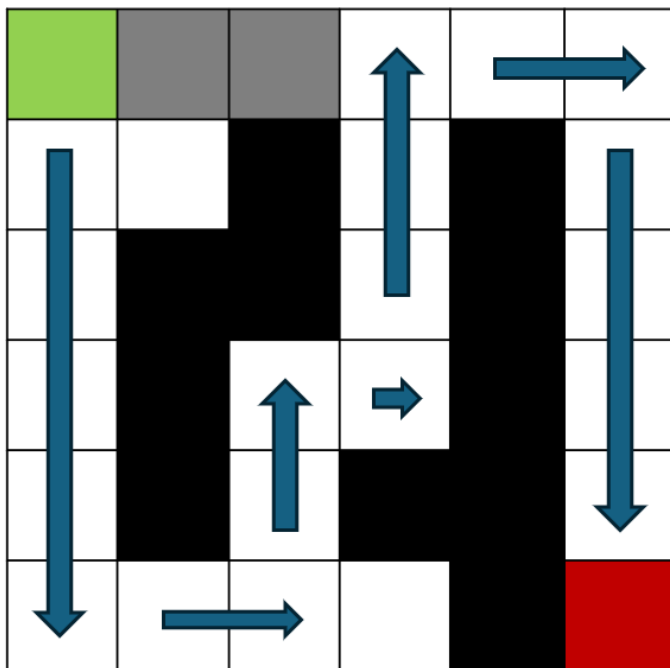
I mentioned after my end user interview just now that I have thought of a few ways to improve the maze generation algorithm. Here I'll go into that a little more.

The problem is, the algorithm doesn't create perfect mazes (that is, mazes with only one path from the entrance to the exit). So we have the issue that the shortest route is often fairly straightforward. The thing is, there is probably a far more complicated route from the entrance to the exit (which I have found many times while testing the explore maze mode).

Let's illustrate my idea with an example. This maze on the right is really quite simple, right? Pretty obviously, the shortest route is the one marked with the arrows.



That's a pretty rubbish maze, though. So let's apply a hypothetical function, `lengthenShortestRoute()`. What this function does is it finds the shortest path through the maze given as a parameter and adds a random wall somewhere along that route. However it will only add a wall to a location that will keep the maze solvable. So in this example, it could add a wall to either of the squares marked in grey below, either of which would change the shortest route to something much more interesting.



And just by adding a single block of wall, we've more than doubled the length of the shortest path!

This is also now a perfect maze.

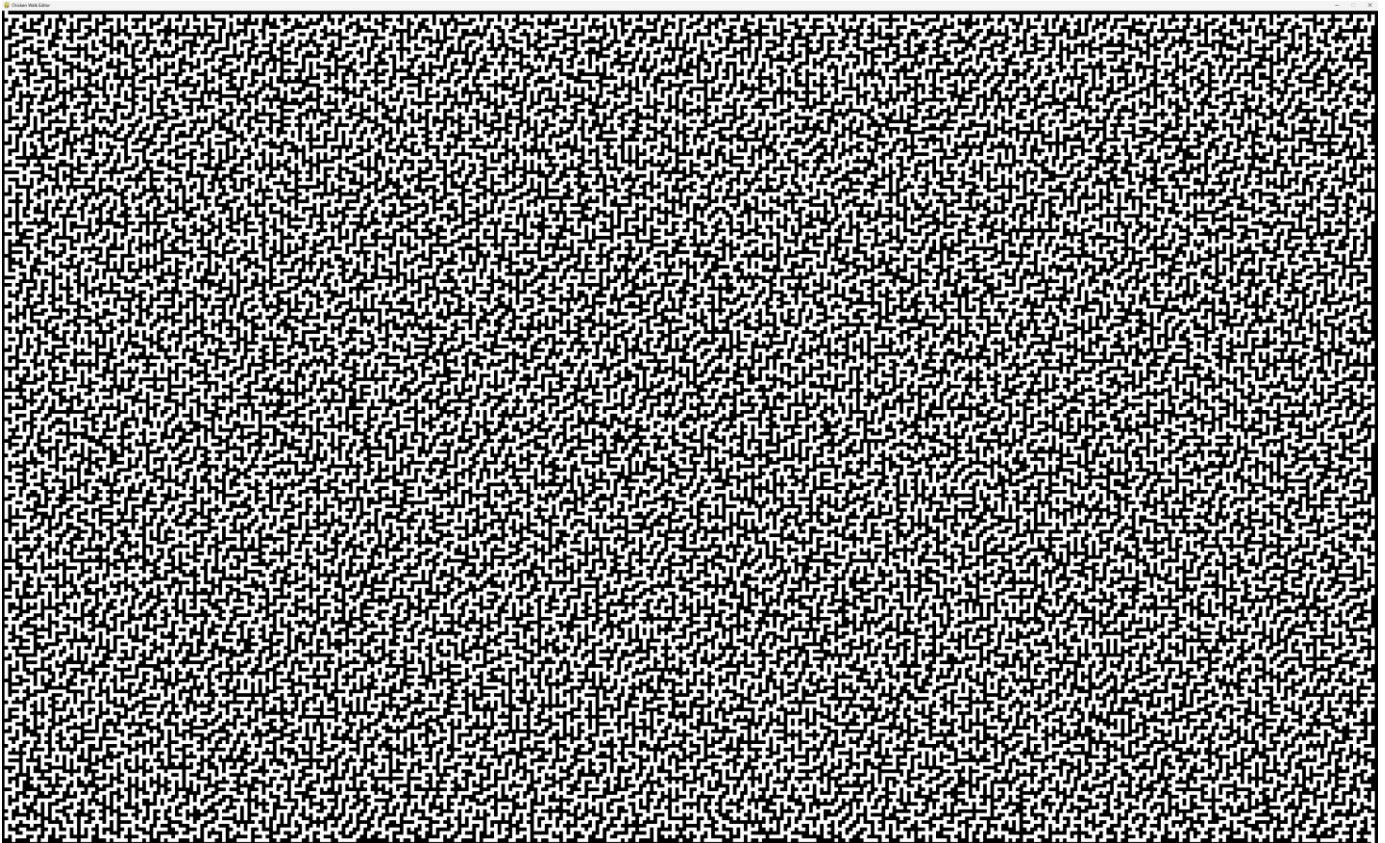
In a much larger maze, this function would need to be applied many times before a perfect maze was found. It would also have to be applied in sync with the other maze rule functions, as adding wall blocks could create illegal formations. (Like adding the leftmost grey block above, which would create an unconnected diagonal).

So obviously, implementation of this function would be difficult, but I do feel it would greatly improve the quality of mazes generated if I could get it to work. It's definitely something I'd like to try in the future!

Variable Window Size:

I know that my end user said that I didn't need to add anything, but I feel she was thinking more about the contents of the window, rather than the window itself. The window is currently set to 1200x700 pixels (just smaller than my laptop screen), meaning that the maximum sized maze that can be generated is 120x60 (each maze block being 10 pixels, minus 100 pixels height for the menu). However, there is theoretically no limit on the size of maze that can be generated.

For reference, this, the biggest maze that I have ever generated. It took about an hour.



So, what if somebody had a monitor the size of a large TV and wanted to really push the maze generator to its limits, but were stuck with a 1200x700 pixel window size?

I'd have to first get the size of the screen that the program was launched on, which is pretty easy. The hard part is then changing basically all the coordinate based user interface code to use variable values of width and height. So for example the max value of the width slider, instead of just being 120, would be `windowWidth//10`.

Final Project Reflection:

Programming this project was great fun! It was a great opportunity to practice using more advanced aspects of programming which I don't have as much experience with. It's definitely made me feel much more confident a programmer.

This project has taught me a lot about how to approach larger programs. My main function is very messy, and I've absolutely learnt to appreciate the 'one function does one thing' ethos. If I were to program this again from the ground up, I would try to break things down more, into more manageable functions. I would also write more comments as I went!

In conclusion, this project has been a success! I've achieved almost all of my objectives, and made a very nice piece of maze software, which, if I applied some of the aforementioned changes to the generation algorithm, would be super competitive with other maze generator programs.

Code Appendix:

Amazing Maze Generator:

Maze Generator Main:

```
def main(mazeWidth, mazeHeight, window, xCoord, yCoord):  
    '''  
    Takes width and height of maze, window to draw maze on  
    and x,y coordinates of maze's top left corner as parameters.  
    Generates a maze, drawing the maze to the specified window  
    after each iteration of the generating process.  
    Returns matrix representing maze  
    '''  
  
    # generate a matrix of specified width and height, entirely consisting of '-1's  
    # these will be replaced shortly  
  
    mazeMatrix = np.array([[ -1 for x in range(mazeWidth)] for y in range(mazeHeight)])  
  
    # replace all '-1's with a randomly generated 1 or 0  
  
    for y, x in np.argwhere(mazeMatrix == -1):  
        mazeMatrix[y][x] = random.randint(0, 1)  
  
    # stores the previous iteration of the maze, to later check if it is equal to the current iteration  
    lastMaze = []  
  
    run = True  
  
    while run == True:  
        drawMaze(window, mazeMatrix, xCoord, yCoord)  
        pygame.display.update()  
  
        mazeMatrix = openContainedSpace(mazeMatrix, mazeWidth, mazeHeight)  
        mazeMatrix = eliminateTwoBlockCorridors(mazeMatrix, mazeWidth, mazeHeight)  
        mazeMatrix = eliminateTwoBlockWalls(mazeMatrix, mazeWidth, mazeHeight)  
        mazeMatrix = eliminateDiagonals(mazeMatrix)  
        mazeMatrix = removeSinglePixels(mazeMatrix)  
  
        # add in entrance, exit and outer walls  
  
        for i in range(0, mazeWidth):  
            mazeMatrix[0][i] = 1  
            mazeMatrix[mazeHeight-1][i] = 1  
        for i in range(0, mazeHeight):  
            mazeMatrix[i][0] = 1  
            mazeMatrix[i][mazeWidth-1] = 1  
  
        mazeMatrix[0][1] = 9  
        mazeMatrix[mazeHeight-1][mazeWidth-2] = 8  
        mazeMatrix[mazeHeight-2][mazeWidth-2] = 0  
        mazeMatrix[mazeHeight-3][mazeWidth-2] = 0  
  
        for event in pygame.event.get():  
            if event.type == pygame.QUIT:  
                pygame.quit()  
                sys.exit()
```

```
# stop running the generator if the maze has stopped changing and path exists from start to finish
mazeCopy = mazeMatrix.copy()
if np.array_equal(lastMaze, mazeMatrix):

    _, connected = connectedEntranceCheck(mazeCopy, [mazeHeight-2, mazeWidth-2])

    if connected == True:
        run = False

lastMaze = mazeMatrix.copy()

return mazeMatrix
```

RemoveSinglePixels:

```
def removeSinglePixels(maze):
    '''
    Takes maze as argument and returns it
    For each wall in the maze that has no other walls orthogonally adjacent, add a wall.
    '''

    for y, x in np.argwhere(maze == 1):

        try:

            # if pixel to the right = 0 and pixel below = 0
            # and pixel to the left = 0 and pixel above = 0

            if maze[y][x+1] == 0 and maze[y+1][x] == 0 \
                and maze[y][x-1] == 0 and maze[y-1][x] == 0:

                # set pixel above to 1
                maze[y-1][x] = 1

        except IndexError:
            pass # index error expected at the end of each row

    return maze
```

EliminateTwoBlockWalls:

```
def eliminateTwoBlockWalls(maze, mazeWidth, mazeHeight):  
    '''  
    Takes maze as argument and returns it  
    For each 2x2 block of wall, set the bottom right pixel to corridor  
    '''  
  
    for y, x in np.argwhere(maze == 1):  
        if x > 0 and y > 0:  
            # if not out of bounds and pixel to the right = 1  
            # and pixel below = 1 and pixel to the right and below = 1  
            if y+1 < mazeHeight and x+1 < mazeWidth and maze[y][x+1] == 1 \  
                and maze[y+1][x] == 1 and maze[y+1][x+1] == 1:  
                # set pixel to the right and below to 0  
                maze[y+1][x+1] = 0  
  
    return maze
```

EliminateTwoBlockCorridors:

```
def eliminateTwoBlockCorridors(maze, mazeWidth, mazeHeight):  
    '''  
    Takes maze as argument and returns it  
    For each 2x2 block of corridor, set the bottom right pixel to wall  
    '''  
  
    for y, x in np.argwhere(maze == 0):  
        # if not out of bounds and pixel to the right = 0  
        # and pixel below = 0 and pixel to the right and below = 0  
        if y+1 < mazeHeight and x+1 < mazeWidth and maze[y][x+1] == 0 \  
            and maze[y+1][x] == 0 and maze[y+1][x+1] == 0:  
            choice = random.randint(1, 2)  
            if choice == 1:  
                # set pixel to the lower right to 1  
                maze[y+1][x+1] = 1  
            elif choice == 2:  
                # set pixel (upper left of 2x2) to 1  
                maze[y][x] = 1  
  
    return maze
```

EliminateDiagonals:

```
def eliminateDiagonals(maze):
    '''
    Each square of wall in the maze checks if the square to the top right
    or the square to the bottom right is purely diagonally connected to it.
    If this is found to be the case, it adds a wall immediately to the right of itself.
    '''

    for y, x in np.argwhere(maze == 1):
        try:

            # square to the upper right is purely diagonally connected
            if maze[y-1][x+1] == 1 and maze[y][x+1] == 0 and maze[y-1][x] == 0:

                # set square immediately to the right to a 1
                maze[y][x+1] = 1

            # square to the lower right is purely diagonally connected
            if maze[y+1][x+1] == 1 and maze[y][x+1] == 0 and maze[y+1][x] == 0:

                # set square immediately to the right to a 1
                maze[y][x+1] = 1

        except IndexError:
            pass # index error expected at the end of each row

    return maze
```

OpenContainedSpace:

```
def openContainedSpace(maze, mazeWidth, mazeHeight):
    '''For each unmarked space in the maze, checks if a path exists to the maze entrance'''

    shadowMaze = maze.copy()

    for x, y in np.argwhere(maze == 0):
        shadowMaze, connected = connectedEntranceCheck(shadowMaze, [x, y])

        if connected == False:

            blockToLose = random.randint(1, 4)
            if x+1 < mazeHeight and blockToLose == 1 and maze[x+1][y] == 1:
                maze[x+1][y] = 0
            elif y+1 < mazeWidth and blockToLose == 2 and maze[x][y+1] == 1:
                maze[x][y+1] = 0
            elif x-1 > -1 and blockToLose == 3 and maze[x-1][y] == 1:
                maze[x-1][y] = 0
            elif y-1 > -1 and blockToLose == 4 and maze[x][y-1] == 1:
                maze[x][y-1] = 0

    return maze
```

ConnectedEntranceCheck:

```
def connectedEntranceCheck(maze, startingCoords):  
    '''  
    Takes a maze and a coordinate pair as parameters.  
    Checks if the coordinate is connected to the entrance of the maze.  
    Returns a version of the maze with 2s for connected points and 4s for non connected points,  
    to make running quicker for subsequent points.  
    Also returns a boolean that states whether the given point is connected to the entrance.  
    '''  
  
    # list which holds the IDs of all active node objects  
    activeNodes = []  
  
    # temporary list that is used when creating new active node objects  
    # needed so that the new active nodes are not considered until next iteration  
    newActiveNodes = []  
  
    # create an active node at the starting point specified by the parameter  
    newNode = ActiveNode(startingCoords[0], startingCoords[1], False)  
    activeNodes.append(newNode)  
  
    connected = False    # not yet known if node is connected to start of maze  
  
    run = True  
  
    while run == True:  
        # if run out of activeNodes without finding the entrance, point is not connected  
        # mark unconnected pixels with a 4  
  
        if activeNodes == []:  
            for x, y in np.argwhere(maze == 3):  
                maze[x][y] = 4  
  
            run = False  
  
        else:  
            for node in activeNodes:  
                # each active node explores the area around it  
                # it marks its current location with a 3,  
                # before using findNewNodes function to find adjacent valid movements  
  
                # marking path taken happens when active node is added to list  
                maze[node.XValue, node.YValue] = 3  
  
                # find valid spaces to move to  
  
                validMovements, connected, _ = node.findNewNodes(maze)
```

```
# if entrance found, we know that all points currently marked as 3 are connected
# mark connected points with a 2

if connected == True:
    for x, y in np.argwhere(maze == 3):
        maze[x][y] = 2

    run = False

else:

    # move from one node to another
    # add all nodes that are in validMovements list to newActiveNode list

    for coordPair in validMovements:
        newNode = ActiveNode(coordPair[0], coordPair[1], node)
        newActiveNodes.append(newNode)

# now that we are done iterating through the activeNodes list,
# we can clear it of all current nodes and add the next lot

# remove all items from activeNodes
activeNodes = []

# add items from newActiveNodes to activeNodes
for item in newActiveNodes:
    activeNodes.append(item)

# clear newActiveNodes
newActiveNodes = []

return maze, connected
```

DrawMaze:

```
def drawMaze(window, maze, xCoord, yCoord, whiteBox = True):  
    '''  
    Parameters are window to draw maze on, maze itself and coordinates where top left corner should go  
    Draws a white rectangle behind the maze, then draws a coloured square for each number in the maze array  
    '''  
  
    if whiteBox == True:  
        # get width and height of maze for white rectangle  
        # times both by 10, as each unit in the maze represents 10 pixels squared  
  
        # len of the maze returns the number of columns  
        mazeHeight = len(maze)*10  
  
        # maze should be square, so each column has the same number of pixels in it  
        # find the number of pixels in the first column, add 10 to mazeWidth for each and then break  
  
        mazeWidth = 0  
  
        for column in maze:  
            for item in column:  
                mazeWidth += 10    # add 10 pixels to width  
  
            break    # break, as all subsequent columns will have the same number of pixels  
  
        # draw white rectangle to make sure maze doesn't clash with existing graphics  
        pygame.draw.rect(window, WHITE, (xCoord, yCoord, mazeWidth, mazeHeight))  
  
    #draw a black pixels for walls  
  
    for y, x in np.argwhere(maze == 1):  
        pygame.draw.rect(window, BLACK, (x*10 + xCoord, y*10 + yCoord, 10, 10))  
  
    # draw green pixel at start  
  
    for y, x in np.argwhere(maze == 9):  
        pygame.draw.rect(window, GREEN, (x*10 + xCoord, y*10 + yCoord, 10, 10))  
  
    # draw red pixel at finish  
  
    for y, x in np.argwhere(maze == 8):  
        pygame.draw.rect(window, RED, (x*10 + xCoord, y*10 + yCoord, 10, 10))  
  
    # draw red pixels along shortest path  
  
    for y, x in np.argwhere(maze == 7):  
        pygame.draw.rect(window, RED, (x*10 + xCoord, y*10 + yCoord, 10, 10))
```

Maze Solver:

Maze Solver Main:

```
def main(maze):

    startingCoords = findStart(maze)

    # list which holds the IDs of all active node objects
    activeNodes = []

    # temporary list that is used when creating new active node objects
    newActiveNodes = []

    # get the starting node of the maze

    newNode = ActiveNode(startingCoords[0], startingCoords[1], False)
    activeNodes.append(newNode)

    run = True

    while run == True:
        for node in activeNodes:

            # mark path taken

            maze[node.XValue, node.YValue] = 3

            # find valid spaces to move to

            validMovements, _, foundExit = node.findNewNodes(maze)

            # if exit has been found, we can stop exploring the maze
            # and start looking for the shortest path

            if foundExit == True:
                run = False
                return node

            # move from one space to another
            # add all spaces that are in validMovements list to newActiveNode list

            for coordPair in validMovements:
                newNode = ActiveNode(coordPair[0], coordPair[1], node)
                newActiveNodes.append(newNode)
```

```
# now that we are done iterating through the activeNodes list,
# we can clear it of all current nodes and add the next lot

# remove all items from activeNodes

activeNodes = []

# add items from newActiveNodes to activeNodes

for item in newActiveNodes:
    activeNodes.append(item)

# clear newActiveNodes

newActiveNodes = []

# allows the user to quit the window while the program is running

for event in pygame.event.get():
    if event.type == pygame.QUIT:
        pygame.quit()
        sys.exit()
```

FindRoute:

```
# trace back the path from the node that finished first

def findRoute(finishNode, maze, window):
    run = True
    finishPath = []

    while run == True:

        # add the current node to the path taken
        # then, find the node that that node originated from and repeat

        finishPath.append(finishNode)
        finishNode = finishNode.parentNode

        # when the parent node = False, reached the start of the maze

        if finishNode == False:
            run = False

    # get rid of all the 3s in the maze so that just the finish path is shown

    for x, y in np.argwhere(maze == 3):
        maze[x][y] = 0

    return maze, finishPath
```

FindStart:

```
def findStart(maze):

    # First, find the start of the maze - marked by a 9
    # numStarts variable holds how many 9s have been found

    numStarts = 0

    for row, column in np.argwhere(maze == 9):
        numStarts += 1
        startNode = [row+1, column]

    # cannot have more than one starting point
    # if numStarts is >1, maze is invalid

    if numStarts > 1:
        print("Error - maze has too many entrances")

    return startNode
```

GetWidth, GetHeight:

```
def getWidth(maze):  
    '''  
    Returns the width of a maze in pixels - every unit of the maze is 10 pixels  
    '''  
  
    width = 0  
    for i in maze[0]:  
        width += 10  
    return width  
  
def getHeight(maze):  
    '''  
    Returns the height of a maze in pixels - every unit of the maze is 10 pixels  
    '''  
  
    height = len(maze)*10  
  
    return height
```

ActiveNode Class:

```
class ActiveNode():

    def __init__(self, XValue, YValue, parentNode):
        '''
        Initialise each instance of node object
        Takes parameters XValue, YValue and parentNode
        XValue and YValue refer to the node's position in the maze
        parentNode is the node that the node originated from
        parentNode = False for the starting node in the maze
        '''

        self.XValue = XValue
        self.YValue = YValue
        self.parentNode = parentNode

    def findNewNodes(self, maze):
        '''
        Given a node in the maze, find adjacent nodes that can be moved to.
        Returns a list of all valid movements,
        as well as a boolean that holds whether the exit has been found.
        '''

        # locate an adjacent space to move to
        # start by looking at all possible movements

        possibleMovements = [[self.XValue - 1, self.YValue], [self.XValue + 1, self.YValue]\
                               , [self.XValue, self.YValue - 1], [self.XValue, self.YValue + 1]]
        validMovements = []

        foundExit = False    # not yet found the exit to the maze
        foundStart = False   # not yet found the start of the maze

        for row, column in possibleMovements:
            try:

                if maze[row][column] == 8:
                    foundExit = True    # found finish!

                elif maze[row][column] == 9:
                    foundStart = True

                elif maze[row][column] == 2:
                    foundStart = True    # spaces connected to the start are marked with a 2

                elif maze[row][column] == 0:
                    validMovements.append([row, column])    # add each valid movement to a list
                    maze[row][column] = 3

            except IndexError: # index error expected when at the edge of the maze
                pass

        return validMovements, foundStart, foundExit
```

Maze Exploration:

Stack Class:

```
class Stack():
    def __init__(self): # initialise the stack and the location of the top pointer
        self.stack = []
        self.topOfStackPointer = -1

    def __str__(self): # return the contents of the stack if it is printed
        return str(self.stack)

    def push(self, item): # add an item to the stack and increment the top pointer
        self.stack.append(item)
        self.topOfStackPointer += 1

    def pop(self): # remove the item closest to the top of the stack and return it
        # also decrease top of stack pointer by 1
        if self.topOfStackPointer > -1:

            data = self.stack[self.topOfStackPointer]
            del self.stack[-1]
            self.topOfStackPointer -= 1

        else: # if the stack pointer = -1, the stack is empty
            print("No item to pop")
            data = False

        return data

    def peek(self): # return the item at the top of the stack without removing it
        if self.topOfStackPointer > -1:
            data = self.stack[self.topOfStackPointer]
        else:
            data = False # if stack empty, return false

        return data
```

HandleUserExploration:

```
def handleUserExploration(maze, exploreCoords, pastExploreList, exploreDirectionStack):  
  
    if event.type == pygame.KEYDOWN:  
        if event.key == pygame.K_UP:  
            # user tries to move upwards  
  
            # move is valid  
            if maze[exploreCoords[1]-1][exploreCoords[0]] != 1 and exploreCoords[1]-1 >= 1:  
  
                # if last movement was down, undo previous move  
                if exploreDirectionStack.peek() == "down":  
                    exploreDirectionStack.pop()  
  
                exploreCoords = [exploreCoords[0], exploreCoords[1]-1]  
                pastExploreList.remove(exploreCoords)  
  
            # else, add new movement to stack and change exploreCoords  
            else:  
                pastExploreList.append(exploreCoords)  
                exploreDirectionStack.push("up")  
  
                exploreCoords = [exploreCoords[0], exploreCoords[1]-1]  
  
        if event.key == pygame.K_DOWN:  
            # user tries to move downwards  
  
            # move is valid  
            if maze[exploreCoords[1]+1][exploreCoords[0]] != 1 and exploreCoords[1]+1 < mazeHeight:  
  
                # if last movement was up, undo previous move  
                if exploreDirectionStack.peek() == "up":  
                    exploreDirectionStack.pop()  
  
                exploreCoords = [exploreCoords[0], exploreCoords[1]+1]  
                pastExploreList.remove(exploreCoords)  
  
            # else, add new movement to stack and change exploreCoords  
            else:  
                pastExploreList.append(exploreCoords)  
                exploreDirectionStack.push("down")  
  
                exploreCoords = [exploreCoords[0], exploreCoords[1]+1]
```

```
if event.key == pygame.K_LEFT:
    # user tries to move left

    if maze[exploreCoords[1]][exploreCoords[0]-1] != 1: # move is valid

        # if last movement was right, undo previous move
        if exploreDirectionStack.peek() == "right":
            exploreDirectionStack.pop()

            exploreCoords = [exploreCoords[0]-1, exploreCoords[1]]
            pastExploreList.remove(exploreCoords)

        # else, add new movement to stack and change exploreCoords
        else:
            pastExploreList.append(exploreCoords)
            exploreDirectionStack.push("left")

            exploreCoords = [exploreCoords[0]-1, exploreCoords[1]]

if event.key == pygame.K_RIGHT:
    # user tries to move right

    if maze[exploreCoords[1]][exploreCoords[0]+1] != 1: # move is valid

        # if last movement was left, undo previous move
        if exploreDirectionStack.peek() == "left":
            exploreDirectionStack.pop()

            exploreCoords = [exploreCoords[0]+1, exploreCoords[1]]
            pastExploreList.remove(exploreCoords)

        # else, add new movement to stack and change exploreCoords
        else:
            pastExploreList.append(exploreCoords)
            exploreDirectionStack.push("right")

            exploreCoords = [exploreCoords[0]+1, exploreCoords[1]]

return maze, exploreCoords, pastExploreList, exploreDirectionStack
```

Slider Class:

```
class Slider():
    def __init__(self, XCoord, YCoord, lowerBound, upperBound, text):
        self.XCoord = XCoord
        self.YCoord = YCoord
        self.lowerBound = lowerBound
        self.upperBound = upperBound
        self.currentValue = lowerBound
        self.sliderWidth = 150
        self.sliderHeight = 50
        self.text = text
        self.selected = False

    def drawSlider(self, window):
        # write whatever the user has named the slider,
        # followed by the slider's current value at a point centered above the slider
        drawText(window, (self.text + " : " + str(self.currentValue)), \
            self.XCoord + self.sliderWidth//2 + 10, self.YCoord - 10)

        # get the X-Coordinate of the moveable part of the slider
        sliderXPosition = self.XCoord + ((self.currentValue-self.lowerBound)\
            /(self.upperBound-self.lowerBound))*self.sliderWidth

        # draw the slider bar
        pygame.draw.line(window, BLACK, (self.XCoord, self.YCoord + self.sliderHeight//2), \
            (self.XCoord + self.sliderWidth - 1 + 20, self.YCoord + self.sliderHeight//2), 5)

        # if slider being moved, draw the movable part of the slider in a different colour
        if self.selected == True:
            sliderRect = pygame.draw.rect(window, YELLOW, (sliderXPosition, \
                self.YCoord + self.sliderHeight//2 - 10, 20, 20), 5)
        else:
            # draw the movable part of the slider
            sliderRect = pygame.draw.rect(window, BLACK, (sliderXPosition, \
                self.YCoord + self.sliderHeight//2 - 10, 20, 20), 5)

        # draw the little white box inside the movable part of the slider
        pygame.draw.rect(window, WHITE, (sliderXPosition+5, self.YCoord \
            + self.sliderHeight//2 - 5, 10, 10))

        return sliderRect, sliderXPosition
```

```
def handleSlider(self, window, mouseX, mouseY, clicked):  
  
    # call the drawslider function  
    sliderRect, sliderXPosition = self.drawSlider(window)  
  
    # if mouse button down and mouse over slider button, slider is selected  
    if clicked == True:  
        if sliderRect.collidepoint(mouseX, mouseY) == True:  
  
            self.selected = True  
  
        else:  
            self.selected = False  
  
    # if slider is selected and mouse is to the right of slider button:  
    # increase number on slider, thus moving it right  
  
    if self.selected == True and mouseX >= sliderXPosition:  
        while mouseX > sliderXPosition + 10 and self.currentValue < self.upperBound:  
  
            self.currentValue += 1  
  
            # call drawSlider again to show the changes to the movable part  
            sliderRect, sliderXPosition = self.drawSlider(window)  
  
    # if slider is selected and mouse is to the left of slider button:  
    # decrease number on slider, moving it left  
  
    if self.selected == True and mouseX < sliderXPosition:  
        while mouseX < sliderXPosition and self.currentValue > self.lowerBound:  
            self.currentValue -= 1  
  
            sliderRect, sliderXPosition = self.drawSlider(window)
```

User Interface:

DrawText:

```
def drawText(text, XCenter, YCenter, small = False):
    '''Input text, X coordinate and Y coordinate, draw text to screen with inputted center.
    'small' parameter defaults to false, but allows for a smaller font size when set to true.'
    Returns the textbox.'''

    if small == True: # set fontsize based on small parameter
        font = pygame.font.SysFont("freesansbold", 36)

    else:
        font = pygame.font.SysFont("freesansbold", 50)

    textSurface = font.render(str(text), True, BLACK, WHITE)

    # create a rectangle containing the text box
    textRectangle = textSurface.get_rect()

    # set the center of the rectangle to the desired coordinate
    textRectangle.center = (XCenter, YCenter)

    # draw the text to the screen at the location of the rectangle
    window.blit(textSurface, textRectangle)

    return textRectangle
```

GetWidthHeight:

```
def getWidthHeight(maze):

    '''
    Takes maze as input.
    Returns width and height of maze.
    '''

    # len of the maze returns the number of rows
    mazeHeight = len(maze)

    # maze should be square, so each row has the same number of pixels in it

    mazeWidth = 0

    for row in maze:
        for item in row:
            mazeWidth += 1    # add 1 pixels to width

        break    # break, as all subsequent rows will have the same number of pixels

    return mazeWidth, mazeHeight
```

(note: comment says 'maze should be square', should be rectangular)

UpdateWindow:

```
def updateWindow(fillWhite, buttonUpdate, mazeUpdate, optionsWindow, savedMazeWindow, mouseX, \
mouseY, clicked, solveMaze, exploreMaze, exploreCoords, pastExploreList, mazeTopLeft, saveSlotChoice):

    if fillWhite == True:

        # completely fill the screen with white
        window.fill(WHITE)

    if exploreMaze == True:

        # draw in blue the nodes user has visited in the past
        for coordinates in pastExploreList:
            pygame.draw.rect(window, BLUE, (mazeTopLeft[0] + \
                coordinates[0]*10, mazeTopLeft[1] + coordinates[1]*10, 10, 10))

        # draw in red the user's current location
        pygame.draw.rect(window, RED, (mazeTopLeft[0] + \
            exploreCoords[0]*10, mazeTopLeft[1] + exploreCoords[1]*10, 10, 10))

    if mazeCreated == True and mazeUpdate == True:

        # redraw the maze to the screen
        updateMaze()

    if optionsWindow == True:

        # draw the options window under generate maze button
        drawGenerationOptionsWindow(window, mouseX, mouseY, clicked)

    if savedMazeWindow == True:

        # draw options window under saved mazes button, returns rectangles for the three mini buttons
        maze1Button, maze2Button, maze3Button = drawSavedMazeWindow(window, mouseX, mouseY, saveSlotChoice)

    else:

        # buttons still have to exist if the window isn't open, so just make them unclickable
        maze1Button = pygame.Rect(0, 0, 0, 0)
        maze2Button = pygame.Rect(0, 0, 0, 0)
        maze3Button = pygame.Rect(0, 0, 0, 0)

    if buttonUpdate == True:

        # draw the buttons to the screen and get a list of their rectangles
        rectList = drawButtons()

    if exploreMaze == True:

        # draw blue rectangle around explore maze button if it is active
        pygame.draw.rect(window, BLUE, (800, 0, 200, 100), 10)

    if solveMaze == True:

        # draw blue rectangle around solve maze button if it is active
        pygame.draw.rect(window, BLUE, (600, 0, 200, 100), 10)

    pygame.display.update()

    return rectList, maze1Button, maze2Button, maze3Button
```

DrawGenerationOptionsWindow:

```
def drawGenerationOptionsWindow(window, mouseX, mouseY, clicked):
    pygame.draw.rect(window, WHITE, (400, 90, 200, 350))
    pygame.draw.rect(window, BLACK, (400, 90, 200, 350), 10)

    heightSlider.handleSlider(window, mouseX, mouseY, clicked)
    widthSlider.handleSlider(window, mouseX, mouseY, clicked)
```

DrawSavedMazeWindow:

```
def drawSavedMazeWindow(window, mouseX, mouseY, saveSlotChoice):

    # draw a white box with a black border to make up the window
    pygame.draw.rect(window, WHITE, (1000, 90, 200, 350))
    pygame.draw.rect(window, BLACK, (1000, 90, 200, 350), 10)

    # draw the three buttons in the window, and get their rect objects
    maze1Button = pygame.draw.rect(window, BLACK, (1030, 130, 140, 60), 7)
    maze2Button = pygame.draw.rect(window, BLACK, (1030, 230, 140, 60), 7)
    maze3Button = pygame.draw.rect(window, BLACK, (1030, 330, 140, 60), 7)

    # draw yellow rectangles around the buttons if they are being hovered over
    if maze1Button.collidepoint(mouseX, mouseY):
        pygame.draw.rect(window, YELLOW, (1030, 130, 140, 60), 7)

    elif maze2Button.collidepoint(mouseX, mouseY):
        pygame.draw.rect(window, YELLOW, (1030, 230, 140, 60), 7)

    elif maze3Button.collidepoint(mouseX, mouseY):
        pygame.draw.rect(window, YELLOW, (1030, 330, 140, 60), 7)

    # when no slot selected, give the user the choice of all 3 slots
    # draw according text
    if saveSlotChoice == False:
        drawText("Maze 1", 1100, 160, True)
        drawText("Maze 2", 1100, 260, True)
        drawText("Maze 3", 1100, 360, True)

    else:

        # when a slot is selected, display the slot number on first button
        # then 'save' and 'load' on 2 lower buttons
        drawText(str("Maze " + str(saveSlotChoice)), 1100, 160, True)
        drawText("Save", 1100, 260, True)
        drawText("Load", 1100, 360, True)
        pygame.draw.rect(window, BLUE, (1030, 130, 140, 60), 7)

    return maze1Button, maze2Button, maze3Button
```

DrawButtons:

```
def drawButtons():
    '''
    Draws the top row of buttons to the screen.
    Changes colour of buttons when mouse is hovering over them.
    Returns a list of 4 rectangles representing the buttons.
    '''

    # write all the labels on the buttons

    drawText("Menu ->", 200, 50)
    drawText("Generate", 500, 28)
    drawText("Maze", 500, 70)
    drawText("Solve", 700, 28)
    drawText("Maze", 700, 70)
    drawText("Explore", 900, 28)
    drawText("Maze", 900, 70)
    drawText("Saved", 1100, 28)
    drawText("Mazes", 1100, 70)

    # draw black box around 'menu' label
    # don't need the rectangle, as isn't a pressable button
    pygame.draw.rect(window, BLACK, (0, 0, 400, 100), 10)

    # rectangle around 'generate maze' label
    drawMazeRect = pygame.draw.rect(window, BLACK, (400, 0, 200, 100), 10)

    # 'solve maze'
    solveMazeRect = pygame.draw.rect(window, BLACK, (600, 0, 200, 100), 10)

    # 'explore maze'
    exploreMazeRect = pygame.draw.rect(window, BLACK, (800, 0, 200, 100), 10)

    # 'saved mazes'
    fileRect = pygame.draw.rect(window, BLACK, (1000, 0, 200, 100), 10)

    # list of the rectangles just created
    rectList = [drawMazeRect, solveMazeRect, exploreMazeRect, fileRect]

    # check if the mouse collides with any of the buttons
    # if it does, draw a yellow rectangle over the top
    for rect in rectList:
        if rect.collidepoint(mouseX, mouseY) == True:
            pygame.draw.rect(window, YELLOW, (rect), 10)

    return rectList
```

UpdateMaze:

```
def updateMaze():  
  
    # centres the maze on the screen using screen width and height, as well as maze width and height  
    mazeGenerator.drawMaze(window, maze, ((windowWidth-(mazeWidth*10))/2), (((windowHeight + 100) - (mazeHeight*10))/2), False)
```

CleanMaze:

```
def cleanMaze(maze):  
    for y, x in np.argwhere(maze):  
  
        if maze[y][x] == 0 or maze[y][x] == 1 or maze[y][x] == 9 or maze[y][x] == 8:  
            pass  
        else:  
            maze[y][x] = 0  
  
    return maze
```

Setup:

```
# import modules, as well as maze generator and solver

import pygame, sys, numpy as np
import Amazing_Maze_Generator as mazeGenerator
import Maze_Solving as mazeSolver
import Slider_Class as slider

pygame.init()

# define colours, font and window

WHITE = [255, 255, 255]
BLACK = [ 0, 0, 0]
YELLOW = [255, 221, 51]
RED = [255, 0, 0]
BLUE = [ 0, 0, 255]

# set up window

windowWidth = 1200
windowHeight = 700

window = pygame.display.set_mode((windowWidth, windowHeight), 0, 32)
pygame.display.set_caption("Amazing Maze Generator")

mouseX = windowWidth // 2
mouseY = windowHeight // 2

mazeWidth = 60
mazeHeight = 60

mazeCreated = False
mazeTopLeft = [0, 1]

exploreMaze = False
exploreCoords = [0, 0]
pastExploreList = []
exploreDirectionStack = Stack()

solveMaze = False

clicked = False

mouseX, mouseY = [0, 0]

widthSlider = slider.Slider(415, 150, 10, 120, "Width")
heightSlider = slider.Slider(415, 300, 10, 60, "Height")

optionsWindow = False
saveSlotChoice = False

savedMazeWindow = False
```

The Main Function:

Opening:

```
run = True
while run == True:

    rectList, maze1Button, maze2Button, maze3Button = updateWindow(True, True, True, \
        optionsWindow, savedMazeWindow, mouseX, mouseY, clicked, solveMaze, exploreMaze, \
        exploreCoords, pastExploreList, mazeTopLeft, saveSlotChoice)

    for event in pygame.event.get():

        if exploreMaze == True:
            maze, exploreCoords, pastExploreList, exploreDirectionStack = \
                handleUserExploration(maze, exploreCoords, pastExploreList, exploreDirectionStack)

        if event.type == pygame.MOUSEMOTION:
            mouseX, mouseY = event.pos

        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()

        if event.type == pygame.MOUSEBUTTONDOWN:
            clicked = True

        elif event.type == pygame.MOUSEBUTTONUP:
            clicked = False
```

Generate Maze Button Pressed:

```
if clicked == True:

    if rectList[0].collidepoint(mouseX, mouseY) == True:

        mouseX, mouseY = 0, 0    # stops accidental double clicking

        optionsWindow = False
        clicked = False

        # clear maze area
        pygame.draw.rect(window, WHITE, (0, 100, windowWidth, windowHeight - 100))

        # draw blue rectangle around generate maze button
        pygame.draw.rect(window, BLUE, (400, 0, 200, 100), 10)

        # get rid of blue rect around solve maze button
        pygame.draw.rect(window, BLACK, (600, 0, 200, 100), 10)

        # get rid of blue rect around explore maze button
        pygame.draw.rect(window, BLACK, (800, 0, 200, 100), 10)

        exploreMaze = False
        solveMaze = False

        # get width and height from sliders
        mazeWidth, mazeHeight = [widthSlider.currentValue, heightSlider.currentValue]

        # work out where to draw the maze from based on its width and height so that it is centred
        mazeTopLeft = [((windowWidth - (mazeWidth * 10)) // 2), ((windowHeight + 100) - (mazeHeight * 10)) // 2]

        # actually generate maze
        maze = mazeGenerator.main(mazeWidth, mazeHeight, window, mazeTopLeft[0], mazeTopLeft[1])
        mazeCreated = True
```

Solve Maze Button Pressed:

```
elif mazeCreated == True and rectList[1].collidepoint(mouseX, mouseY) == True:
    optionsWindow = False
    mouseX, mouseY = 0, 0    # stops accidental double clicking
    clicked = False
    solveMaze = not solveMaze

    if solveMaze == True:
        pygame.draw.rect(window, BLUE, (600, 0, 200, 100), 10) # draw blue box over solve maze button

        # actually call the solving algorithm and get shortest path
        finishNode = mazeSolver.main(maze)
        maze, finishPath = mazeSolver.findRoute(finishNode, maze, window)

        # reveal the shortest path one pixel at a time
        for counter in range(len(finishPath)-1, -1, -1):

            maze[finishPath[counter].XValue][finishPath[counter].YValue] = 7

            updateMaze()
            pygame.display.update()

    # if solveMaze now false, remove all 7s in the maze, hiding the shortest path
    else:
        for y,x in np.argwhere(maze == 7):
            maze[y][x] = 0
```

Explore Maze Button Pressed:

```
elif mazeCreated == True and rectList[2].collidepoint(mouseX, mouseY) == True:

    exploreCoords = [1, 0]
    pastExploreList = []

    exploreMaze = not exploreMaze
```

SavedMazeWindow is True:

Button 1 Pressed:

```
elif savedMazeWindow == True:

    # if top button pressed
    if maze1Button.collidepoint(mouseX, mouseY):
        mouseX, mouseY = 1000, 200    # stops accidental double clicking
        clicked = False

        # if no slot selected, select slot 1
        if saveSlotChoice == False:
            saveSlotChoice = 1

    else:
        # if slot selected, go back to main menu
        saveSlotChoice = False
```

Button 2 Pressed:

```
# if middle button pressed
elif maze2Button.collidepoint(mouseX, mouseY):
    mouseX, mouseY = 1000, 200    # stops accidental double clicking
    clicked = False

    # if no slot selected, select slot 2
    if saveSlotChoice == False:
        saveSlotChoice = 2

    # if save slot selected, button 2 is the save button
    elif mazeCreated == True:

        # save to slot 1
        if saveSlotChoice == 1:
            file = open("Saved_Maze_1.txt", "w")

        # save to slot 2
        elif saveSlotChoice == 2:
            file = open("Saved_Maze_2.txt", "w")

        # save to slot 3
        elif saveSlotChoice == 3:
            file = open("Saved_Maze_3.txt", "w")

        # get rid of all 2s, 3s and 4s in the maze
        mazeToSave = maze.copy()
        mazeToSave = cleanMaze(mazeToSave)

        for line in mazeToSave:

            # write each number in the line to file
            for character in line:
                file.write(str(character))

            # start new line in file
            file.write("\n")

        # done writing, so close file
        file.close()
```

Button 3 Pressed:

```
# if bottom button pressed
elif maze3Button.collidepoint(mouseX, mouseY):
    mouseX, mouseY = 1000, 200    # stops accidental double clicking
    clicked = False

    # if no save slot selected, choose slot 3
    if saveSlotChoice == False:
        saveSlotChoice = 3

    # if save slot selected, button 3 is load maze button
    else:

        # load from slot 1
        if saveSlotChoice == 1:
            file = open("Saved_Maze_1.txt", "r")

        # load from slot 2
        elif saveSlotChoice == 2:
            file = open("Saved_Maze_2.txt", "r")

        # load from slot 3
        elif saveSlotChoice == 3:
            file = open("Saved_Maze_3.txt", "r")

        # sort out some settings
        mazeCreated = True
        solveMaze = False
        exploreMaze = False

        # start with an empty maze
        maze = []

        # lines in the file are a string of numbers, with no characters between
        for line in file:

            # list(line) returns a list of all the characters in the line
            list1 = list(line)

            # remove the newline character from the end of the list
            list1.remove(list1[-1])

            # cast each number from string to integer
            for i in range(len(list1)):
                list1[i] = int(list1[i])

            # append the list to the maze
            maze.append(list1)

        # change maze data type to numpy.array
        maze = np.array(maze)

        # finished reading, so close file
        file.close()

        # get data about maze to draw it to screen
        mazeWidth, mazeHeight = getWidthHeight(maze)
        mazeTopLeft = [((windowWidth-(mazeWidth*10))/2), \
                       (((windowHeight + 100) - (mazeHeight*10))/2)]
```

Handle Generation Options Window:

```
# get rid of the generation options window when mouse over solve maze button
if rectList[1].collidepoint(mouseX, mouseY) == True:
    optionsWindow = False

# draw options window if mouse over draw maze button
elif rectList[0].collidepoint(mouseX, mouseY):
    optionsWindow = True

# if options window already showing, be more lenient about when to hide it
elif optionsWindow == True and mouseX >= 300 and mouseX <= 700 and mouseY <= 440:
    optionsWindow = True

else:
    optionsWindow = False
```

Handle Saved Maze Window (featuring fps Clock):

```
# if window is closed, forget any choice the user made
if savedMazeWindow == False:
    saveSlotChoice = False

# get rid of save maze window when mouse over explore maze button
if rectList[2].collidepoint(mouseX, mouseY):
    savedMazeWindow = False

# show the window if the mouse is hovering over it
if rectList[3].collidepoint(mouseX, mouseY):
    savedMazeWindow = True

# if window already showing, be lenient on when to hide it
elif savedMazeWindow == True and mouseX >= 900 and mouseY <= 440:
    savedMazeWindow = True

else:
    savedMazeWindow = False
```

```
pygame.time.Clock().tick(60)
```

References:

<https://www.hrp.org.uk/hampton-court-palace/whats-on/the-maze> - stats about Hampton court maze

<https://mazesolver.xyz/>

<https://mazesforfun.com/maze-generator>

<https://stackoverflow.com> - programming troubleshooting

<https://www.pygame.org/docs/> - Pygame documentation

<https://www.madebyevan.com/fsm/> - used to make finite state machine diagrams

<https://www.pngall.com/explosion-png/> - explosion image